

Description and Operation of a Program to Numerically Simulate
Large Signal Phase-locked Loop Behavior
Shawn M. Logan
August 5, 2026, v1.0 (*trans_pll* v1.76)
smlogan@ieee.org

1.	Background and Outline.....	2
1.1	Development Background	2
1.2	Outline	2
2.	Program Description and Input and Output Files	3
3.	Detailed Program Operation	4
3.1	Basic Operation: Transient Simulation	4
3.1.1	Graphical Output using Microsoft Excel	11
3.1.2	Graphical Output using Gnuplot and Octave.....	13
3.2	Basic Operation: Phase-locked loop Lock Detection.....	17
3.3	Basic Operation: Phase-locked Loop Jitter Transfer Simulation	18
4.	Phase-locked Loop Responses to Input Phase and Frequency Steps	23
4.1	Impact of Input Phase Steps on PLL Output Phase	23
4.2	Impact of Input Frequency Steps on PLL Output Frequency	27
5.	Impact of a Non-linear Phase Detector on PLL Jitter Transfer	29
6.	Impact of Divider Latency on PLL Jitter Transfer	31
7.	Examples of Spread Spectrum Clocking on Phase-locked Loop Responses.....	31
7.1	Introduction to Spread Spectrum Simulation Capability	31
7.2	Phase Tracking Ability of a 100 MHz, 8.35 MHz Loop Bandwidth Phase-locked Loop to Spread Spectrum Clocking	32
7.3	Phase Tracking Ability of Two 100 MHz Phase-locked Loops to various 33 kHz SSC Amplitudes	37
7.4	Impact of 0/-5000 ppm SSC Frequency on Tracking Ability of a 100 MHz, 8.35 MHz Bandwidth Phase-locked Loop.....	40
8.	Windowed Phase Detector Phase-locked Loop Simulations	42
8.1	Description of Windowed Phase Detector	42
8.2	Windowed Phase Detector Simulations	43
8.2.1	Phase-locked Loop Acquisition Simulation	43
8.2.2	Impact of Initial Clock Phase and initial Control Voltage on Acquisition Time.....	43
8.2.3	Impact of Jitter Amplitude on Jitter Transfer.....	50
9.	Comparison of <i>trans_pll</i> Simulation Result with Measured Data	55
10.	Summary	58
Appendix I	Installation Instructions	59

1. Background and Outline

1.1 Development Background

In the early 1990 time frame, I was faced with analyzing the large signal performance of non-linear quartz based oscillator phase-locked loops. Commercial simulators to model the large-signal behavior of a quartz based phase-locked loop (PLL) in a reasonable compute time were not available. Even today, simulating a non-linear quartz based oscillator phase-locked loop with a behavioral simulator such as MATLAB or a circuit simulator such as Synopsis HSPICE or Cadence Spectre, is time consuming as it requires solving a set of stiff differential equations.

Therefore, I developed a large-signal simulator in the C language to drastically reduce compute times. The software has evolved significantly since that time and simulates the response of frequency synthesis phase-locked loops and clock and data recovery phase locked loops to various stimuli.

The program has been extremely useful for linear and non-linear phase-locked loops. Numerous correlations of its results with small-signal linear analyses and measured data of non-linear loops suggest the software provides an accurate estimate of small-signal and large-signal phase-locked loop performance.

1.2 Outline

Section 2 is an overview of the program and its input and output files. A more detailed description is included in Section 3. Section 4 provides examples of basic simulations. Sections 5, 6, 7, and 8 provide examples of more complex simulations including: the jitter transfer of a non-linear phase detector based phase-locked loop; the impact of feedback divider latency on jitter transfer; the impact of spread-spectrum input signals on the output spectrum of phase-locked loops; and the performance of a windowed phase detector based phase-locked loop. Section 9 contains a comparison of simulator results to the measured results of a non-linear phase-locked loop. Section 10 is a summary of the effort.

2. Program Description and Input and Output Files

The program includes gate level models for phase detectors used in frequency synthesizers and clock and data recovery phase locked loops. The delay times of their internal combinational and flip-flop elements may be adjusted to examine their impact on loop performance.

Input parameters and program settings are placed in an ASCII text file which is parsed by the executable. The file contains parameters chosen for simulation as well as the names of files for an input serial data pattern (in a data recovery based phase-locked loop) and the simulation results. The program performs some checking of the parameters supplied in the input ASCII text file and provides a diagnostic error or warning to indicate an input parameter may be in error or to indicate a particular setting is in use.

Simulation results are placed in one or two output files. Simulation of the acquisition performance of a phase-locked loop produces a single ASCII data file whose columns contain a number of signals. The header of the output file includes the set of parameters used in the simulation based on the parameters contained in the ASCII input file. If a jitter transfer analysis is requested, the program output consists of a single file containing the transfer function of the phase-locked loop, the phase lock times for each jitter frequency and its respective simulation time. The individual time domain simulation files for each jitter transfer frequency are not saved by default.

The phase detector type and its programmable output impedance allow one to choose between a phase detector with a voltage or a charge pump output stage. Supply voltage limits for the output voltage of the phase detector are included to model the fact that the phase detector output may not be capable of exceeding its supply voltage.

Up to a second order programmable loop filter is modeled. The voltage controlled oscillator (VCO) models allow for a non-linear output frequency - control voltage characteristic. The VCO model also includes a parameter to set its modulation bandwidth.

Several analyses and simulation options are provided. In its simplest form, the program simulates the acquisition performance of a phase-locked loop. More complex analyses include large signal jitter transfer, the impact of spread-spectrum signals, the introduction of phase or frequency steps, and an estimate of data recovery jitter tolerance.

The integration timestep is determined by the program or may be specified manually. The timestep used in the output file of transient results is determined by setting the number of printed data points and the total simulation time.

The input parameter TSTART in the ASCII input file may be set to a non-zero value to delay the simulation time when data is written to the output file. The analysis will start at a time of 0 but data will not be included in the output file until it the time equals or exceeds TSTART.

In addition to using TSTART to limit the size of the output file, the input ASCII file includes the maximum number of points in the analysis. Specifically, the numerical integration time can be set to a value much less than the time interval between output time points to accommodate the small integration timestep required relative to the time constant of a set of stiff differential equations. For example, a numerical integration time can be set to 1 ps, and the print time between data points can be set to 1 us.

The comma-separated variable output file may serve as the input to any graphical software that allows one to import a data file in comma-separated variable format. This includes software such as Microsoft Excel, MATLAB, Kaleidagraph, or DeltaGraph as well as numerous other PC, Macintosh, or UNIX charting applications. A Microsoft Excel file template to plot simulator results that is designed for the program's comma-separated variable output file is included in the installation.

Alternately, if Gnuplot[®] [1] or the Octave [2] UNIX software packages are installed and contained in the UNIX executable path, the results will be plotted by either Gnuplot or Octave (specified as the last entry in ASCII input file shown in Figure 2) and saved as Portable Network Graphic (png) files.

3. Detailed Program Operation

3.1 Basic Operation: Transient Simulation

Figure 1 is a block diagram of the phase locked loop model. The parameters for the program are shown in the template for the input ASCII file in Figure 2. This file template may be generated by issuing the UNIX command “*trans_pll ?*” and will create the template file “sample_input_file_v1p76.txt”. Any text file editor may be used to modify the file to reflect the input parameters for a simulation. Also included in the distribution package is the Microsoft Excel file “sample_input_file_v1p76.xlsx” which may also be used to enter the simulation parameters. When saved as a tab delimited text file, the UNIX shell script “convert_spreadsheet_tab_delimited_file_to_trans_pll.sh” may be used to convert the tab delimited file to the format required by *trans_pll*. Also included in the distribution is a second UNIX shell script “convert_trans_pll_input_file_to_tab_delimited.sh” that will convert a *trans_pll* input ASCII file to a tab delimited file for import into the Microsoft Excel file template. Each shell script requires as its single argument the name of the tab delimited file or the name of the *trans_pll* input file. Neither of the two scripts will overwrite the input file. Each script creates a new file whose name contains the name of its input file.

There are many options and hence many entries contained in the input ASCII file. In its simplest form as a frequency synthesis transient simulation tool, many entries are not required. *trans_pll* will indicate if an entry from the input ASCII file is missing or incorrect for the selected analysis or option. Figure 3 provides the content of the input ASCII file used for an example transient simulation of a phase-frequency detector (PFD) and charge-pump based phase-locked loop with a 12 GHz VCO and a feedback divider of 120.

The command line syntax used to execute the program and the output written to the stdio for the example simulation is shown in Figure 4.

Figure 5 illustrates the content of the resulting simulation output file. Also shown in Figure 5 are the signal outputs the program provides. Many other outputs can be provided if a specific parameter is under study. However, this set was chosen as it satisfies the needs of most simulations.

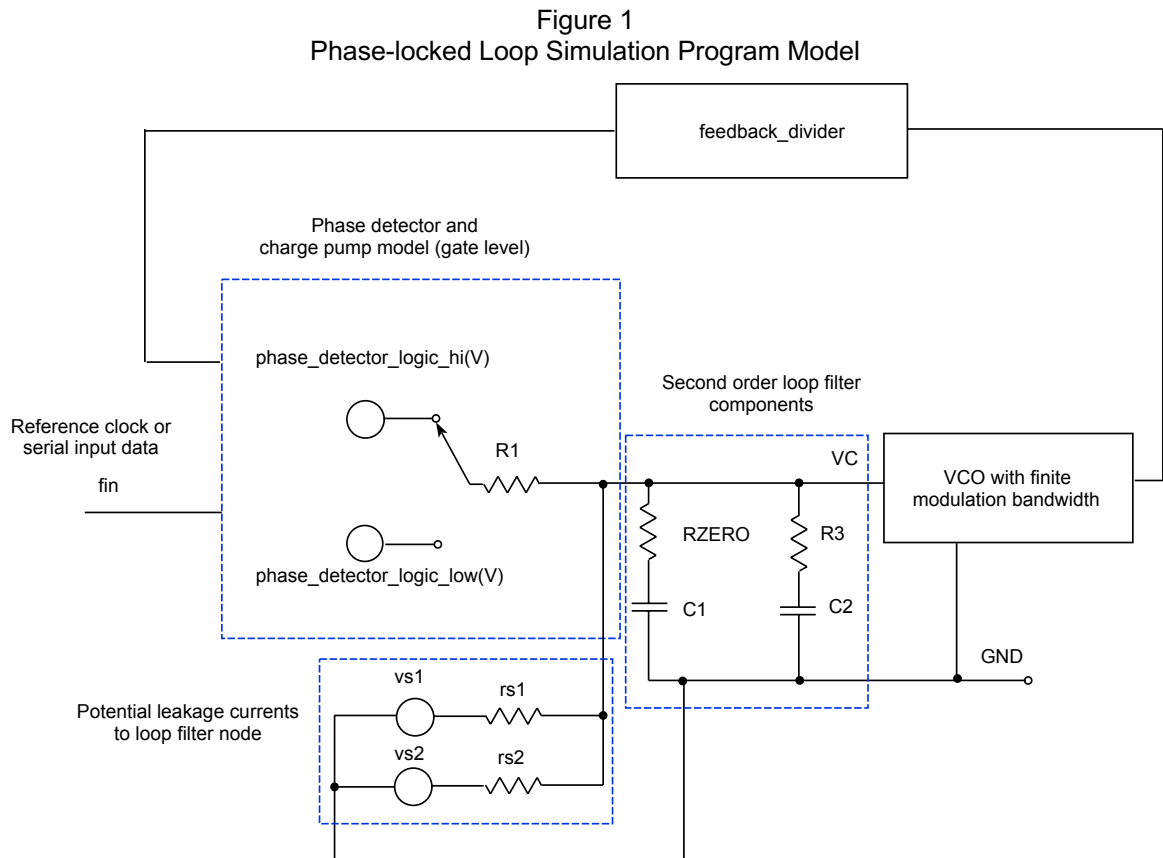


Figure 2
Example of a trans_pll v1.76 Simulation Input File

* Comment lines begin with a "*"
* Phase detector options, choose dff, pfd, pd_windowed, exor, none_vpd=1,
* none_vpd=0, hogge, alexander
*

phase_detector	<Enter your choice for phase detector>
----------------	--

*
* Phase detector related inputs (not all are used for each phase detector)
*

tauff(sec)	<Enter flip-flop delay in sec>
tauh(sec)	<Enter logic gate delay in sec>
taucp_min(sec)	<Enter minimum charge pump width in sec>
pfd_deadzone(ui)	<Enter pfd deadzone width in UI>

*
* pd_windowed phase detector related inputs
*

tw(sec)	<Enter width of window>
tacq_max(sec)	<Enter maximum allowed acquisition time in sec>

*
* CDR phase detector options
*
* Phase threshold beyond which amplitude of Alexander phase detector saturates
*

alexander_phase_detector_phase_threshold(UI)	<Enter positive threshold in UI or the word "default">
--	--

*
* VCO related inputs
*

fo(VCO_freq_in_Hz)	<Enter VCO frequency in Hz>
vco_clock_duty_cycle(fo)	<Enter duty cycle of VCO as a fraction of 1.0 (0.50 => 50%)>

*
* Choose model for control voltage characteristic
* "polynomial" or "poly" allows up to ninth order polynomial $\text{deltaf} = c_0 + c_1*vc + c_2*vc + \dots c_8*vc$
* "tanh" function requires four coefficients $\text{deltaf} = c_0 + c_1*\tanh(c_2*(vc-c_3))$
*

vco_control_voltage_model	<Enter choice for VCO control voltage model>
vc_polynomial_coeff(hz/V)	<Enter VCO control voltage polynomial coefficients as comma-separated list>

*
* Example $k_{vco} = c_1*fo = 2.0*500e6 = 1.0 \text{ GHz/V}$
* $V_{co} = -(c_0/c_1) = -(-1/2) = 0.5 \text{ V}$
*

vco_bandwidth(Hz)	<Enter modulation bandwidth of VCO in Hz>
-------------------	---

*
* Clock or data input parameters
*

fin(input_freq_in_Hz)	<Enter frequency of input clock or data in Hz>
feedback_divider	<Enter feedback divider>
feedback_divider_delay(sec)	<Enter latency of feedback divider (delay) in sec>

*
* Feedback clock duty cycle (not used if divider is 1.0)
*

feedback_divider_clock_duty_cycle	<Enter duty cycle of feedback divided clock as a fraction of 1.0 (0.50 => 50%)>
-----------------------------------	---

*
* Input data pattern (single bit per line, text file)
*

filename_with_input_data_pattern	<Enter filename containing input data pattern>
----------------------------------	--

*
* Enter type of noise on input (gaussian or uniform)
* random_noise_sigma is sigma for gaussian
* noise, and for uniform noise range of [-sigma,+sigma]
*

noise_type	<Enter choice for noise type>
random_noise_sigma(UI_on_fin)	<Enter sigma for input noise in unit intervals (UI)>
noise_bandwidth(Hz)	<Enter bandwidth of input noise in Hz>

*
* Add sinusoidal jitter or perform jitter transfer
*

jitter_amp(UIpp)	<Enter any sinusoidal jitter amplitude or value to use in jitter transfer in UI>
jitter_freqs(Hz)	<Enter comma-separated list of jitter frequencies to use in jitter transfer in Hz>

*
* Initial phase of input
*

phase0(degrees)	<Enter any initial phase offset in degrees>
-----------------	---

Figure 2 (cont.)
Example of a trans_pll v1.76 Simulation Input File

*
* Start and stop times of printed output, sinusoidal input jitter starts at TSTART_jt
*

TSTART(sec)	<Enter start time to begin printing output data in sec>
TSTART_jt(sec)	<Enter start time to apply sinusoidal jitter in jitter transfer analysis in sec>
TSTOP(sec)	<Enter stop time of simulation in sec (overwritten in jitter transfer analysis)>

*
* Enter desired timestep or the word "default"
*

deltat(sec)	<Enter desired timestep in sec>
-------------	---------------------------------

*
* Enter desired lock detector time constant - enter 0 for no lock detector function
*

lock_detect_tau(sec)	<Enter lock detector time constant in sec>
----------------------	--

*
* Enter phase-lock detector thresholds (used if lock detector time constant > 0)
*

lock_detect_delta_vc_limit(%)	<Enter maximum control voltage variation from center voltage (%) or "default">
lock_detect_rphase_pp_limit(UIpp)	<Enter maximum relative phase (UIpp) or "default">
lock_detect_rphase_pp_loss_lock_limit(UIpp)	<Enter relative phase threshold to declare out of lock (UIpp) or "default">

*
* Enter desired number of points to save
*

number_of_print_points	<Enter the number of points to save to output file>
------------------------	---

*
* Inputs for phase or frequency step
*

phase_freq_step_time(sec)	<Enter time to apply frequency or phase step in sec>
phase_step(sec)	<Enter magnitude of phase step in sec>
freq_step(ppm)	<Enter magnitude of frequency step in ppm from input frequency>

*
* Inputs for application of SSC
*

ssc_start_time(sec)	<Enter time to apply spread-spectrum in sec>
max_pos_ssc(ppm)	<Enter maximum positive SSC in ppm>
max_neg_ssc(ppm)	<Enter minimum negative SSC in ppm>
freq_ssc(Hz)	<Enter nominal spread spectrum frequency in Hz>

*
* Phase detector and loop filter inputs
*

phase_detector_logic_hi(V)	<Enter phase detector logic high in V>
phase_detector_logic_low(V)	<Enter phase detector logic low in V>
signal_source_vs1(v)	<Enter optional first leakage current source voltage in V>
signal_source_vs2(v)	<Enter optional second leakage current source voltage in V>
power_supply_max(V)	<Enter maximum possible signal value (limiting) in V>
power_supply_min(V)	<Enter minimum possible signal value (limiting) in V>
rs1(ohms)	<Enter source resistance for optional first leakage current source voltage in ohms>
rs2(ohms)	<Enter source resistance for optional second leakage current source voltage in ohms>
R1(ohms)	<Enter source resistance for phase detector logic high and low values in ohms>
RZERO(ohms)	<Enter loop filter zero resistor in ohms>
R3(ohms)	<Enter any resistance in series with loop filter capacitor in ohms>
C1(farads)	<Enter loop filter capacitor in farads>
C2(farads)	<Enter loop filter ripple capacitor in farads>
initvc(V)	<Enter initial voltage for loop filter capacitor in V>

*
* Output filename and format (csv only)
*

output_base_filename	<Enter base name for output file (no file extension)>
----------------------	---

*
* Optionally plot output characteristics and vco control voltage characteristics.
* Enter value of zero to omit all plots or any non-zero value to create plots.
* Note - plotting requires Octave or gnuplot be installed. The value of plot_outputs
* is forced to zero if one or the other programs is not installed
*

plot_outputs	<Enter 1 or 0 to enable or disable output plots>
--------------	--

*
* Enter preferred plotting tool of "octave" or "gnuplot"
* Plots will be created using preferred plotting tool if
* the executable is found.
*

plot_preference	<Enter choice for plotting tool>
-----------------	----------------------------------

*
* End of trans_pll inputs

Figure 3
trans_pll Input ASCII File "sample_gnuplot_pfd_pll_input_file.txt" used in Simulation Example

```

* Comment lines begin with a *
* Phase detector options, choose dff, pfd, pd_windowed, exor, none_vpd=1,
* none_vpd=0, hogge, alexander
*
phase_detector: pfd
*
* Phase detector related inputs (not all are used for each phase detector)
*
tauff(sec): 1E-11
tauh(sec): 1E-11
taucp_min(sec): 1E-12
pfd_deadzone(ui): 0
*
* pd_windowed phase detector related inputs
*
tw(sec): <Enter width of window>
tacq_max(sec): <Enter maximum allowed acquisition time in sec>
*
* CDR phase detector options
*
* Phase threshold beyond which amplitude of Alexander phase detector saturates
*
alexander_phase_detector_phase_threshold(UI): <Enter positive threshold in UI or the word default>
*
* VCO related inputs
*
fo(VCO_freq_in_Hz): 12e9
vco_clock_duty_cycle(fo): 0.4
*
* Choose model for control voltage characteristic
* polynomial or poly allows up to ninth order polynomial deltaf = c0 + c1*vc +c2*vc +...c8*vc
* tanh function requires four coefficients deltaf = c0 + c1*tanh(c2*(vc-c3))
*
vco_control_voltage_model: poly
vc_polynomial_coeff(hz/V): -0.036984,0.087021
*
* Example kvco = c1*fo = 2.0*500e6 = 1.0 GHz/V
* Vco = -(c0/c1) = -(-1/2) = 0.5 V
*
vco_bandwidth(Hz): 40000000
*
* Clock or data input parameters
*
fin(input_freq_in_Hz): 100000000
feedback_divider: 120
feedback_divider_delay(sec): 0
*
* Feedback clock duty cycle (not used if divider is 1.0)
*
feedback_divider_clock_duty_cycle: 0.1
*
* Input data pattern (single bit per line, text file)
*
filename_with_input_data_pattern: <Enter filename containing input data pattern>
*
* Enter type of noise on input (gaussian or uniform)
* random_noise_sigma is sigma for gaussian
* noise, and for uniform noise range of [-sigma,+sigma]
*
noise_type: <Enter choice for noise type>
random_noise_sigma(UI_on_fin): 0
noise_bandwidth(Hz): <Enter bandwidth of input noise in Hz>
*
* Add sinusoidal jitter or perform jitter transfer
*
jitter_amp(UIpp): 0
jitter_freqs(Hz): 0
*
* Initial phase of input
*
phase0(degrees): 0

```

Figure 3 (cont.)

```

*
* Start and stop times of printed output, sinusoidal input jitter starts at TSTART_jt
*
TSTART(sec): 0
TSTART_jt(sec): 0.000002
TSTOP(sec): 0.000002
*
* Enter desired timestep or the word "default"
*
deltat(sec): 1E-13
*
* Enter desired lock detector time constant - enter 0 for no lock detector function
*
lock_detect_tau(sec): 100e-09
*
* Enter phase-lock detector thresholds (used if lock detector time constant > 0)
*
lock_detect_delta_vc_limit(%): default
lock_detect_rphase_pp_limit(Ulpp): default
lock_detect_rphase_pp_loss_lock_limit(Ulpp): default
*
* Enter desired number of points to save
*
number_of_print_points: 5001
*
* Inputs for phase or frequency step
*
phase_freq_step_time(sec): 1
phase_step(sec): 0
freq_step(ppm): 0
*
* Inputs for application of SSC
*
ssc_start_time(sec): 1
max_pos_ssc(ppm): 0
max_neg_ssc(ppm): 0
freq_ssc(Hz): 0
*
* Phase detector and loop filter inputs
*
phase_detector_logic_hi(V): 600
phase_detector_logic_low(V): -600
signal_source_vs1(v): 0
signal_source_vs2(v): 0
power_supply_max(V): 0.85
power_supply_min(V): 0
rs1(ohms): 1E+12
rs2(ohms): 1E+12
R1(ohms): 1000000
RZERO(ohms): 12000
R3(ohms): 1
C1(farads): 2.407E-11
C2(farads): 3.6E-13
initvc(V): 0.2
*
* Output filename and format (csv only)
*
output_base_filename: sample_gnuplot_output_file_v1p76
*
* Optionally plot output characteristics and vco control voltage characteristics.
* Enter value of zero to omit all plots or any non-zero value to create plots.
* Note - plotting requires Octave or gnuplot be installed. The value of plot_outputs
* is forced to zero if one or the other programs is not installed
*
plot_outputs: 1
*
* Enter preferred plotting tool of octave or gnuplot
* Plots will be created using preferred plotting tool if
* the executable is found.
*
plot_preference: gnuplot
*
* End of trans_pll inputs*
*

```

Figure 4
Example of a *trans_pll* Interactive Simulation Session
(output contained in log file "sample_gnuplot_pfd_pll_input_file_080526_16_05_30.log")

```
trans_pll v1.76 7/18/2026

Input filename: sample_gnuplot_pfd_pll_input_file.txt
Output filename: sample_gnuplot_output_file_v1p76_080526_16_05_30.csv
Phase-frequency detector deadzone = 0.00 UI
Value of deltat is 1.00e-13
Data will be printed every 400.0000 ps.
Sample signals every 1000000 time points to determine lock status (100.000 ns)
Lock detect thresholds:
    Control voltage tolerance about center voltage for phase-lock: 100.0 m%
    Relative phase tolerance for phase-lock: 150.0 mUI
    Relative phase threshold to declare loss of phase-lock: 300.0 mUI
Using polynomial of degree 1 to approximate VCO transfer curve.
Time = 0.000 s   0 % complete, Vc = 200.000 mV
    Averages: vc = 200.000 mV (deltaV = -52.94 %), relative phase = 0.000 UI (0.000 UIpp).
Time = 200.000 ns  10 % complete, Vc = 342.817 mV
    Averages: vc = 433.549 mV (deltaV = 2.01 %), relative phase = -19.548 mUI (7.579
mUIpp).
Time = 400.000 ns  20 % complete, Vc = 385.392 mV
    Averages: vc = 429.341 mV (deltaV = 1.02 %), relative phase = -9.419 mUI (3.841 mUIpp).
Time = 600.000 ns  30 % complete, Vc = 405.962 mV
    Averages: vc = 427.102 mV (deltaV = 494.54 m%), relative phase = -4.521 mUI (1.854
mUIpp).
Time = 800.000 ns  40 % complete, Vc = 415.853 mV
    Averages: vc = 426.006 mV (deltaV = 236.62 m%), relative phase = -2.166 mUI (886.078
uUIpp).
Time = 1000.000 ns  50 % complete, Vc = 420.604 mV
    Averages: vc = 425.484 mV (deltaV = 113.86 m%), relative phase = -1.036 mUI (426.040
uUIpp).
Lock detected at time 1.1000 us.
Time = 1.200 us   60 % complete, Vc = 422.895 mV
    Averages: vc = 425.233 mV (deltaV = 54.94 m%), relative phase = -494.883 uUI (204.564
uUIpp).
Time = 1.400 us   70 % complete, Vc = 423.984 mV
    Averages: vc = 425.108 mV (deltaV = 25.44 m%), relative phase = -245.512 uUI (94.650
uUIpp).
Time = 1.600 us   80 % complete, Vc = 424.332 mV
    Averages: vc = 424.990 mV (deltaV = -2.28 m%), relative phase = -123.390 uUI (44.344
uUIpp).
Time = 1.800 us   90 % complete, Vc = 424.684 mV
    Averages: vc = 425.015 mV (deltaV = 3.58 m%), relative phase = -108.993 uUI (34.399
uUIpp).
Time = 2.000 us  100 % complete, Vc = 424.860 mV
    Averages: vc = 425.047 mV (deltaV = 11.10 m%), relative phase = -96.871 uUI (45.645
uUIpp).
Maximum CLKIN duty cycle = 50.00 % ( $\pm 1.0$  m%), Minimum CLKIN duty cycle = 50.00 % ( $\pm 1.0$  m%)
(difference of 2.0 m%)
Done! Last phase-lock event observed at about 1.1000 us.
Elapsed time: 14.991179 seconds
```

Figure 5
Example of a *trans_pll* Simulation Output File
"sample_gnuplot_output_file_v1p76_080526_16_05_30.csv"

```
phase_detector: pfd
fo(VCO_freq_in_Hz): 1.200000e+10
VCO_clock_dutycycle: 0.4000
tauff(sec): 1.0000e-11
tauh(sec): 1.0000e-11
taucp_min(sec): 1.0000e-12
pfd_deadzone(UI): 0.0000
fin(input_freq_in_Hz): 1.000000e+08
feedback_divider: 120
feedback_divider_delay(sec): 0.0000e+00
feedback_divider_clock_duty_cycle: 0.1000
jitter_amp(UIpp): 0.00 UI
Jitter_freqs(Hz): 0.0000e+00
phase0(degrees): 0.00
TSTART(sec): 0.000000e+00
TSTART_jt(sec): 2.000000e-06
TSTOP(sec): 2.000000e-06
deltat(sec): 1.000000e-13
lock_detect_tau(sec): 1.000000e-07
lock_detect_delta_vc_limit(%): 1.000000e-01
lock_detect_rphase_pp_limit(UIpp): 1.500000e-01
lock_detect_rphase_pp_loss_lock_limit(UIpp): 3.000000e-01
number_of_print_points: 5001
phase_freq_step_time(sec): 1.000000e+00
phase_step(sec): 0.0000e+00
freq_step(ppm): 0.0000e+00
ssc_start_time(sec): 1.000000e+00
max_pos_ssc(ppm): 0.00
max_neg_ssc(ppm): 0.00
freq_ssc(Hz): 0.0000e+00
phase_detector_logic_hi(V): 600.000
phase_detector_logic_low(V): -600.000
signal_source_vs1(V): 0.000
signal_source_vs2(V): 0.000
vc_model: poly
vc_polynomial_coeff.(Hz/V): -3.698400e-02,8.702100e-02
vco_bandwidth(Hz): 4.0000e+07
power_supply_max(V): 0.8500
power_supply_min(V): 0.0000
random_noise_sigma(UI_on_fin): 0.0000
RS1(ohms): 1.0000e+12
RS2(ohms): 1.0000e+12
R1(ohms): 1.0000e+06
RZERO(ohms): 1.2000e+04
R3(ohms): 1.0000e+00
C1(farads): 2.4070e-11
C2(farads): 3.6000e-13
initvc(V): 0.2000
Output_base_filename "sample_gnuplot_output_file_v1p76"
Time(s),vpd(V),vs1(V),vs2(V),vc1(V),vc(V),phasein(UI),phaseo(UI),phaseo_delayed(UI),rphase(UI),vc2(V),VIN,
CLK,VCO_CLK,CLKIN,freq (ppm from fin),VOUT
0.000000000000e+00,-6.000000000000e+02,0.000000000000e+00,0.000000000000e+00,2.000000000000e-01,2.
000000000000e-01,0.000000000000e+00,0.000000000000e+00,0.000000000000e+00,0.000000000000e+00,2.
000000000000e-01,0,1,1,1,-3.698400000000e+04,0
```

Header includes relevant parameters contained in *trans_pll* ASCII input file

Following header rows, the columns contain input and output signals from the transient analysis for $\approx$ *number_of_print_points* rows

3.1.1 Graphical Output using Microsoft Excel

To facilitate quick use of the software, a Microsoft Excel file template "sample_output_file_v1p76.xlsx" is included in the examples directory to provide a means of plotting simulation results. The comma-separated variable file produced by a simulation may be copied and pasted to replace the data in the last workbook tab "data.csv". Workbook tabs "loop_filter_plot", "frequency_plot", and "phase_plot" provide a view of the control voltage and loop filter capacitor voltages, the divided VCO output frequency relative to the input frequency, and the divided VCO output phase, input phase and phase difference for the simulation. The control voltage, output frequency and output phase plots produced by the ASCII output file presented in this note are shown in Figure 6, Figure 7, and Figure 8 respectively. The scales, title, and axis labels on each plot tab may be changed to accommodate the results of any given specific simulation. Excel workbook tab labeled "Variables" contains a common set of parameters used to construct the chart titles.

A limitation of using the Microsoft Excel template is its responsiveness to data sets with many records. Hence, the author has personally chosen to use other software to view simulation results. Nevertheless, the Microsoft Excel template provides a easily customizable view of *trans_pll* simulation results.

Figure 6

Phase-locked Loop Control Voltage and Loop Filter Capacitor Voltage Variation with Time
PFD input frequency = 100.0 MHz, VCO frequency = 12.0 GHz
SSC Range = 0 ppm, SSC frequency = 0 kHz

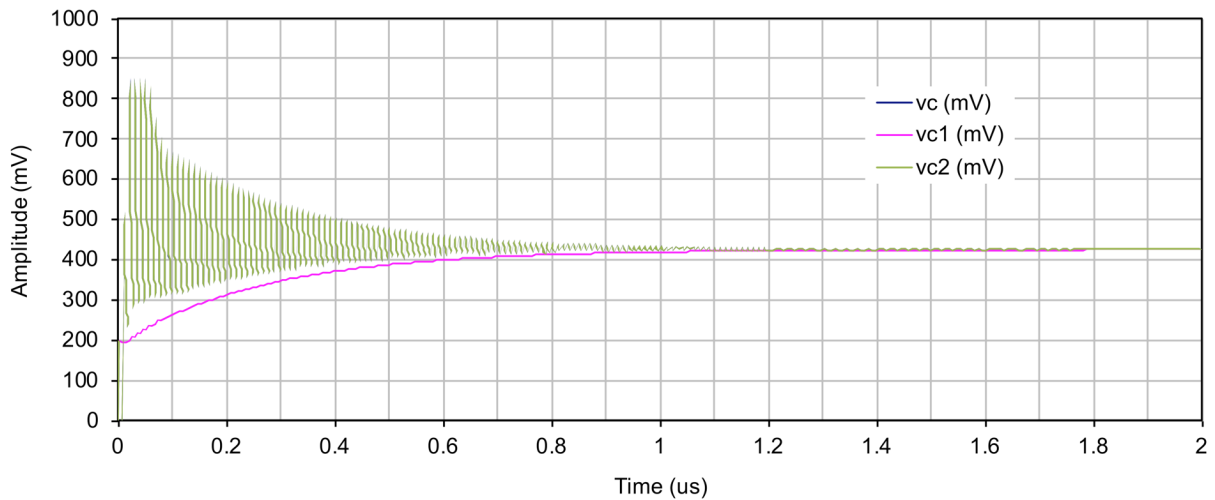


Figure 7

Phase-locked Loop VCO Divided Frequency Variation with Time
PFD input frequency = 100.0 MHz, VCO frequency = 12.0 GHz
SSC Range = 0 ppm, SSC frequency = 0 kHz

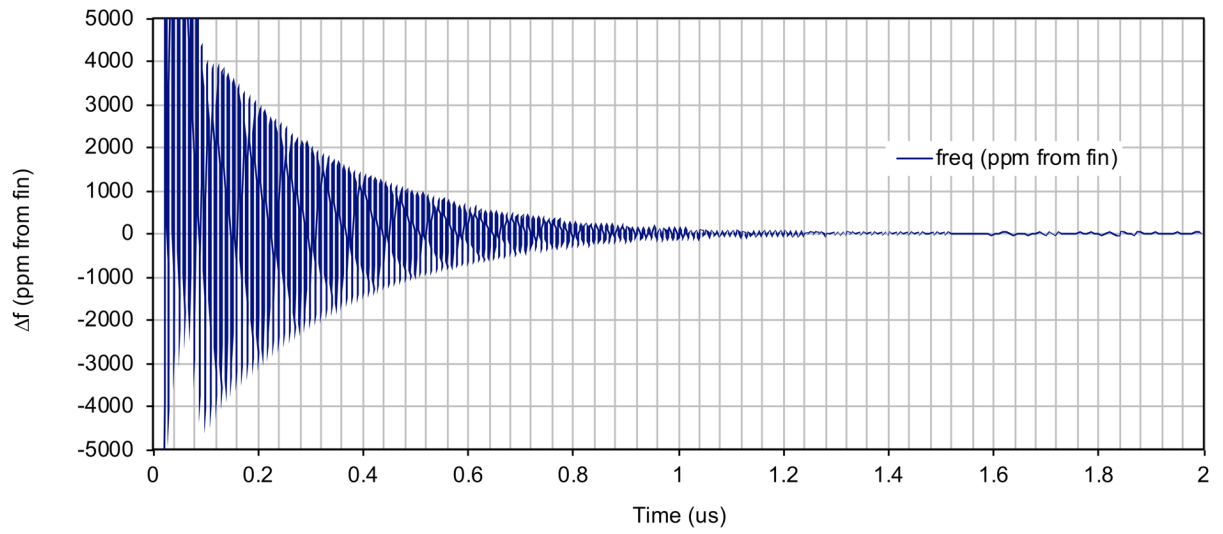
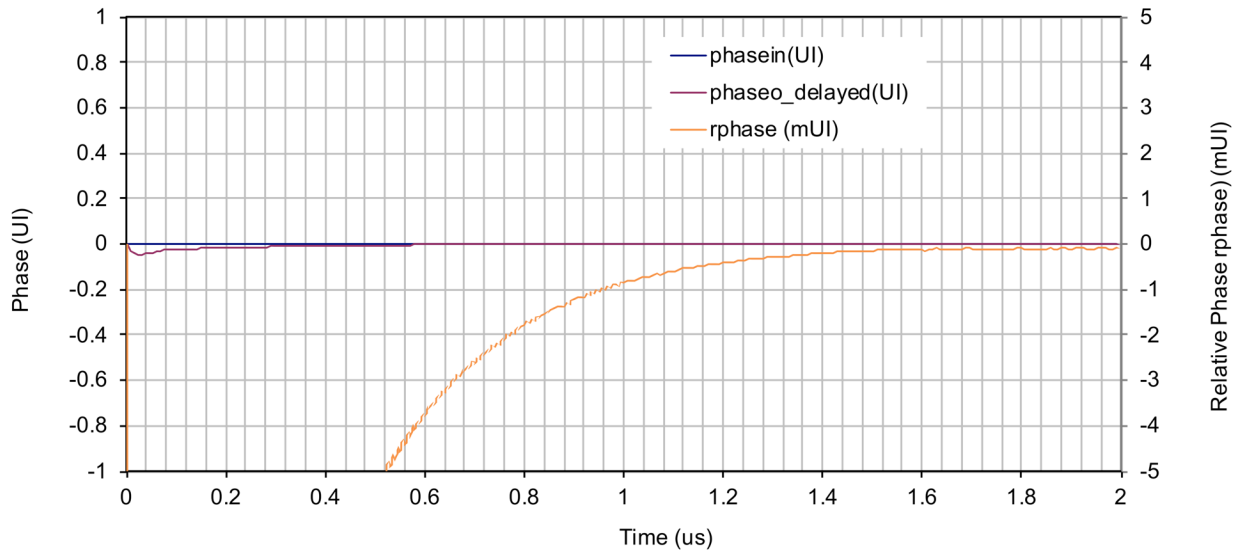


Figure 8

Phase-locked Loop Input, Output, and Phase Error Variation with Time
PFD input frequency = 100.0 MHz, VCO frequency = 12.0 GHz
SSC Range = 0 ppm, SSC frequency = 0 kHz



3.1.2 Graphical Output using Gnuplot and Octave

With Gnuplot installed and selected as the plotting tool in the input ASCII file, and in the UNIX search path, *trans_pll* will create plots of the VCO output voltage versus control voltage characteristic, the control voltage versus time, and the input and output phase response of the phase-locked loop. For the example input ASCII file simulation, three of the four plots are illustrated in Figure 9, Figure 10, and Figure 11. Not shown is a fourth plot that provides an expanded x-axis view of the control voltage versus time.

If Octave is chosen as the plotting tool in the input ASCII file in lieu of Gnuplot, *trans_pll* creates the plots shown in Figure 12 through Figure 16. Excluded from this set of plots, once again, is an expanded x-axis plot of the control voltage.

As shown in Figure 16, the octave plotting tool option includes a plot of the power spectral density of the output phase using the last 2^N points of the simulation where 2^N is the largest binary submultiple of points that is less than or equal to the number of data points. In this example where there are 5000 data points, the PSD is computed from the last 4096 data points. The power spectral density plot is not included as a plot when Gnuplot is selected. However, the auxiliary program *psd_sppowr* in reference [2] is available for download and may be used to compute and display the power spectral density of the output phase using the phase data in the *trans_pll* comma-separated variable output file.

Figure 9

12.0000 GHz VCO Output Frequency Control Voltage Characteristics
Center voltage = 425.0 mV, Kvco = 1.04 GHz/V at center voltage

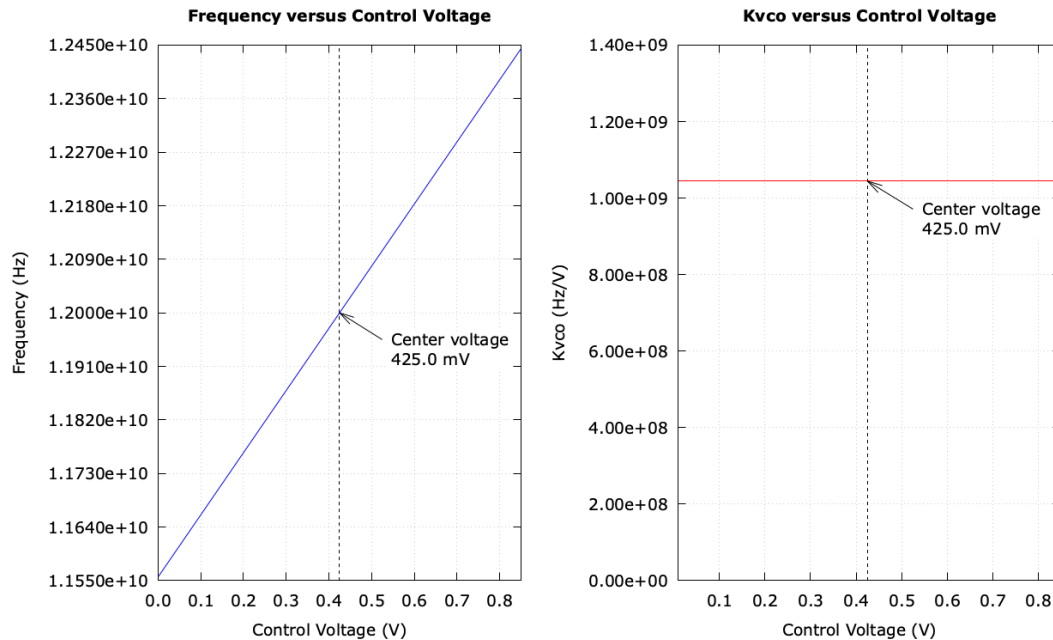


Figure 10

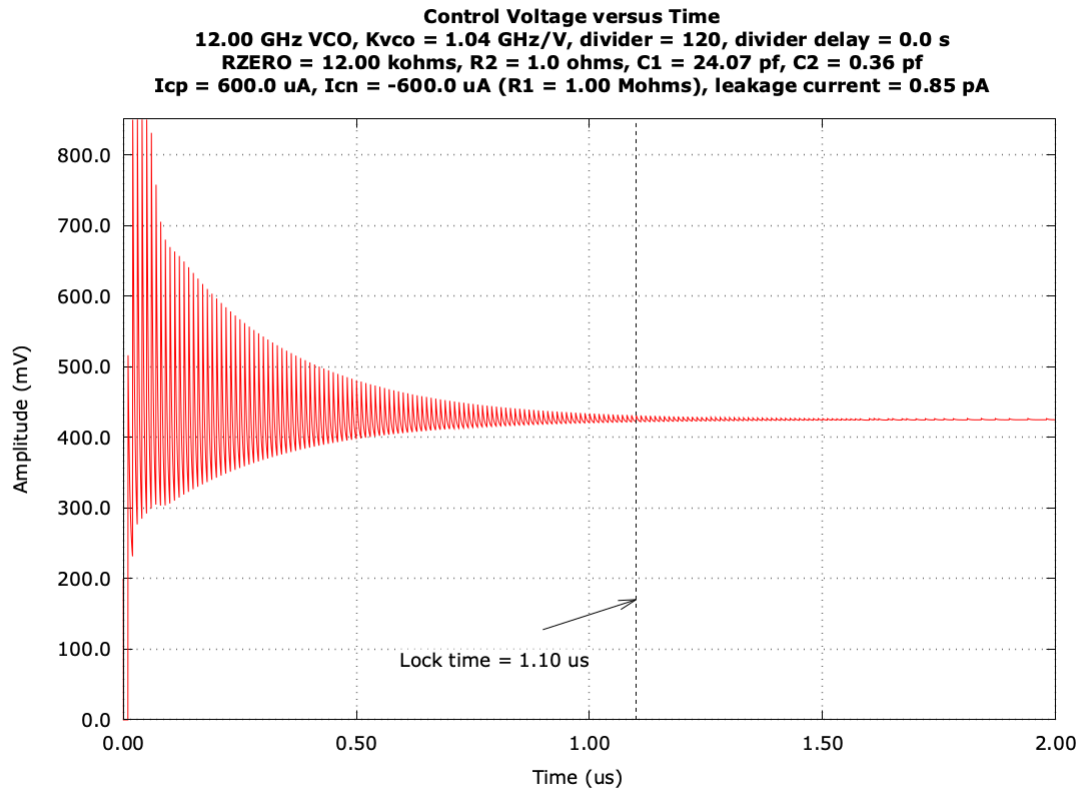


Figure 11

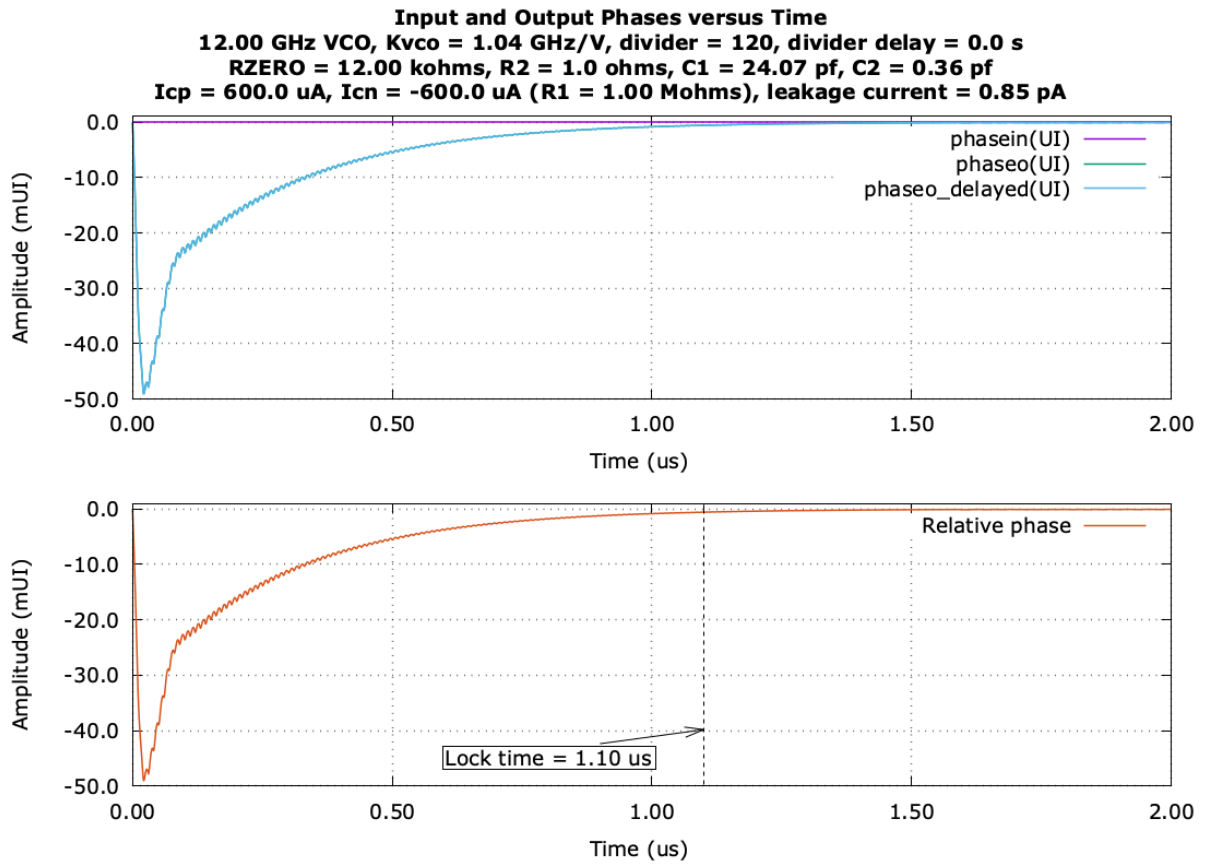


Figure 12

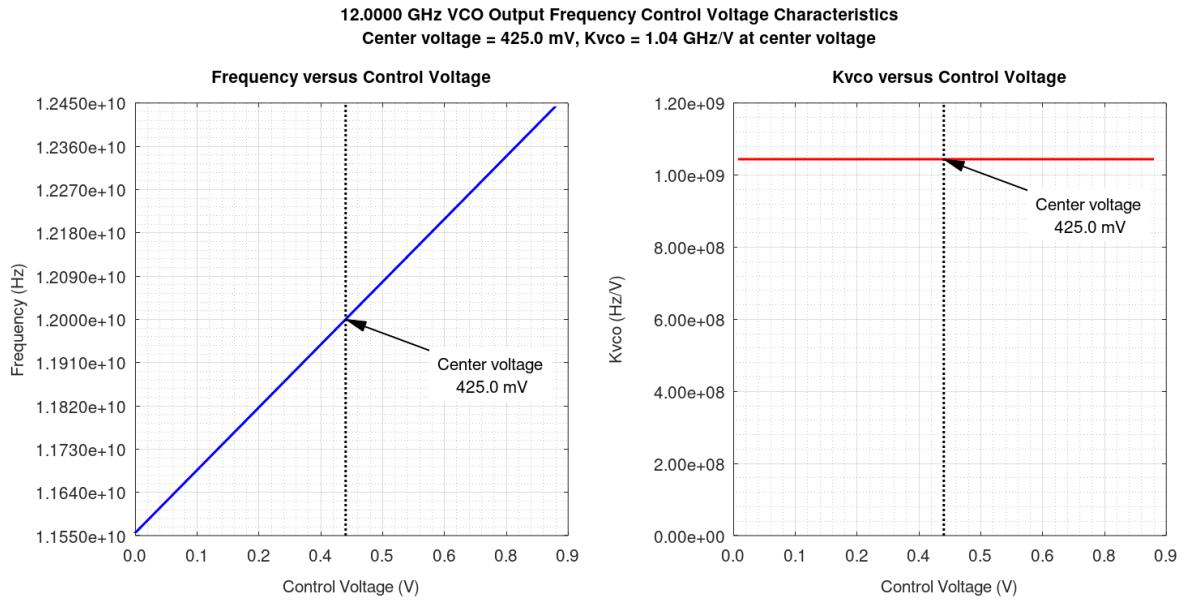


Figure 13

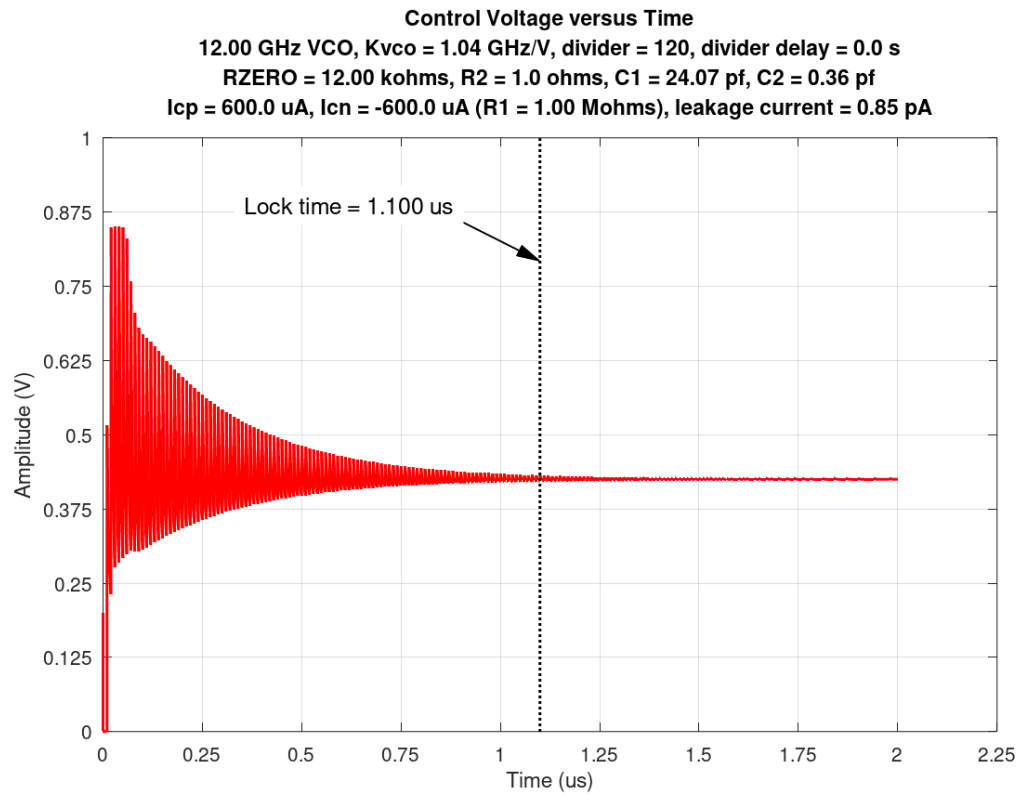


Figure 14

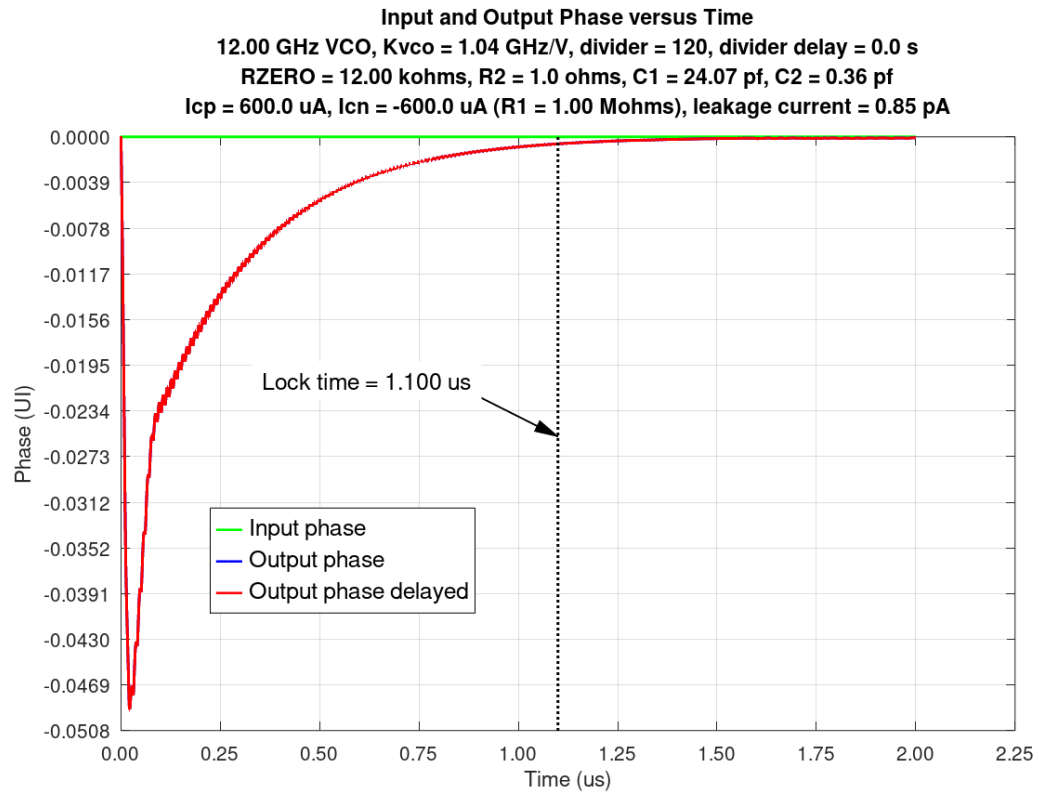


Figure 15

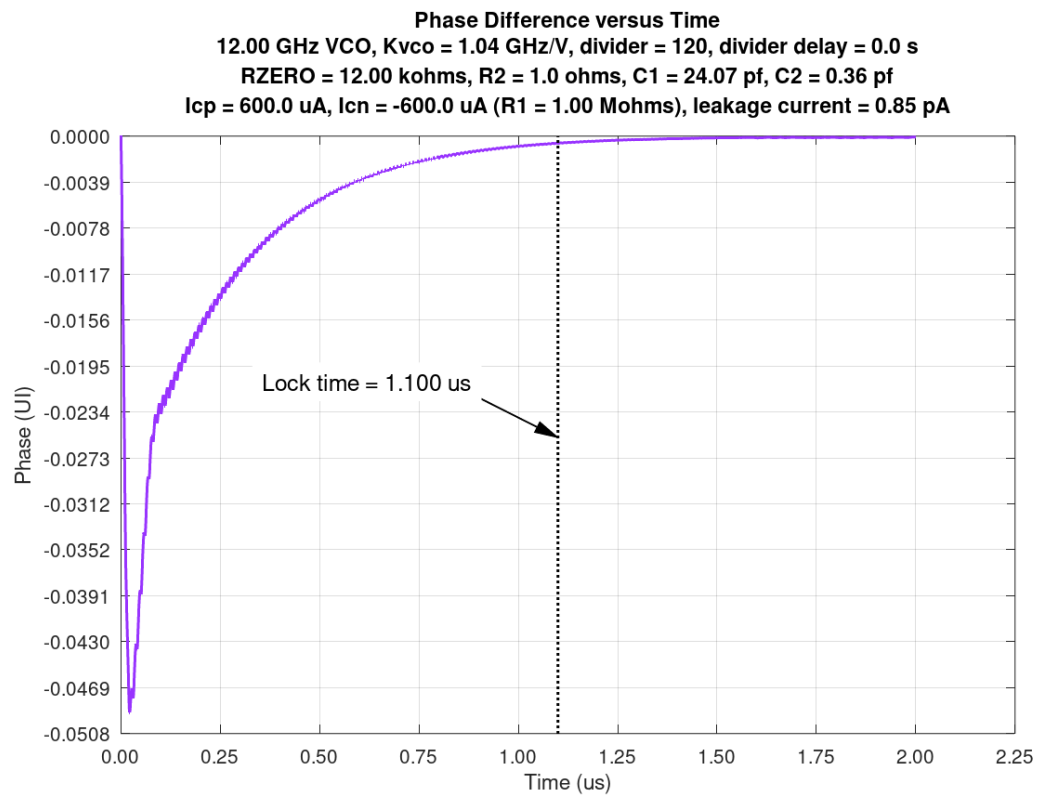
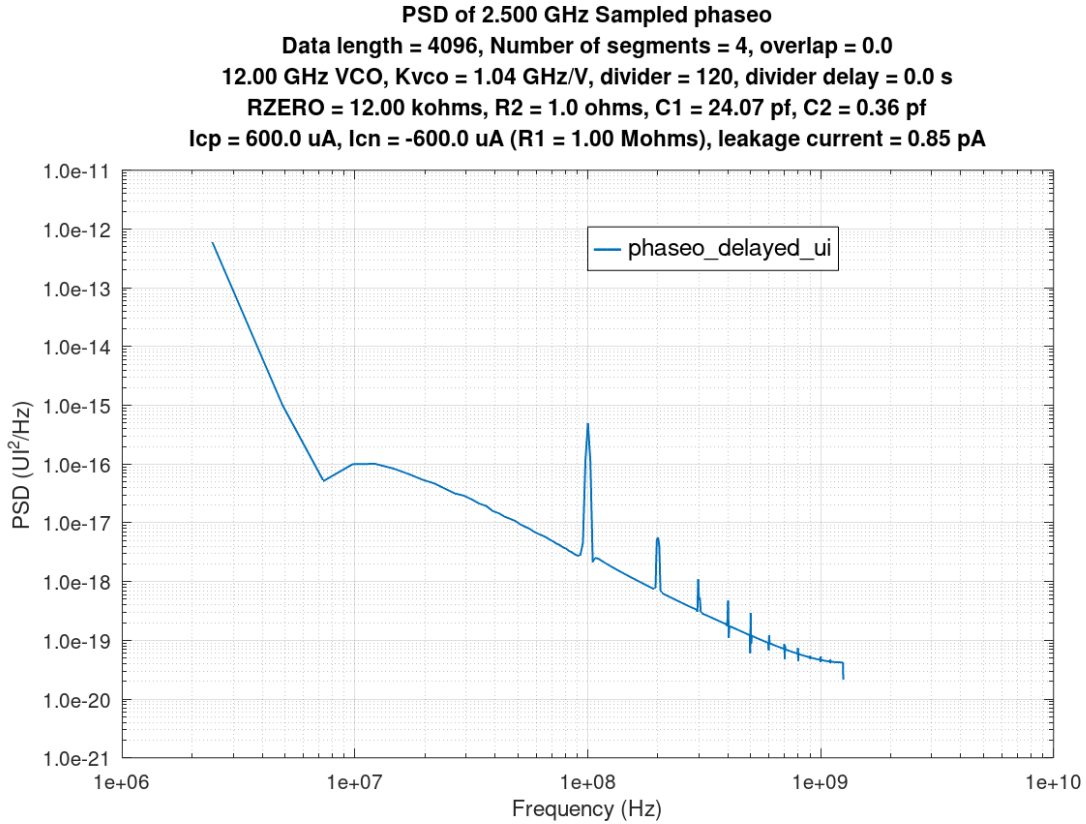


Figure 16



3.2 Basic Operation: Phase-locked loop Lock Detection

trans_pll includes an optional lock detection feature. The time constant of the lock detector, τ , is specified by the “lock_detect_tau(sec)” parameter in the ASCII input file. With τ set to 0, the lock detector is disabled. With a positive and non-zero value of τ , the histories of several loop parameters are recorded at each time point for τ seconds. The running averages, peak-to-peak values and standard deviations of the control voltage and relative phase as well as the phase slope are computed over the history time interval τ and used to gauge phase lock status. If the value of τ chosen is much less than the time constant of the phase-locked loop or the expected lock time, the resulting phase lock status may not accurately indicate phase lock. A small value for τ limits the number of time samples used to compute loop parameter running averages and standard deviations. Hence, its value should be chosen as a reasonable fraction of the loop bandwidth or expected lock time of the phase-locked loop.

Values for three optional thresholds may be included in the *trans_pll* ASCII input file to allow additional adjustment of phase-lock status. The three parameters are shown in Figure 2 and are: the maximum difference in percent between the average VCO control voltage and its center voltage to declare phase-lock; the peak-to-peak difference between the input and output phase of the phase-locked loop to declare phase-lock (expressed in Upp); and the peak-to-peak difference between the input and output phase of the phase-locked loop to declare an out of lock condition (also expressed in Upp). If these three thresholds are either not included in the ASCII input file or they are assigned to the value of “default”, *trans_pll* will use its default value for the respective parameter.

The lock detector feature is especially useful when performing a set of jitter transfer simulations. The jitter transfer gain and phase results will only include those simulations where phase lock is detected. If the lock detector τ is set to 0 (disabled) or incorrectly chosen, the jitter transfer results reported may include simulations for which the loop does not phase-lock.

In the example provided in Section 3.1, a lock detector τ of 100 ns selected and is a reasonable value relative to the bandwidth of the phase-locked loop and its lock time of about 1.1 us. Note that the resolution of the lock time is set by the value of τ .

3.3 Basic Operation: Phase-locked Loop Jitter Transfer Simulation

To initiate a jitter transfer simulation, a comma delimited list of non-zero frequencies and a non-zero sinusoidal jitter amplitude is included in the simulation input file. Parameter TSTART_jt allows one to choose the time to start the jitter transfer analysis to allow for any transient waveform settling. The stop time of the simulation is automatically set to include at least 32 cycles of the input phase modulation period. A bandpass filter centered about the input phase modulation frequency is applied to the output data at times greater than TSTART_jt and the jitter transfer is computed as the ratio of the output phase modulation divided by the input phase modulation. The jitter transfer is echoed to the stdio after each jitter frequency analysis, and each value is appended to a jitter transfer comma-separated variable file. The jitter transfer filename is based on the output_base_filename string in the ASCII input file.

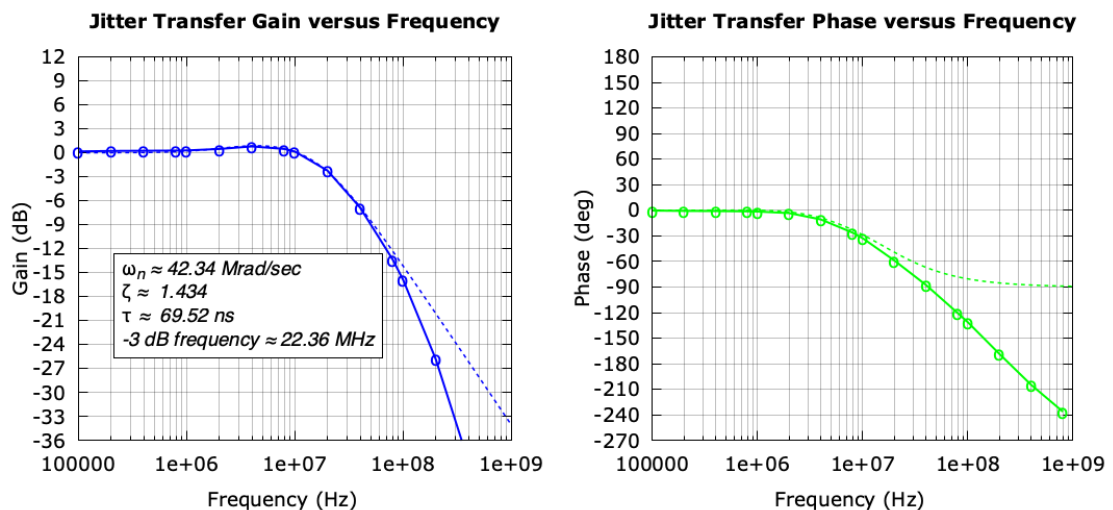
The ASCII input file "jitter_amp_100_mui_061626_16_53_16.txt" shown in Figure 20 was used to provide an example jitter transfer simulation. The ASCII file includes parameters for a 16 GHz VCO PFD based phase-locked loop with an input reference frequency of 2 GHz. The jitter amplitude is set to 100 mUI for the jitter transfer simulation and there are 17 jitter frequencies between 10 kHz and 1 GHz. The output filename from the jitter transfer simulation reflects the base filename of "jitter_amp_100_mui". With Gnuplot installed and in the search path, *trans_pll* produces the transfer function gain and phase plot provided in Figure 17 and its data file "jitter_amp_100_mui_jitter_transfer_061626_16_53_16.csv" shown in Figure 21. Also shown as dashed lines in Figure 17 are the result of curve fits to the transfer gain and phase to provide an estimate of the -3 dB corner frequency. If *trans_pll* determines the jitter transfer gain and phase data are consistent with either a type 1 first-order, a type 2 second-order phase-locked loop, or a type 2 third-order phase-locked loop, it attempts to fit the gain data to the "best fit" transfer function and estimates the -3 dB corner frequency. For a second-order loop, it also provides an estimate for ω_n , its zero time constant $\tau = \frac{1}{\omega_z}$, and the damping factor ζ .

The example provided is a type 2 third-order phase-locked loop. Hence, its long-term phase has an additional 90 degree component beyond the long-term phase of a type 2 second-order curve fit. This is evident in the phase transfer curve in in Figure 17. Its gain transfer function, however, is reasonably well modeled by the type 2 second-order loop curve fit for frequencies less than about 100 MHz.

If a curve fit is performed, a separate log file is created containing the equation used to fit the gain transfer function and its fitted parameter values. In this example, the curve fit log filename is "second_order_jitter_transfer_curve_fit_061626_16_53_16.log" and its contents are shown in Figure 22.

Figure 17

**16.00 GHz VCO, Kvco = 1.39 GHz/V, divider = 8, divider delay = 0.0 s, RZERO = 2.8 kohms, R2 = 1.0 ohms
C1 = 24.70 pf, C2 = 0.36 pf, Icp = 230.5 uA, Icn = -230.5 uA (Ro = 1.0 Mohms), leakage current = 0.85 pA**



If the -3 dB corner frequency of the curve fit to the transfer function gain differs significantly from the simulated -3 dB corner frequency, the curve fit results are not included in the jitter transfer gain and phase plots, and the curve fit file is not saved.

Figure 18 and Figure 19 compare the large signal and small-signal jitter transfer gain and phase responses respectively. The large and small-signal results for frequencies less than 100 MHz correlate well. For frequencies over 100 MHz, large signal effects cause the phase results to differ from small-signal results.

Figure 18

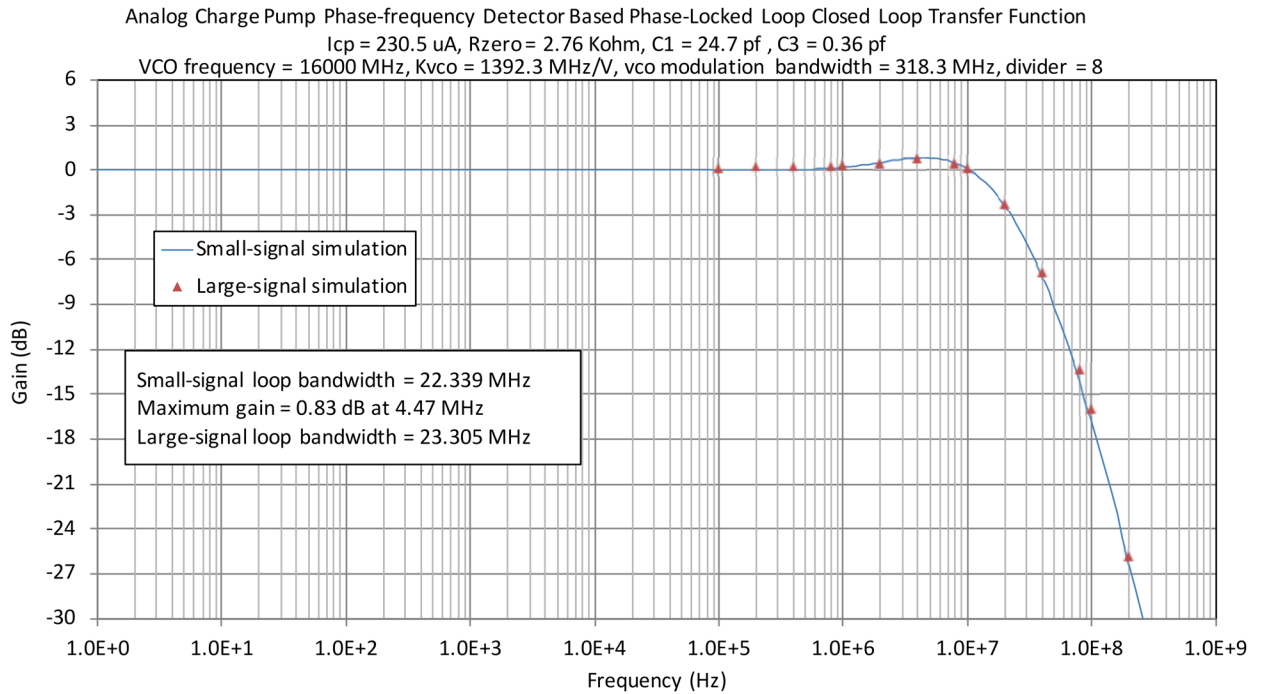


Figure 19

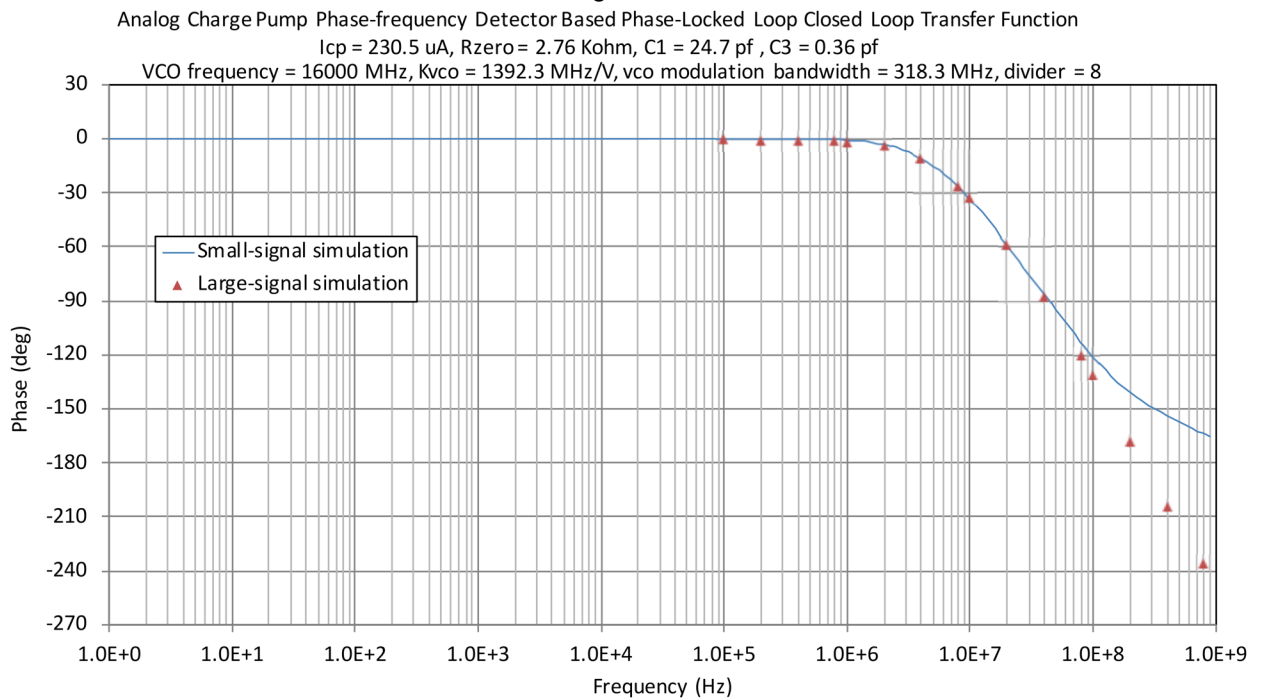


Figure 20
Contents of trans_pll ASCII Input File "sample_pfd_pll_input_file.txt"

```
* Comment lines begin with a *
* Phase detector options, choose dff, pfd, pd_windowed, exor, none_vpd=1,
* none_vpd=0, hogge, alexander
*
phase_detector: pfd
*
* Phase detector related inputs (not all are used for each phase detector)
*
tauff(sec): 1E-11
tauh(sec): 1E-11
taucp_min(sec): 1E-12
pfd_deadzone(ui): 0
*
* pd_windowed phase detector related inputs
*
tw(sec): <Enter width of window>
tacq_max(sec): <Enter maximum allowed acquisition time in sec>
*
* CDR phase detector options
*
* Phase threshold beyond which amplitude of Alexander phase detector saturates
*
alexander_phase_detector_phase_threshold(UI): <Enter positive threshold in UI or the word default>
*
* VCO related inputs
*
fo(VCO_freq_in_Hz): 12000000000
vco_clock_duty_cycle(fo): 0.4
*
* Choose model for control voltage characteristic
* polynomial or poly allows up to ninth order polynomial  $\Delta f = c_0 + c_1*vc + c_2*vc + \dots c_8*vc$ 
* tanh function requires four coefficients  $\Delta f = c_0 + c_1*\tanh(c_2*(vc-c_3))$ 
*
vco_control_voltage_model: poly
vc_polynomial_coeff(hz/V): -0.036984,0.087021
*
* Example  $k_{vco} = c_1*fo = 2.0*500e6 = 1.0$  GHz/V
*  $V_{co} = -(c_0/c_1) = -(-1/2) = 0.5$  V
*
vco_bandwidth(Hz): 40000000
*
* Clock or data input parameters
*
fin(input_freq_in_Hz): 100000000
feedback_divider: 120
feedback_divider_delay(sec): 0
*
* Feedback clock duty cycle (not used if divider is 1.0)
*
feedback_divider_clock_duty_cycle: 0.1
*
* Input data pattern (single bit per line, text file)
*
filename_with_input_data_pattern: <Enter filename containing input data pattern>
*
* Enter type of noise on input (gaussian or uniform)
* random_noise_sigma is sigma for gaussian
* noise, and for uniform noise range of [-sigma,+sigma]
*
noise_type: <Enter choice for noise type>
random_noise_sigma(UI_on_fin): 0
noise_bandwidth(Hz): <Enter bandwidth of input noise in Hz>
*
* Add sinusoidal jitter or perform jitter transfer
*
jitter_amp(UIpp): 0
jitter_freqs(Hz): 0
*
* Initial phase of input
*
phase0(degrees): 0
```


Figure 20 (cont.)
Contents of trans_pll ASCII Input File "sample_pfd_pll_input_file.txt"

```
*
* Start and stop times of printed output, sinusoidal input jitter starts at TSTART_jt
*
TSTART(sec): 0
TSTART_jt(sec): 0.000002
TSTOP(sec): 0.000002
*
* Enter desired timestep or the word default
*
deltat(sec): 1E-13
*
* Enter desired lock detector time constant - enter 0 for no lock detector function
*
lock_detect_tau(sec): 100e-09
*
* Enter desired number of points to save
*
number_of_print_points: 5001
*
* Inputs for phase or frequency step
*
phase_freq_step_time(sec): 1
phase_step(sec): 0
freq_step(ppm): 0
*
* Inputs for application of SSC
*
ssc_start_time(sec): 1
max_pos_ssc(ppm): 0
max_neg_ssc(ppm): 0
freq_ssc(Hz): 0
*
* Phase detector and loop filter inputs
*
phase_detector_logic_hi(V): 600
phase_detector_logic_low(V): -600
signal_source_vs1(v): 0
signal_source_vs2(v): 0
power_supply_max(V): 0.85
power_supply_min(V): 0
rs1(ohms): 1E+12
rs2(ohms): 1E+12
R1(ohms): 1000000
RZERO(ohms): 12000
R3(ohms): 1
C1(farads): 2.407E-11
C2(farads): 3.6E-13
initvc(V): 0.2
*
* Output filename and format (csv only)
*
output_base_filename: sample_output_file_v1p76
*
* Optionally plot output characteristics and vco control voltage characteristics.
* Enter value of zero to omit all plots or any non-zero value to create plots.
* Note - plotting requires Octave or gnuplot be installed. The value of plot_outputs
* is forced to zero if one or the other programs is not installed
*
plot_outputs: 1
*
* Enter preferred plotting tool of octave or gnuplot
* Plots will be created using preferred plotting tool if
* the executable is found.
*
plot_preference: gnuplot
*
* End of trans_pll inputs
*
```

Figure 21
Contents of Jitter Transfer Output File "jitter_amp_100_mui_jitter_transfer_061626_16_53_16.csv"

```
Frequency (Hz),Gain (dB),Phase (degrees),Lock time (s),TSTOP (s)
1.000000000000e+05,1.724770846445e-01,-2.755377470246e-01,1.000000000002e-07,3.205000000000e-04
2.000000000000e+05,2.303876785378e-01,-7.043449396481e-01,1.000000000002e-07,1.605000000000e-04
4.000000000000e+05,2.547381565930e-01,-8.798922230501e-01,1.000000000002e-07,8.050000000000e-05
8.000000000000e+05,2.777934673530e-01,-1.198887694824e+00,1.000000000002e-07,4.050000000000e-05
1.000000000000e+06,2.953097334025e-01,-1.395296962377e+00,1.000000000002e-07,3.250000000000e-05
2.000000000000e+06,4.777115445758e-01,-3.227315568928e+00,1.000000000002e-07,1.650000000000e-05
4.000000000000e+06,7.917161860670e-01,-1.070363797701e+01,1.000000000002e-07,8.500000000000e-06
8.000000000000e+06,4.826311762765e-01,-2.611380711737e+01,1.000000000002e-07,4.500000000000e-06
1.000000000000e+07,1.471831622737e-01,-3.269271813823e+01,1.000000000002e-07,3.700000000000e-06
2.000000000000e+07,-2.235964986342e+00,-5.879923728623e+01,1.000000000002e-07,2.100000000000e-06
4.000000000000e+07,-6.859771333060e+00,-8.751902703232e+01,1.000000000002e-07,1.300000000000e-06
8.000000000000e+07,-1.335585624157e+01,-1.203872512115e+02,1.000000000002e-07,2.100000000000e-06
1.000000000000e+08,-1.590265989080e+01,-1.312153956534e+02,1.000000000002e-07,1.780000000000e-06
2.000000000000e+08,-2.581211311723e+01,-1.68457592137e+02,1.000000000002e-07,1.780000000000e-06
4.000000000000e+08,-3.858027495817e+01,-2.045979878372e+02,1.000000000002e-07,1.780000000000e-06
8.000000000000e+08,-5.395298151121e+01,-2.359638531397e+02,1.000000000002e-07,1.780000000000e-06
1.000000000000e+09,-1.496645653105e+02,nan,1.000000000002e-07,1.524000000000e-06
```

Figure 22
Jitter Transfer Curve Fit Log File "second_order_jitter_transfer_curve_fit_061626_16_53_16.log"

```
Choosing second-order or third-order curve fit.

*****
Tue Jun 16 17:14:25 2026

FIT:      data read from "tempfile100_061626_16_53_16" u ($1*two_pi):2:6
         format = x:z:s
         #datapoints = 17
         function used for fitting: log_h2(x)
         log_h2(x) = 20.0*log10((((wn_square_root**2)*(wn_square_root**2))**2) +
(x*(tau_sec_square_root**2)*(wn_square_root**2)*(wn_square_root**2))**2**0.50)/((((wn_squ
are_root**2)**2 - x**2)**2 +
(2.0*(damping_square_root**2)*x*(wn_square_root**2))**2**0.50))
         fitted parameters initialized with current variable values

         46 1.5938951626e+04  -6.93e-01  7.85e-04    6.506999e+03   1.197310e+00   2.636583e-04

After 46 iterations the fit converged.
final sum of squares of residuals : 15939
rel. change during last iteration : -6.92677e-06

degrees of freedom      (FIT_NDF)                : 14
rms of residuals        (FIT_STDFIT) = sqrt(WSSR/ndf)    : 33.7416
variance of residuals (reduced chisquare) = WSSR/ndf    : 1138.5
p-value of the Chisq distribution (FIT_P)              : 0

Final set of parameters      Asymptotic Standard Error
=====
wn_square_root = 6507        +/- 722.2          (11.1%)
damping_square_ = 1.19731    +/- 0.1203         (10.05%)
tau_sec_square_ = 0.000263658 +/- 6.031e-05      (22.87%)

correlation matrix of the fit parameters:
      wn_squ dampin tau_se
wn_square_root  1.000
damping_square_-0.968  1.000
tau_sec_square_-0.996  0.985  1.000
second-order curve fit result: wn = 4.2341e+07, damping = 1.43355, wc = 1.4051e+08,
tau_sec = 6.95157e-08

Second-order gain error mean = 0.552205, Second-order gain error sigma = 1.52707,
Second-order gain error sum_sq = 36.9161 Second-order phase error mean = 30.4264
```

Figure 22 (cont.)

Jitter Transfer Curve Fit Log File "second_order_jitter_transfer_curve_fit_061626_16_53_16.log"

```
*****
Tue Jun 16 17:14:25 2026

FIT:    data read from "tempfile100_061626_16_53_16" u ($1*two_pi):2:6
        format = x:z:s
        #datapoints = 17
function used for fitting: log_h3(x)
        log_h3(x) = 20.0*log10(b0*((1.0 + (c0*x)**2)*0.50)/(((1.0 - (a2*x)**2)**2 + (a1*x
- (a0*x)**3)**2)**0.50)
fitted parameters initialized with current variable values

    11 2.9318366334e+04  -4.24e-02  1.26e+00  -1.073983e-01  3.508403e-04  1.000019e-02
-7.364285e-02

After 11 iterations the fit converged.
final sum of squares of residuals : 29318.4
rel. change during last iteration : -4.2387e-07

degrees of freedom      (FIT_NDF)                : 13
rms of residuals        (FIT_STDFIT) = sqrt(WSSR/ndf) : 47.4896
variance of residuals (reduced chisquare) = WSSR/ndf : 2255.26
p-value of the Chisq distribution (FIT_P)            : 0

Final set of parameters      Asymptotic Standard Error
=====
c0      = -0.107398          +/- 3.818          (3555%)
a0      = 0.00035084         +/- 0.008305        (2367%)
a1      = 0.0100002          +/- 7.125e+07        (7.125e+11%)
a2      = -0.0736428         +/- 3.741          (5080%)

correlation matrix of the fit parameters:
      c0      a0      a1      a2
c0      1.000
a0     -1.000  1.000
a1      0.382 -0.383  1.000
a2      0.306 -0.305 -0.763  1.000

Third-order curve fit result: b0 = 1, c0 = -0.107398, a0 = 0.00035084, a1 = 0.0100002 a2 =
-0.0736428, wc = 1.38056e+08
Third-order gain error mean = 9.52178, Third-order gain error sigma = 27.3187,
Third-order gain error sum_sq = 14228.6 Third-order phase error mean = 120.525
Choosing second-order curve fit over third-order curve fit.
```

4. Phase-locked Loop Responses to Input Phase and Frequency Steps

trans_pll includes the ability to introduce a phase or frequency step to the phase locked loop input signal. The three parameters *phase_freq_step_time(sec)*, *phase_step(sec)*, and *freq_step(ppm)* set the time at which the phase or frequency step is introduced in the simulation and the magnitude of the phase and frequency step. The program accepts both positive and negative phase and frequency steps.

To illustrate this capability, simulations of the 16 GHz VCO PFD based phase-locked loop described in Section 3.3 for phase steps between -800 mUI and +800 mUI and for frequency steps between -4000 ppm and +4000 ppm were performed.

4.1 Impact of Input Phase Steps on PLL Output Phase

A summary of the phase step results is provided in Figure 23. A close examination indicates that the transient responses are a function of the phase step size. When the phase response results are normalized to the step size and the average phase just prior to the phase step in Figure 24, it is evident that there is a difference in transient response dependent on the magnitude and the sign of the phase step. The output phase for small positive input phase steps includes a positive offset from the applied input phase step. If, as shown in Figure 25, the initial control voltage of the phase-locked loop is set to 650 mV (225 mV above the VCO center voltage of 425 mV) in lieu of 200 mV (225 mV below the center voltage), the output phase for small negative input phase steps includes a positive offset from the applied

input phase step. This information, combined with the observation that the magnitude of the offset is a function of the magnitude of the input phase step, suggests that the difference in pulse response may be a result of the minimum charge pump pulse width. In each of these simulations, parameter τ_{cp_min} is 1.0 ps or 2 mUI. As is evident in Figure 26, if τ_{cp_min} is reduced to 0.10 ps (0.20 mUI) or 0.01 ps (0.02 mUI), the output phase offsets observed for input phase steps of ± 40 mUI and ± 100 mUI are significantly less. With τ_{cp_min} set to 0.01 ps, Figure 27 illustrates that the normalized phase step responses are no longer a function of the magnitude nor the sign of the input phase step. This analysis allows the designer to specify a value for the required charge pump pulse width.

Figure 27 also superimposes the theoretical third-order pulse response over the simulated responses and indicates the excellent correlation between the large-signal simulated and theoretical pulse responses.

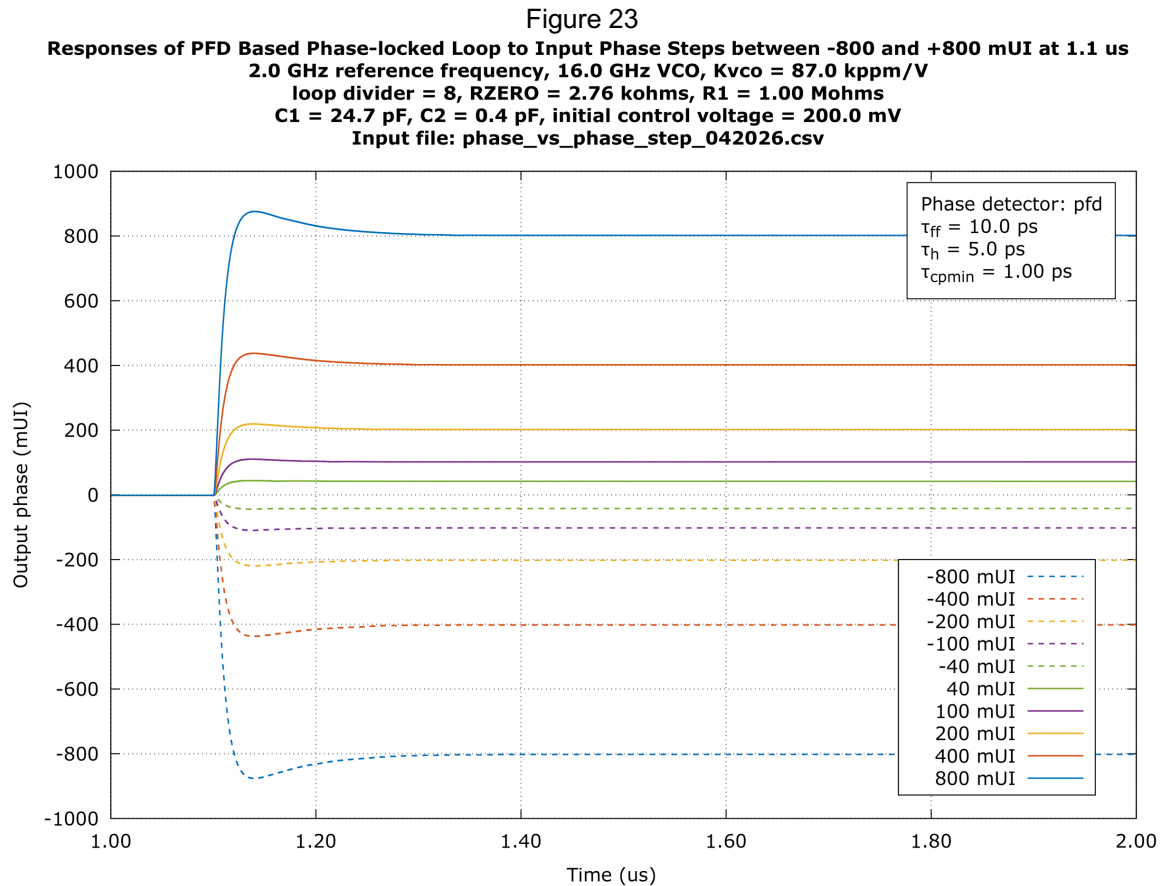


Figure 24

Responses of PFD Based Phase-locked Loop to Input Phase Steps between -800 and +800 mUI at 1.1 us
2.0 GHz reference frequency, 16.0 GHz VCO, $K_{vco} = 87.0$ kppm/V
loop divider = 8, RZERO = 2.76 kohms, R1 = 1.00 Mohms
C1 = 24.7 pF, C2 = 0.4 pF, initial control voltage = 200.0 mV
Input file: normalized_phase_vs_phase_step_042026.csv

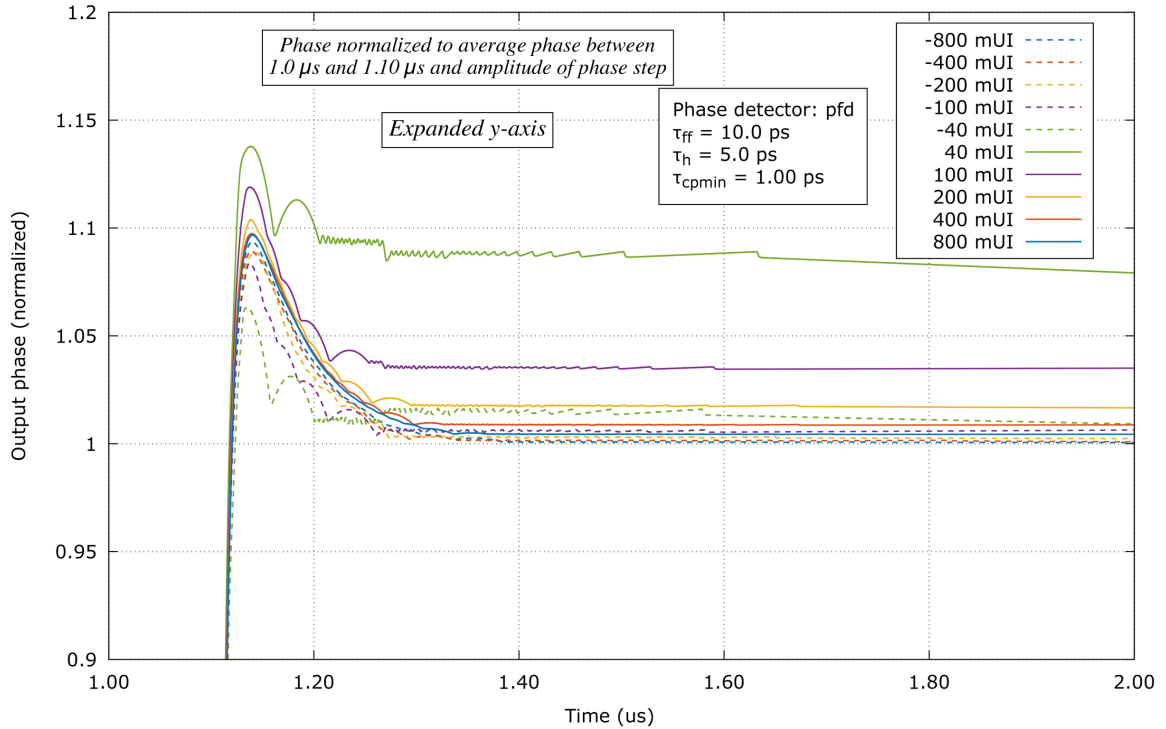


Figure 25

Responses of PFD Based Phase-locked Loop to Input Phase Steps between -800 and +800 mUI at 1.1 us
2.0 GHz reference frequency, 16.0 GHz VCO, $K_{vco} = 87.0$ kppm/V
loop divider = 8, RZERO = 2.76 kohms, R1 = 1.00 Mohms
C1 = 24.7 pF, C2 = 0.4 pF, initial control voltage = 650.0 mV
Input file: normalized_phase_vs_phase_step_042226.csv

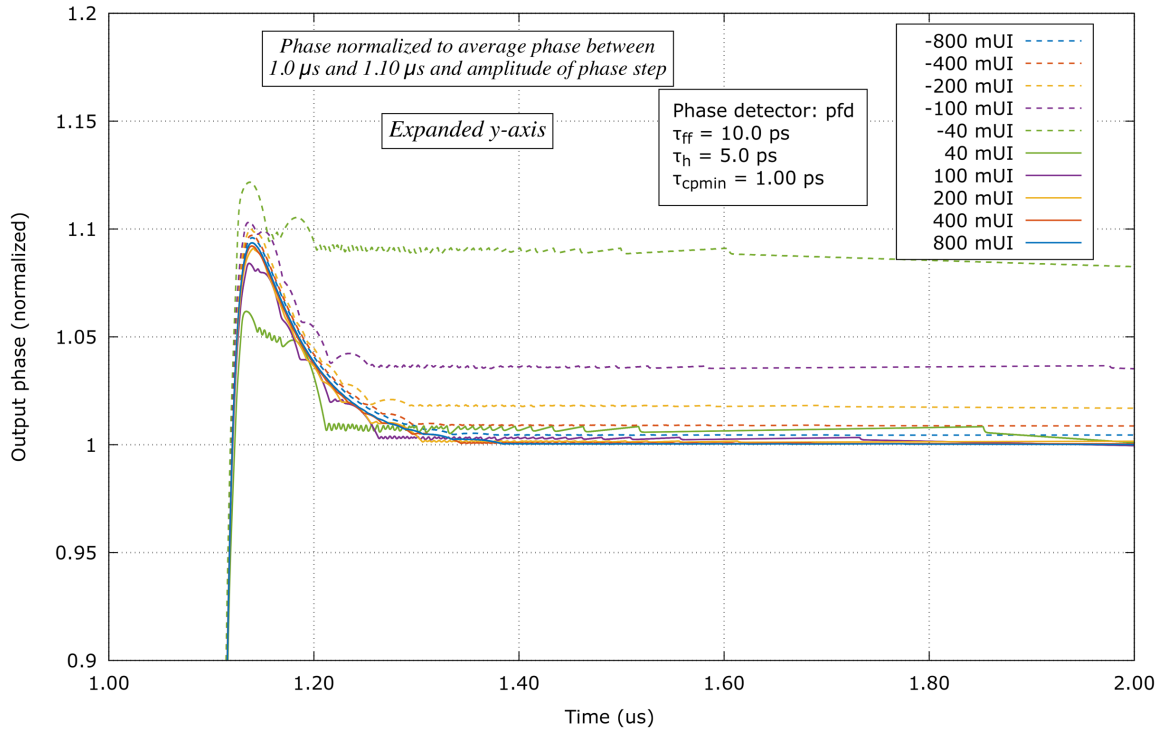


Figure 26

Responses of PFD Based Phase-locked Loop to ± 40 mUI Phase Steps at 1.1 μ s
2.0 GHz reference frequency, 16.0 GHz VCO, $K_{vco} = 87.0$ kppm/V
loop divider = 8, RZERO = 2.76 kohms, R1 = 1.00 Mohms
C1 = 24.7 pF, C2 = 0.4 pF, initial control voltage = 200.0 mV
Input file: 40_mui_normalized_phase_step_vs_tau_cpmin.csv

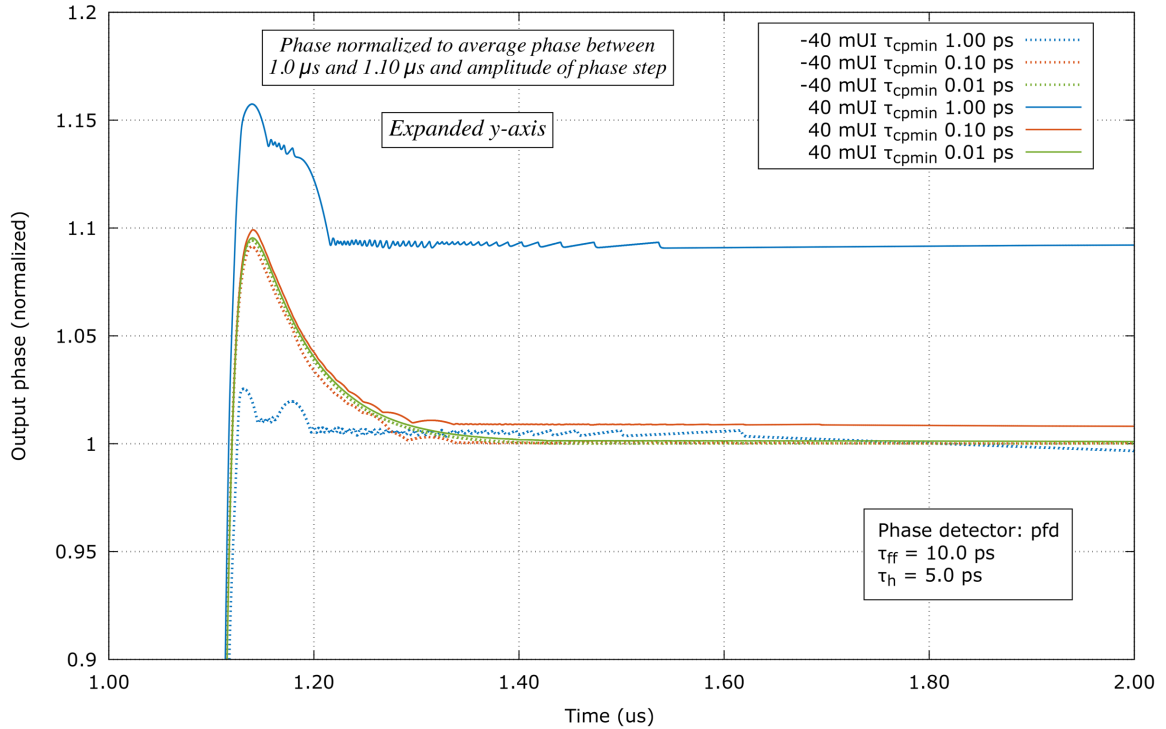
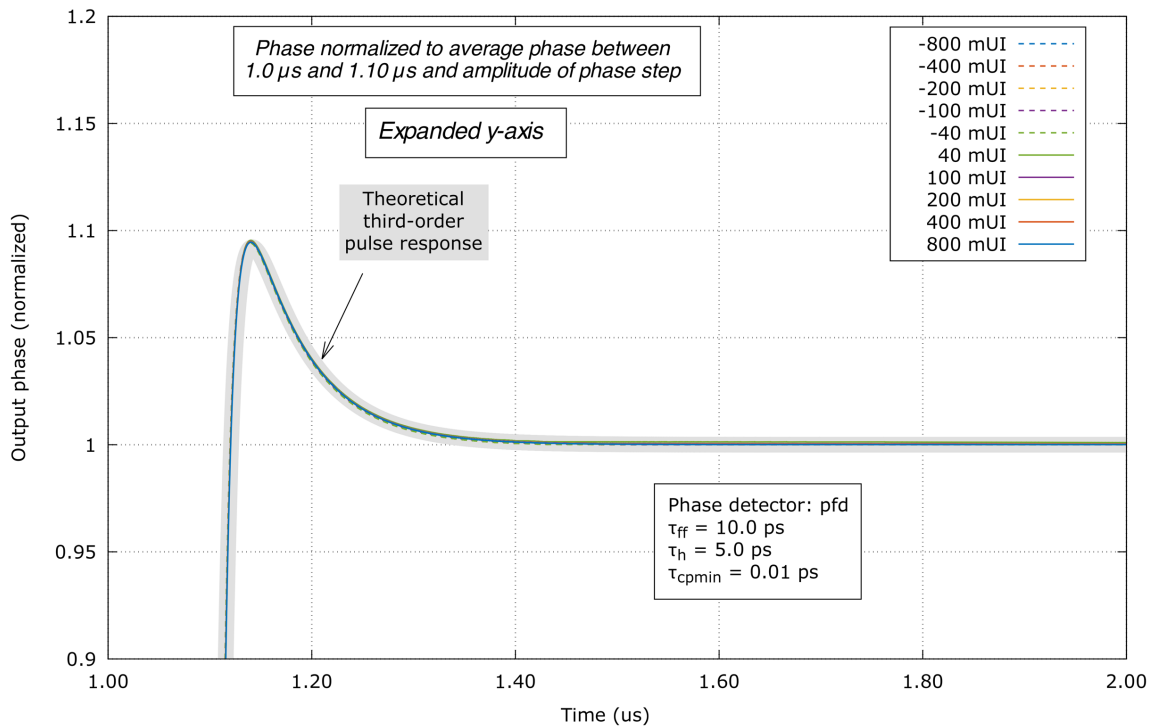


Figure 27

Responses of PFD Based Phase-locked Loop to Input Phase Steps between -800 and +800 mUI at 1.1 μ s
2.0 GHz reference frequency, 16.0 GHz VCO, $K_{vco} = 87.0$ kppm/V
loop divider = 8, RZERO = 2.76 kohms, R1 = 1.00 Mohms
C1 = 24.7 pF, C2 = 0.4 pF, initial control voltage = 200.0 mV
Input file: normalized_phase_vs_phase_step_042026.csv



4.2 Impact of Input Frequency Steps on PLL Output Frequency

The phase-locked loop responses to input frequency steps is provided in Figure 28. Normalizing the output frequency for each frequency step to its magnitude and to its average value just prior to the introduction of the input frequency step provides the results in Figure 29. As observed in the phase step responses with τ_{cp_min} set to 1 ps, the frequency step responses are also a function of the frequency step. When the value of τ_{cp_min} is reduced to 0.10 ps and 0.01 ps, the normalized frequency step responses show less dependence on the magnitude of the step (Figure 30). In Figure 31, with τ_{cp_min} set to 0.01 ps, the normalized frequency step responses are essentially identical and independent of the magnitude of the input frequency step.

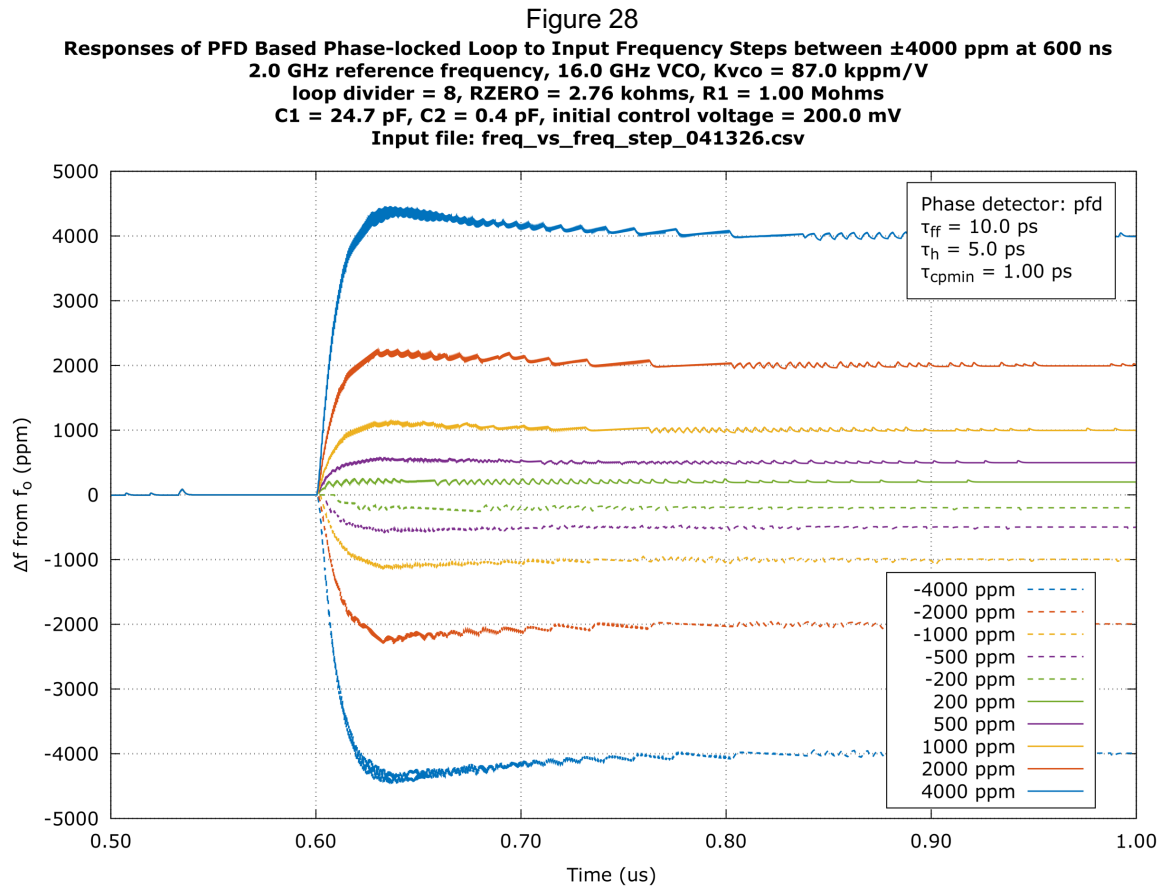


Figure 29

Responses of PFD Based Phase-locked Loop to Input Frequency Steps between ± 4000 ppm at 1.1 μ s
 2.0 GHz reference frequency, 16.0 GHz VCO, $K_{vco} = 87.0$ kppm/V
 loop divider = 8, RZERO = 2.76 kohms, R1 = 1.00 Mohms
 C1 = 24.7 pF, C2 = 0.4 pF, initial control voltage = 200.0 mV
 Input file: normalized_freq_vs_freq_step_042326.csv

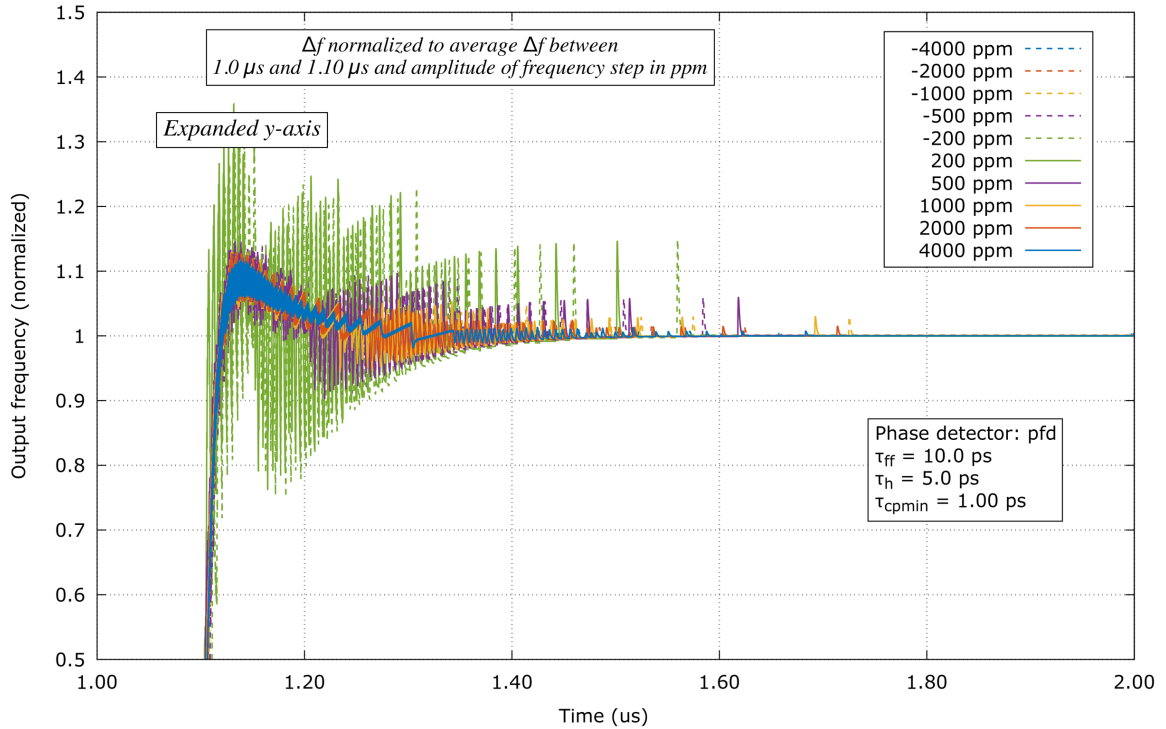


Figure 30

Responses of PFD Based Phase-locked Loop to ± 200 ppm Frequency Steps at 1.1 μ s
 2.0 GHz reference frequency, 16.0 GHz VCO, $K_{vco} = 87.0$ kppm/V
 loop divider = 8, RZERO = 2.76 kohms, R1 = 1.00 Mohms
 C1 = 24.7 pF, C2 = 0.4 pF, initial control voltage = 200.0 mV
 Input file: 200_ppm_normalized_freq_step_vs_tau_cpmin.csv

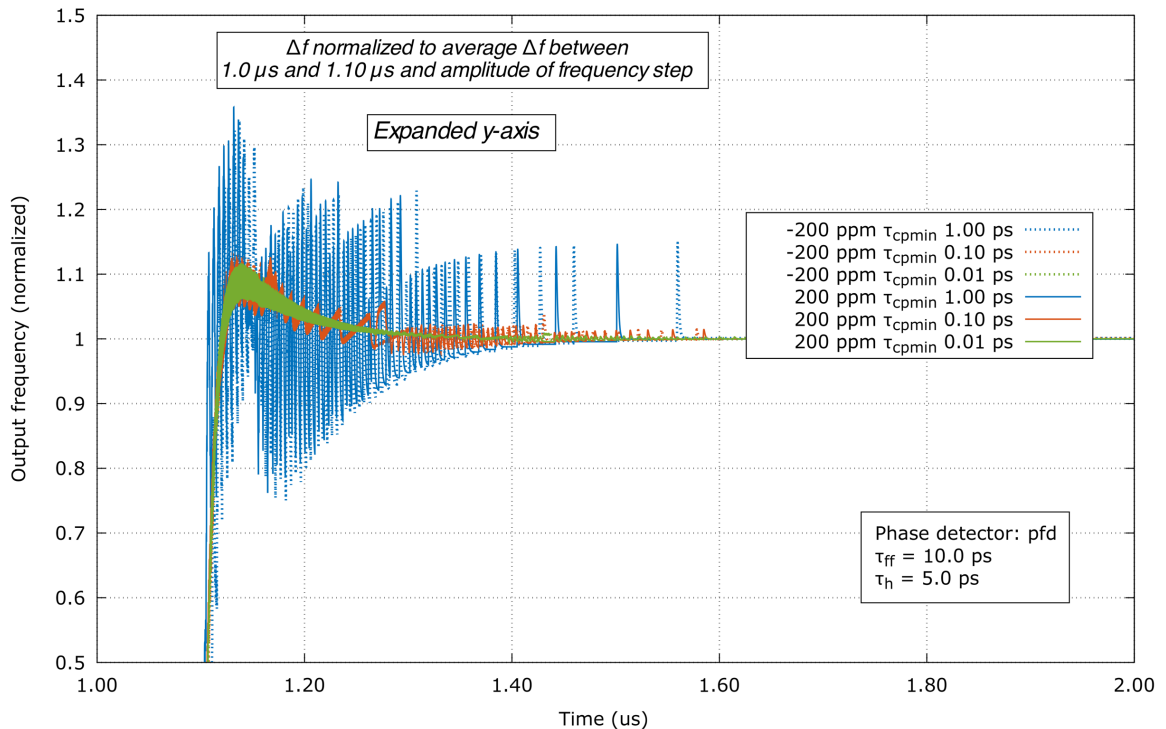
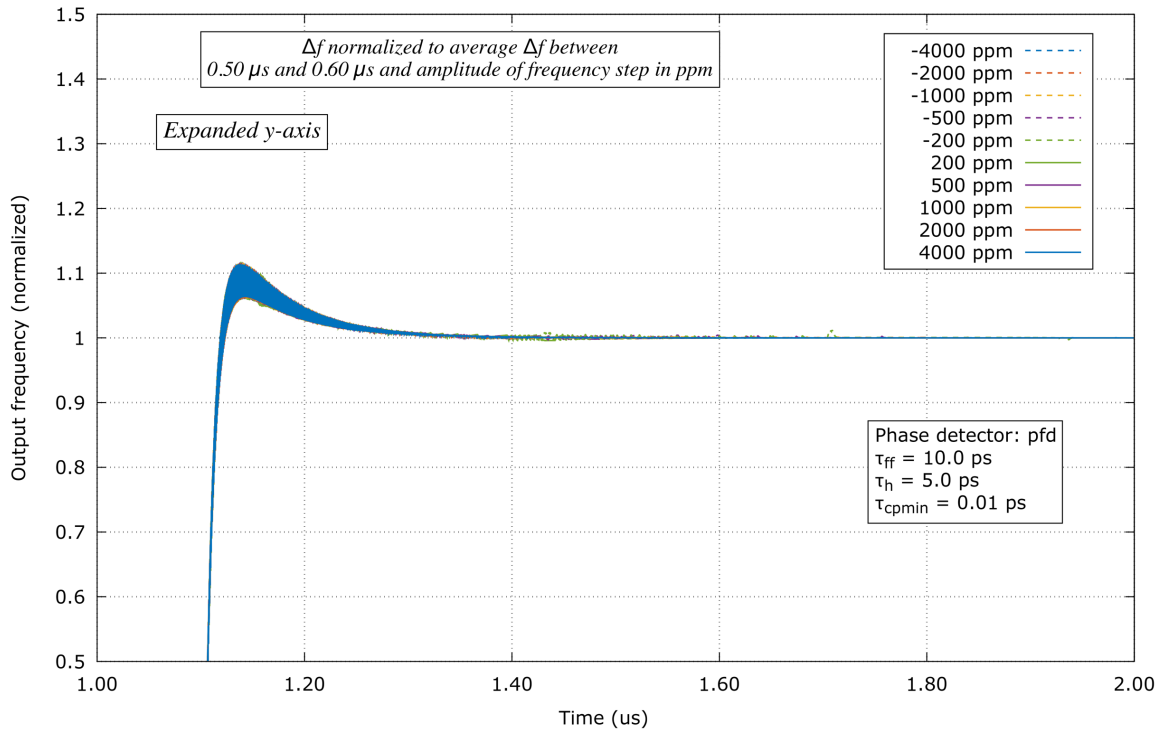


Figure 31

Responses of PFD Based Phase-locked Loop to Input Frequency Steps between ± 4000 ppm at 1.1 μ s
2.0 GHz reference frequency, 16.0 GHz VCO, $K_{vco} = 87.0$ kppm/V
loop divider = 8, RZERO = 2.76 kohms, R1 = 1.00 Mohms
C1 = 24.7 pF, C2 = 0.4 pF, initial control voltage = 200.0 mV
Input file: normalized_freq_vs_freq_step_042326.csv



5. Impact of a Non-linear Phase Detector on PLL Jitter Transfer

Traditionally, the analysis of a phase-locked loop with a non-linear phase detector is not straightforward and requires some type of simulation. The non-linearity results in a phase detector gain that varies with the magnitude of the input phase and hence the jitter transfer function is also a function of the magnitude of the input phase. To illustrate this dependence, the jitter transfer characteristics of a linear phase detector and a non-linear phase detector clock and data recovery phase-locked loop were simulated for various input jitter amplitudes and combined into a single plot.

Examples of a linear and non-linear phase detector include a Hogge phase and an Alexander phase detector. Models for each of these phase detectors are included in *trans_pll*. The jitter transfer functions for each were analyzed with input jitter amplitudes between 100 mUIpp and 1 UIpp and 100% transition density data. Figure 32 illustrates that the transfer function of the Hogge based clock and data recovery loop is independent of the jitter amplitude. However, as shown in in Figure 33, the transfer function of the Alexander based clock and data recovery loop shows a strong dependence on input jitter amplitude.

Figure 32

Impact of Input Jitter Amplitude on Jitter Transfer
Hogge phase detector based CDR, 4 GHz VCO, $K_{vco} = 0.20$ GHz/V, 4 Gbs, 100% transition density data
 $I_{cp} = 400$ uA, RZERO = 800 ohms, C1 = 5 nF, C2 = 1 pf, $\Delta V = 30$ mV from center voltage

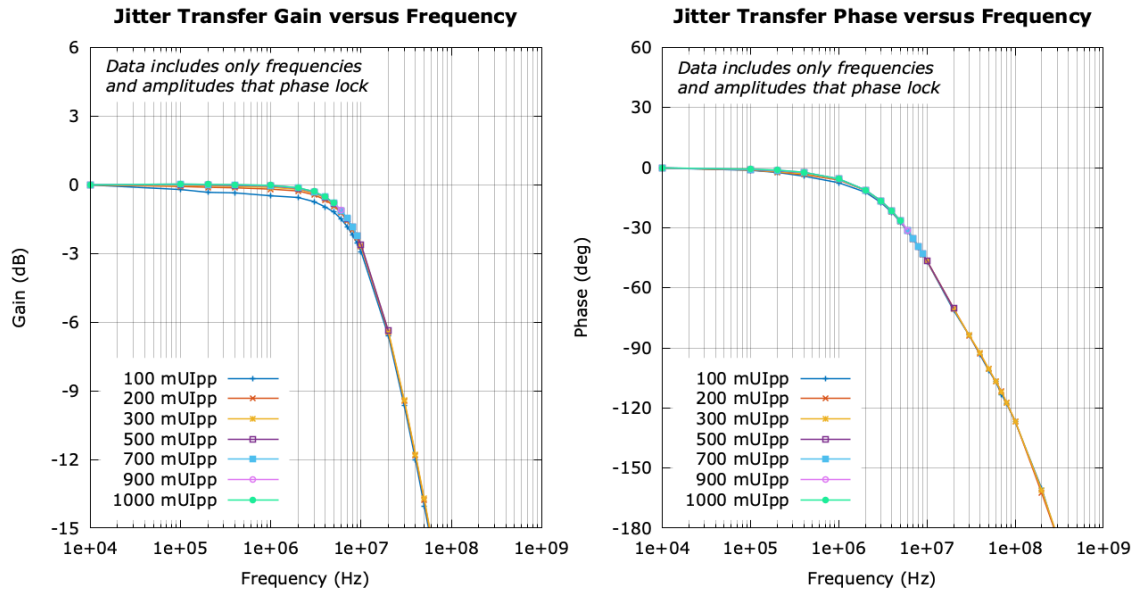
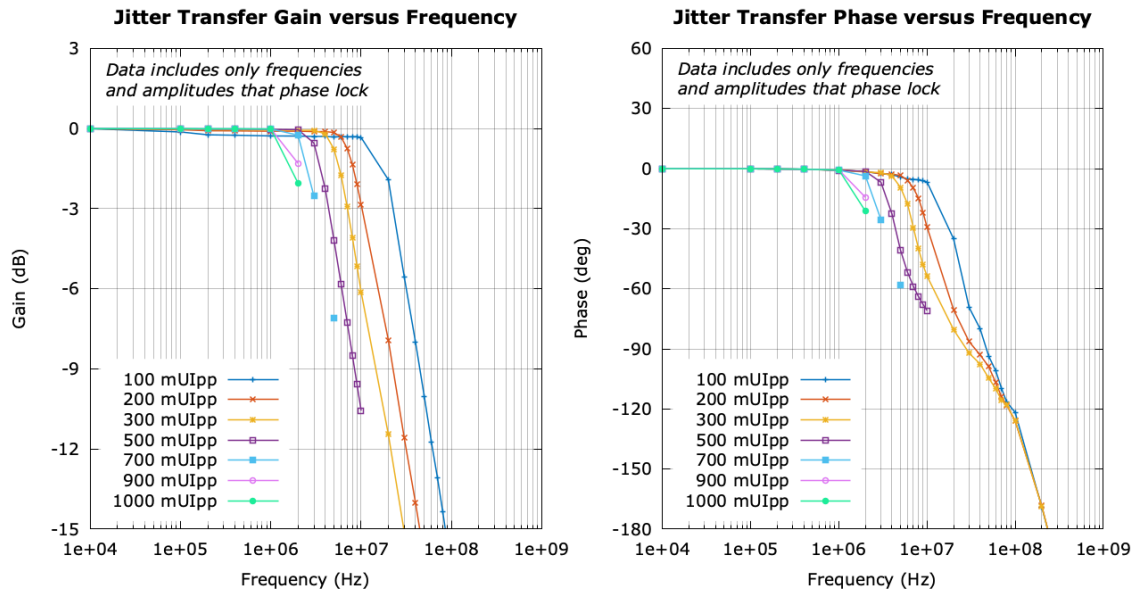


Figure 33

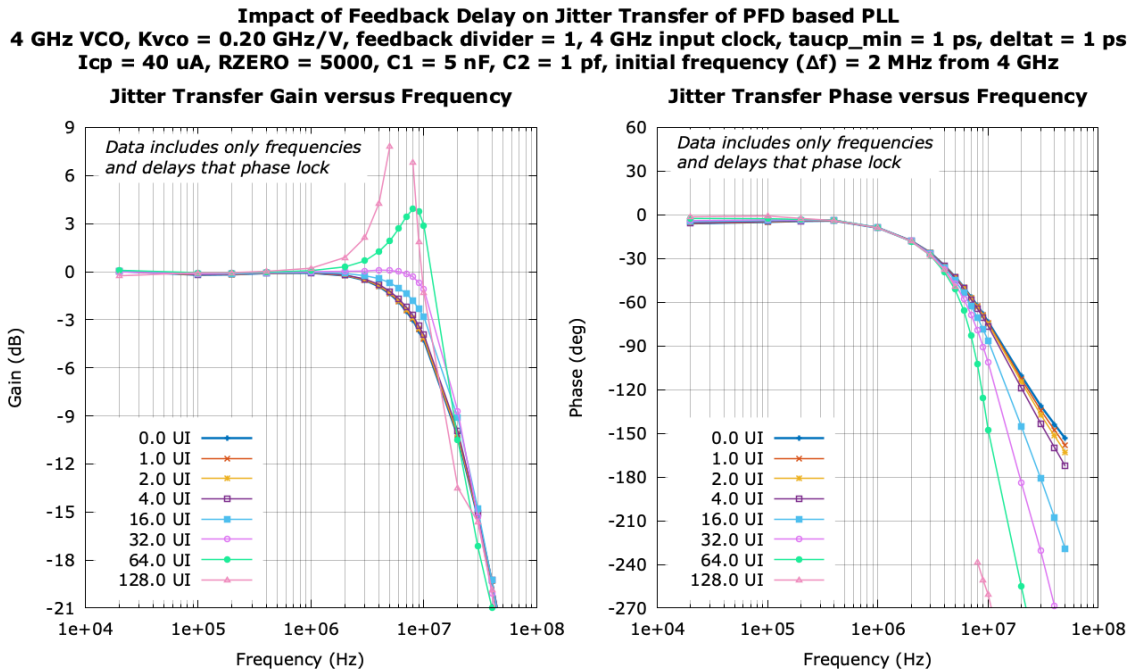
Impact of Input Jitter Amplitude on Jitter Transfer
Alexander phase detector based CDR, 4 GHz VCO, $K_{vco} = 0.20$ GHz/V, 4 Gbs, 100% transition density data
 $I_{cp} = 40$ uA, RZERO = 500 ohms, C1 = 5 nF, C2 = 1 pf, $\Delta V = 20$ mV from center voltage



6. Impact of Divider Latency on PLL Jitter Transfer

As an additional example of the capability of *trans_pll* to analyze large-signal effects in a phase-locked loop, the impact of finite feedback divider delay on the jitter transfer function is considered using a phase-frequency detector (PFD) based phase-locked loop. The 4 VCO within the phase-locked loop is followed by a unity divider and the divided 4 GHz frequency is presented as the input to the phase-frequency detector with the 4 GHz reference clock frequency. The delay of the frequency divider was varied from 0 UI (no delay) up to 128 UI, and the jitter transfer gain and phase for each are shown in Figure 34. As the delay increased above 16 UI, the jitter transfer gain includes significant peaking relative to lesser divider delays. As this is a large-signal effect, the presence of the jitter peaking is not present in a small-signal analysis of this phase-locked loop.

Figure 34



7. Examples of Spread Spectrum Clocking on Phase-locked Loop Responses

7.1 Introduction to Spread Spectrum Simulation Capability

trans_pll includes the ability to apply triangular frequency modulation to the input clock source to study the ability of a phase-locked loop to track the input clock in the presence of spread spectrum clocking (SSC). Parameters in its ASCII input file specify the start time for the added frequency modulation, the peak maximum and peak negative frequency modulation ¹, and the frequency of the modulation. Typically, spread spectrum frequency modulation rates fall between 30 kHz and 33 kHz.

Section 7.2 provides an example of the phase tracking ability of a 8.35 MHz loop bandwidth phase locked loop with a reference clock of 100 MHz and a 12 GHz VCO frequency to a 33 kHz spread spectrum clock with 5000 ppm down spreading².

Section 7.3 compares the phase tracking of the 8.35 MHz 100 MHz phase-locked loop with a 1.14 MHz 100 MHz phase locked loop to 33 kHz SSC modulation with down spreading between 1000 ppm and 16,000 ppm.

¹ The maximum positive and negative frequency deviations are expressed in parts per million (ppm) from the nominal frequency.

² Down spreading indicates that the input frequency peak positive modulation amplitude is 0 ppm.

Finally, Section 7.4 compares the phase tracking capability of a 100 MHz, 8.35 MHz loop bandwidth, phase-locked loop with SSC modulation of 0/-5000 ppm to SSC frequencies between 33 kHz and 400 kHz.

7.2 Phase Tracking Ability of a 100 MHz, 8.35 MHz Loop Bandwidth Phase-locked Loop to Spread Spectrum Clocking

The response of a 100 MHz phase-locked loop with a PFD and charge-pump based phase detector to a 5000 ppm down spread 33 kHz SSC was studied. The SSC was introduced into the 100 us simulation at 5 us. The input file for the 0/-5000 ppm SSC simulation is shown in Figure 35. The small signal and the 100 mUIpp large signal transfer functions of the phase locked loop with its loop parameters in Figure 36 and Figure 37. The large-signal transfer function differs from the small-signal response as the response of the charge-pump and loop filter slew rate is limited. Hence the large signal -3 dB bandwidth is limited to 8.35 MHz.

The large-signal gain and phase data results for input jitter frequencies greater than 50 MHz are not valid since the sample rate of the phase detector is 100 MHz.

Figure 38 and Figure 39 illustrate the output frequency of the phase-locked and the relative phase between the input clock and output clock. The phase of the output clock is within ± 0.25 mUI of the input clock.

Figure 35

Input File for 100 MHz, 8.35 MHz Loop Bandwidth, Phase-locked Loop Simulation with 0/-5000 ppm SSC

```
* Comment lines begin with a *
* Phase detector options, choose dff, pfd, pd_windowed, exor, none_vpd=1,
* none_vpd=0, hogge, alexander
*
phase_detector: pfd
*
* Phase detector related inputs (not all are used for each phase detector)
*
tauff(sec): 1E-11
tauh(sec): 1E-11
taucp_min(sec): 1E-12
pfd_deadzone(ui): 0
*
* pd_windowed phase detector related inputs
*
tw(sec): 3.00E-07
tacq_max(sec): 0.95
*
* CDR phase detector options
*
* Phase threshold beyond which amplitude of Alexander phase detector saturates
*
alexander_phase_detector_phase_threshold(UI): 0
*
* VCO related inputs
*
fo(VCO_freq_in_Hz): 1.20E+10
vco_clock_duty_cycle(fo): 0.4
*
* Choose model for control voltage characteristic
* polynomial or poly allows up to ninth order polynomial deltaf = c0 + c1*vc +c2*vc +...c8*vc
* tanh function requires four coefficients deltaf = c0 + c1*tanh(c2*(vc-c3))
*
vco_control_voltage_model: poly
vc_polynomial_coeff(hz/V): -0.036984,0.087021
*
* Example kvco = c1*fo = 2.0*500e6 = 1.0 GHz/V
* Vco = -(c0/c1) = -(-1/2) = 0.5 V
*
vco_bandwidth(Hz): 4.00E+08
*
* Clock or data input parameters
*
fin(input_freq_in_Hz): 1.00E+08
feedback_divider: 120
feedback_divider_delay(sec): 0.0
*
* Feedback clock duty cycle (not used if divider is 1.0)
*
feedback_divider_clock_duty_cycle: 0.1
*
* Input data pattern (single bit per line, text file)
*
filename_with_input_data_pattern: <Enter filename containing input data pattern>
*
* Enter type of noise on input (gaussian or uniform)
* random_noise_sigma is sigma for gaussian
* noise, and for uniform noise range of [-sigma,+sigma]
*
noise_type: gaussian
random_noise_sigma(UI_on_fin): 0
noise_bandwidth(Hz): 1.00E+09
*
* Add sinusoidal jitter or perform jitter transfer
*
jitter_amp(UIpp): 0
jitter_freqs(Hz): 0
*
* Initial phase of input
*
phase0(degrees): 0.0
```

Figure 35 (cont.)

Input File for 100 MHz, 8.35 MHz Loop Bandwidth, Phase-locked Loop Simulation with 0/-5000 ppm SSC

```
*
* Start and stop times of printed output, sinusoidal input jitter starts at TSTART_jt
*
TSTART(sec): 20e-06
TSTART_jt(sec): 2.00E-06
TSTOP(sec): 2.624242424242e-04
*
* Enter desired timestep or the word default
*
deltat(sec): default
*
* Enter desired lock detector time constant - enter 0 for no lock detector function
*
lock_detect_tau(sec): 100e-09
*
* Enter phase-lock detector thresholds (used if lock detector time constant > 0)
*
lock_detect_delta_vc_limit(%): default
lock_detect_rphase_pp_limit(UIpp): 1.0
lock_detect_rphase_pp_loss_lock_limit(UIpp): 1.5
*
* Enter desired number of points to save
*
number_of_print_points: 5001
*
* Inputs for phase or frequency step
*
phase_freq_step_time(sec): 1.0
phase_step(sec): 0.0
freq_step(ppm): 0.0
*
* Inputs for application of SSC
*
ssc_start_time(sec): 5.00E-06
max_pos_ssc(ppm): 0
max_neg_ssc(ppm): 5000
freq_ssc(Hz): 33000
*
* Phase detector and loop filter inputs
*
phase_detector_logic_hi(V): 600
phase_detector_logic_low(V): -600
signal_source_vs1(V): 0
signal_source_vs2(V): 0
power_supply_max(V): 0.85
power_supply_min(V): 0
rs1(ohms): 1E+12
rs2(ohms): 1E+12
R1(ohms): 1000000
RZERO(ohms): 12000
R3(ohms): 1
C1(farads): 2.407E-11
C2(farads): 3.6E-13
initvc(V): 0.2
*
* Output filename and format (csv only)
*
output_base_filename: 100meg_ref_ssc
*
* Optionally plot output characteristics and vco control voltage characteristics.
* Enter value of zero to omit all plots or any non-zero value to create plots.
* Note - plotting requires Octave or gnuplot be installed. The value of plot_outputs
* is forced to zero if one or the other programs is not installed
*
plot_outputs: 1
*
* Enter preferred plotting tool of octave or gnuplot
* Plots will be created using preferred plotting tool if
* the executable is found.
*
plot_preference: gnuplot
*
* End of trans_pll inputs
*
```

Figure 36

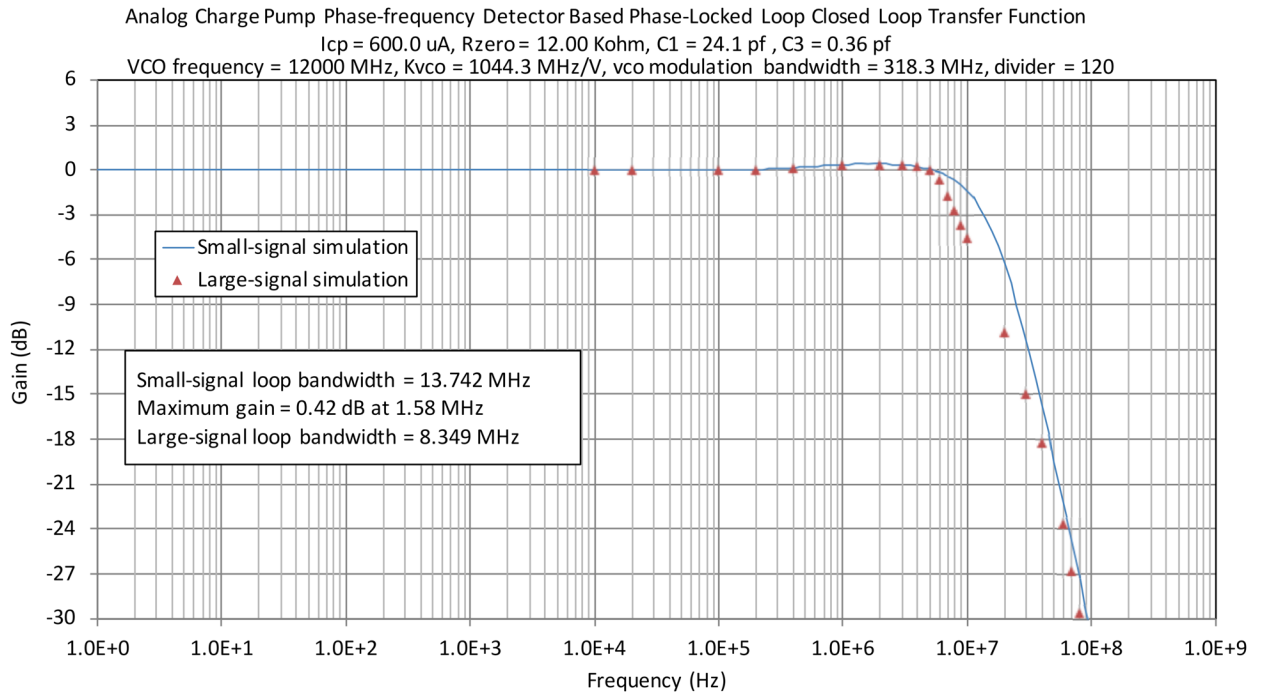


Figure 37

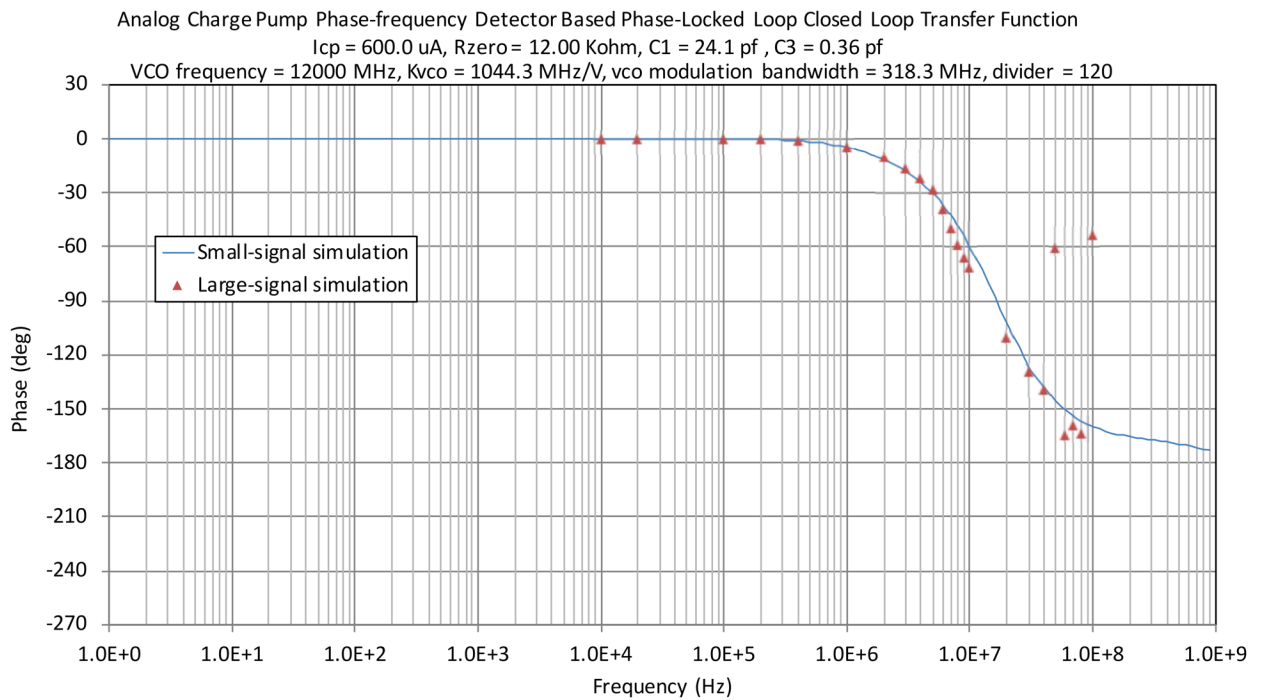


Figure 38

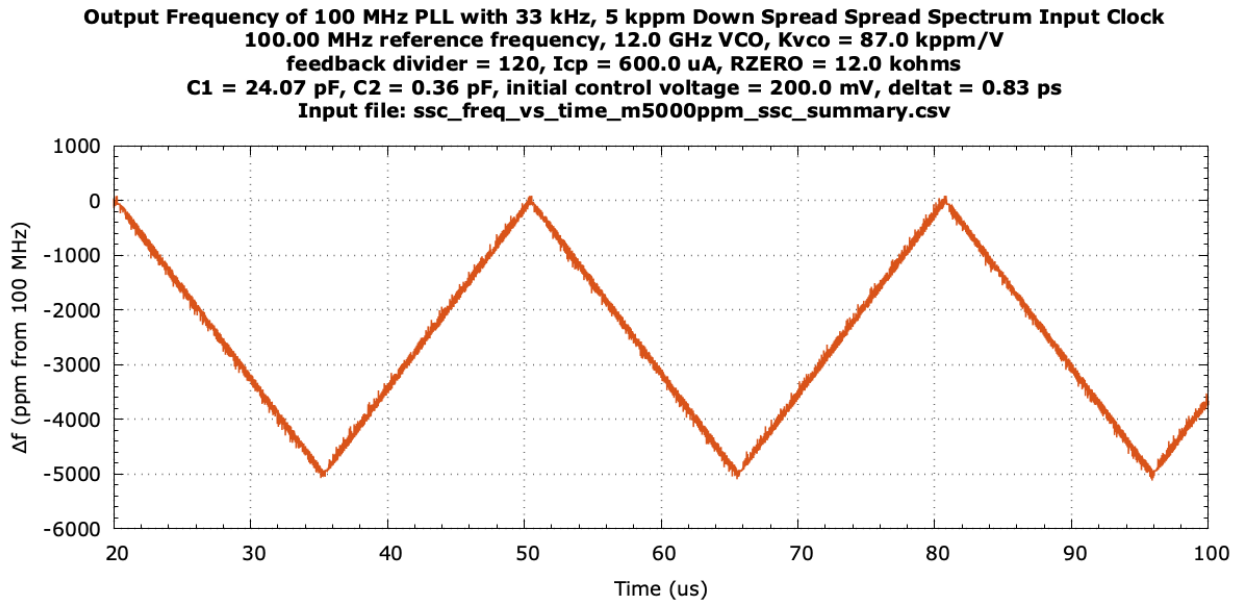
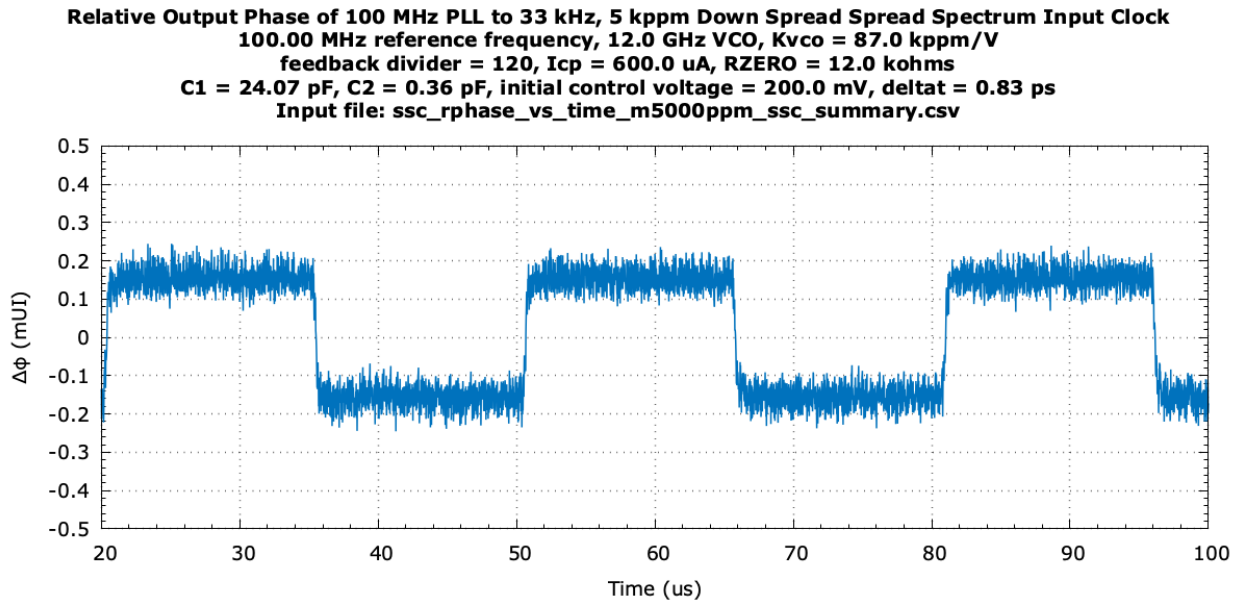


Figure 39

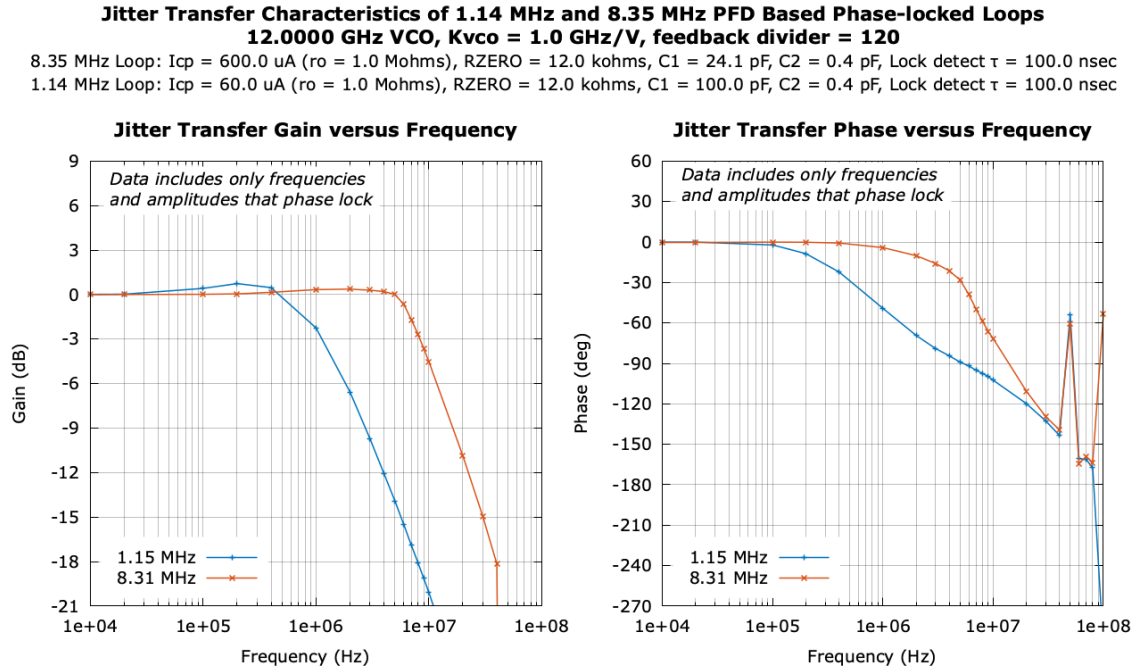


7.3 Phase Tracking Ability of Two 100 MHz Phase-locked Loops to various 33 kHz SSC Amplitudes

Using a down spread 33 kHz SSC input clock, the phase responses of a 1.14 MHz and the 8.35 MHz bandwidth phase locked loop were studied as a function of the maximum negative modulation amplitude. The large-signal transfer functions, simulated with a jitter amplitude of 100 mUIpp, for each phase-locked loop are compared in Figure 40.

SSC down spreading amplitudes between 0 ppm and 16,000 ppm were applied in a series of simulations. The SSC was introduced at 2 us in the 8.35 MHz loop bandwidth 100 us simulation. To allow for the longer settling time of the 1.14 MHz loop bandwidth, the SSC was initiated at 20 us in 120 us simulations.

Figure 40



The output frequency for each amplitude of SSC is shown as a function of time for the 8.35 MHz phase-locked loop in Figure 41, and the relative output phase in Figure 42. The output frequency and relative phase of the 1.14 MHz bandwidth phase-locked loop are provided in Figure 43 and Figure 44 respectively. compares the maximum peak-to-peak phase error for each amplitude of SSC spreading for both the 8.35 MHz and 1.14 MHz loop bandwidth phase-locked loops. It is evident that the ability of the 1 MHz bandwidth phase-locked loop to track the frequency modulation of the spread spectrum clock is significantly less than the 8.35 MHz loop bandwidth phase-locked loop. A direct comparison of the phase errors for each phase-locked loop as a function of the down spread amplitude is shown in Figure 45.

Figure 41

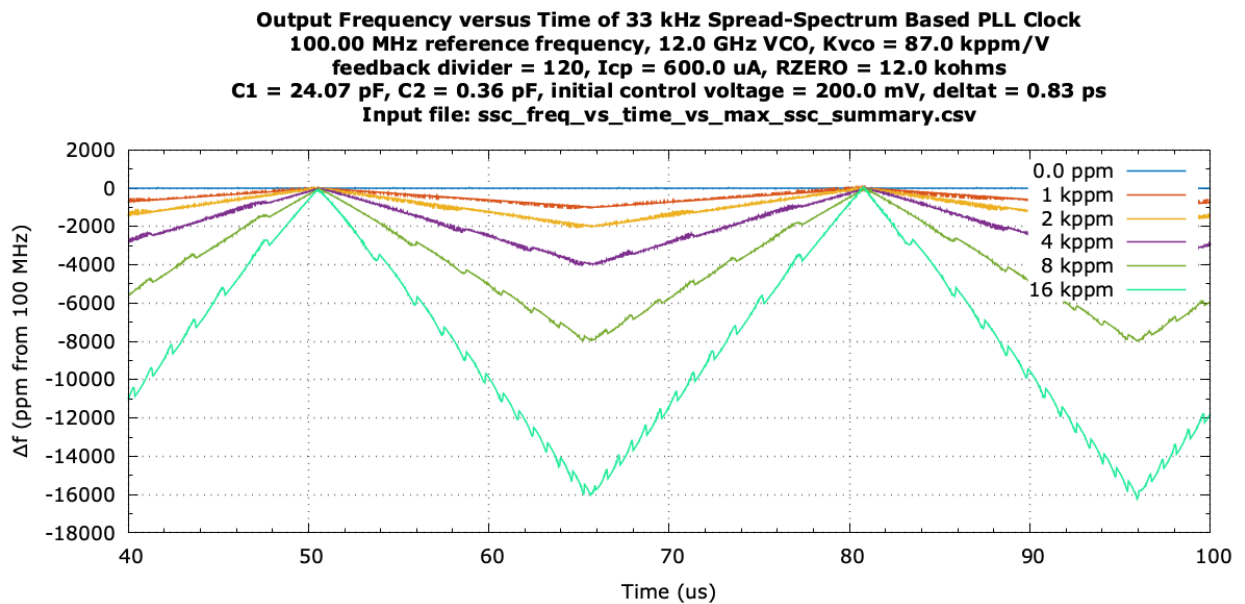


Figure 42

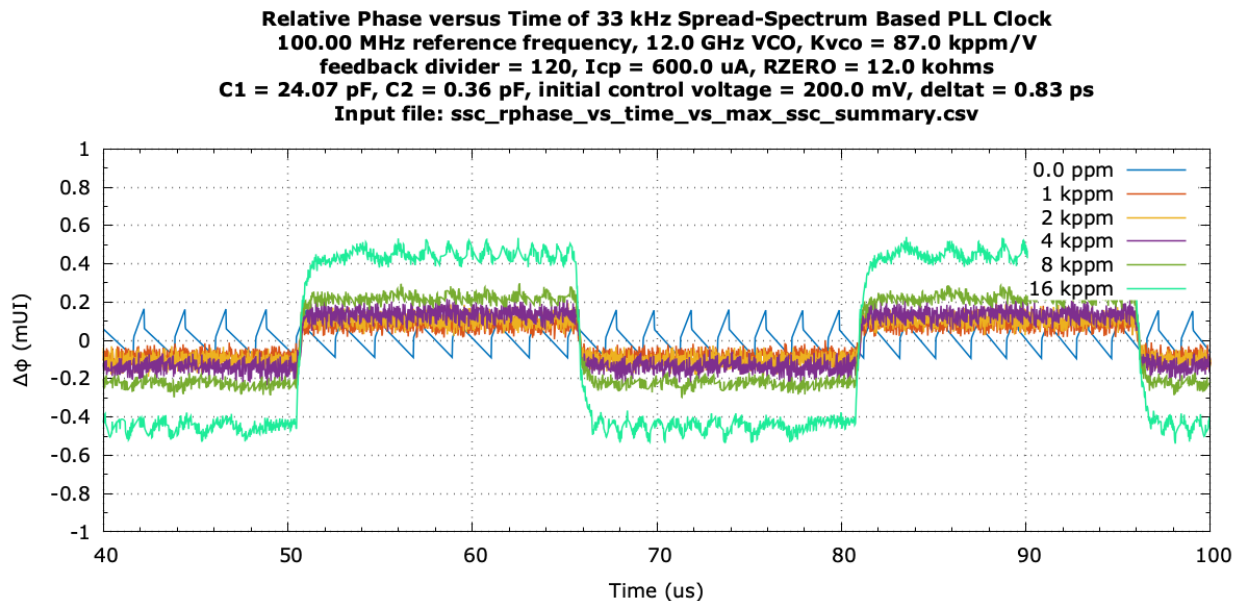


Figure 43

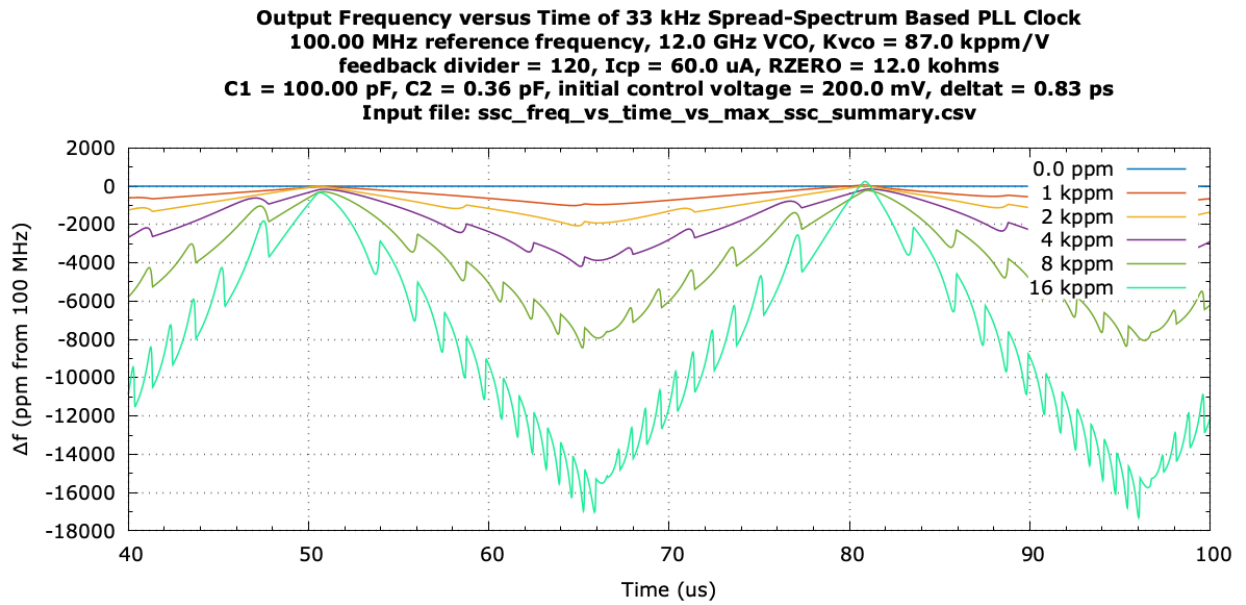


Figure 44

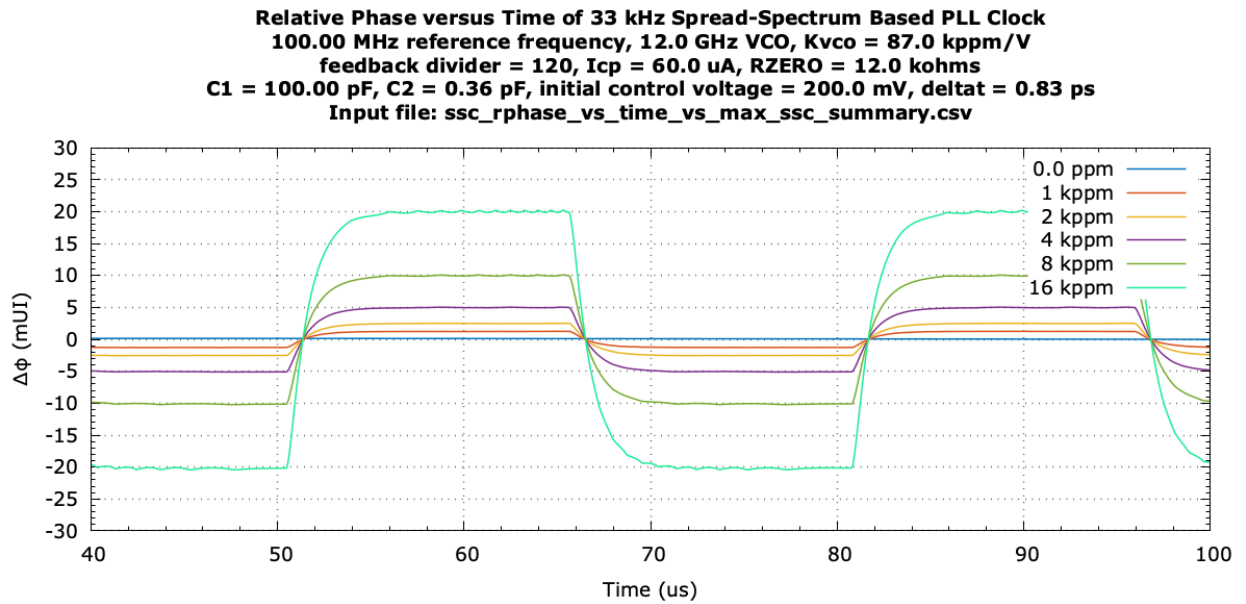
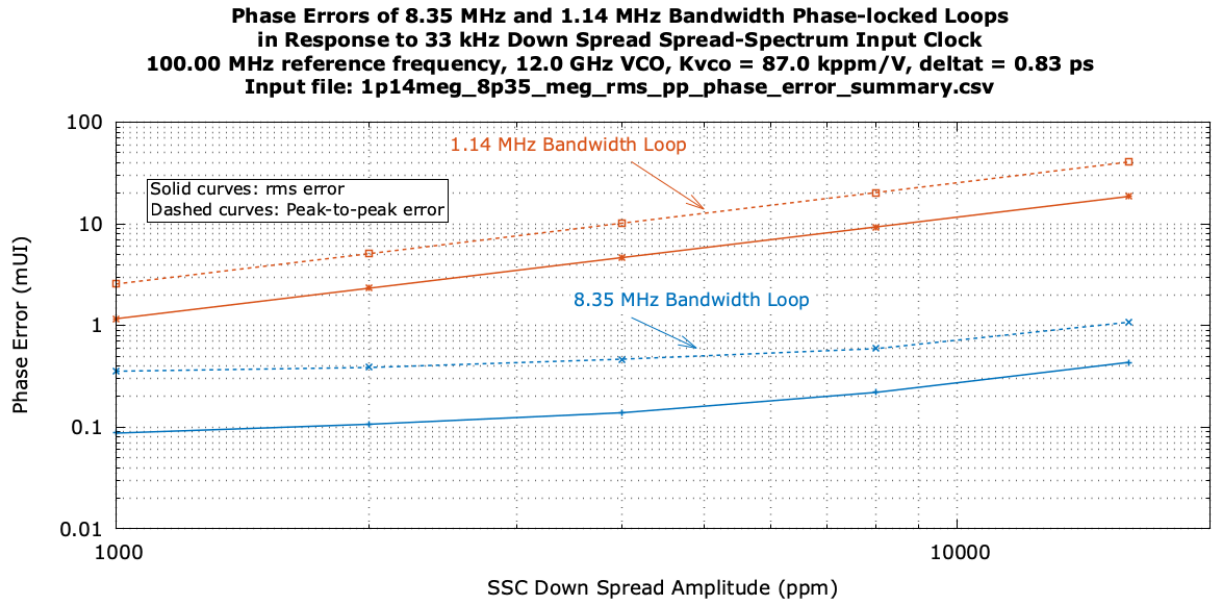


Figure 45



7.4 Impact of 0/-5000 ppm SSC Frequency on Tracking Ability of a 100 MHz, 8.35 MHz Bandwidth Phase-locked Loop

Using the parameters for the 100 MHz, 8.35 MHz loop bandwidth phase-locked loop provided in Section 7.2, and SSC modulation amplitudes of 0/-5000 ppm, the frequency of the applied SSC was increased from its nominal value of 33 kHz to 400 kHz to study its impact on phase tracking. The time of each simulation was set to 20 μ s plus 8 periods of the SSC frequency, and the SSC was introduced at 5 μ s. The output frequency of the phase-locked loop and the phase error between the phase-locked loop input and output clocks as a function of time normalized to the period of the SSC is provided in Figure 46 for SSC modulation rates between 33 kHz and 400 kHz. Figure 47 details the peak-to-peak phase error of each simulation as a function of the modulation rate. As the SSC modulation rate increases, the phase tracking error increases and the output frequency variation exhibits greater deviation from the triangular modulation of its input clock. The rms and peak-to-peak phase errors as a function of SSC frequency are illustrated in Figure 48.

Figure 46

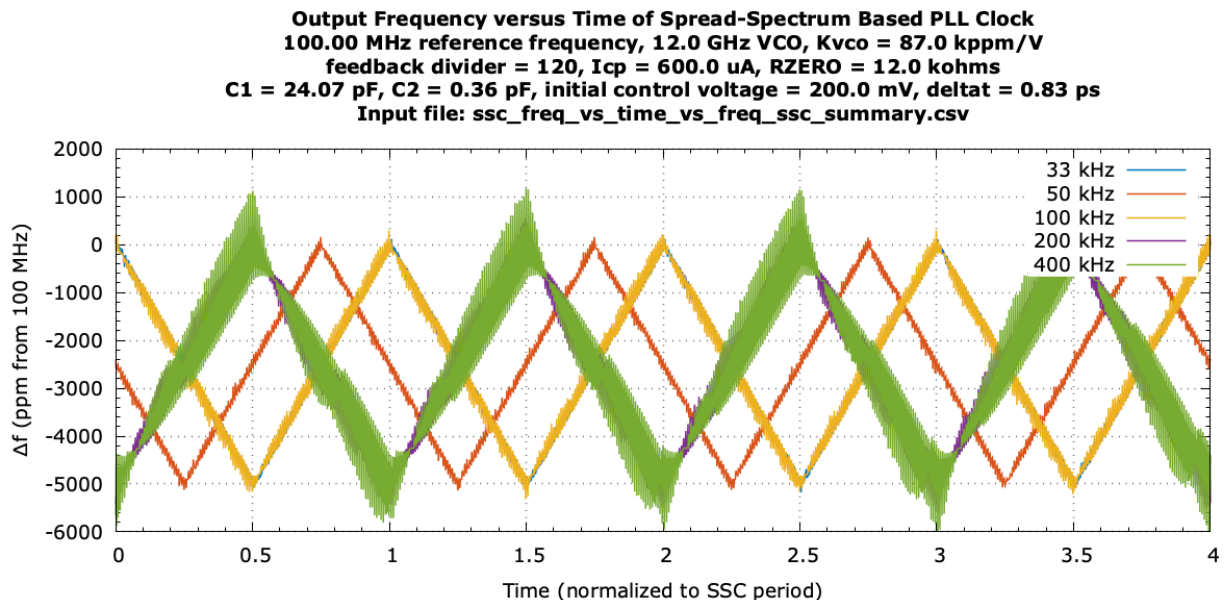


Figure 47

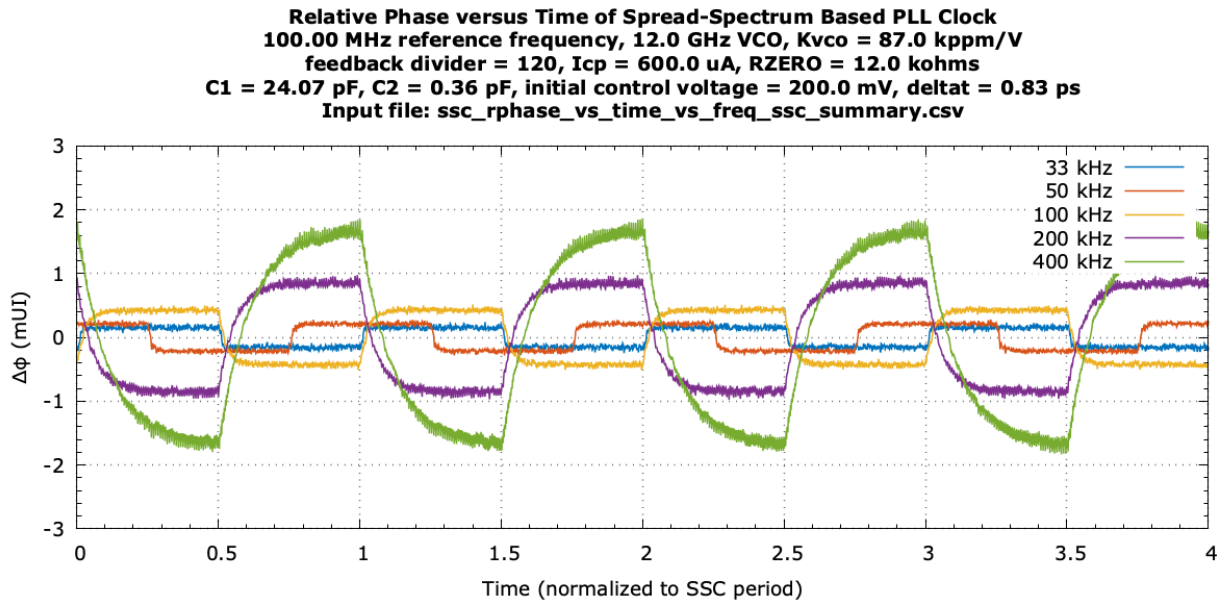
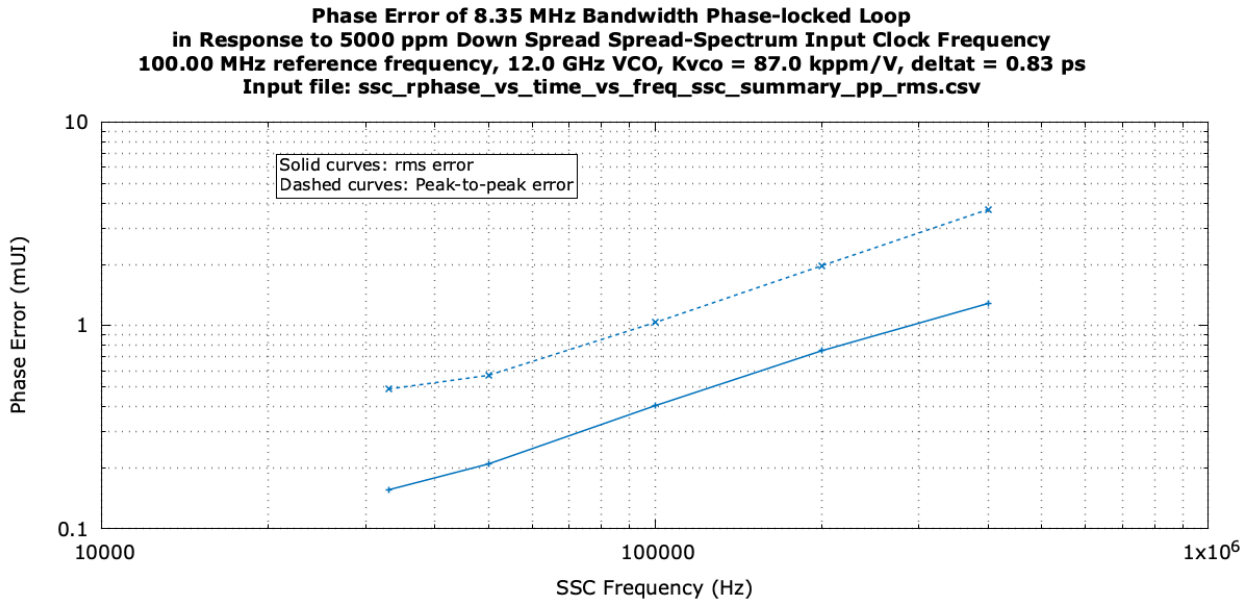


Figure 48

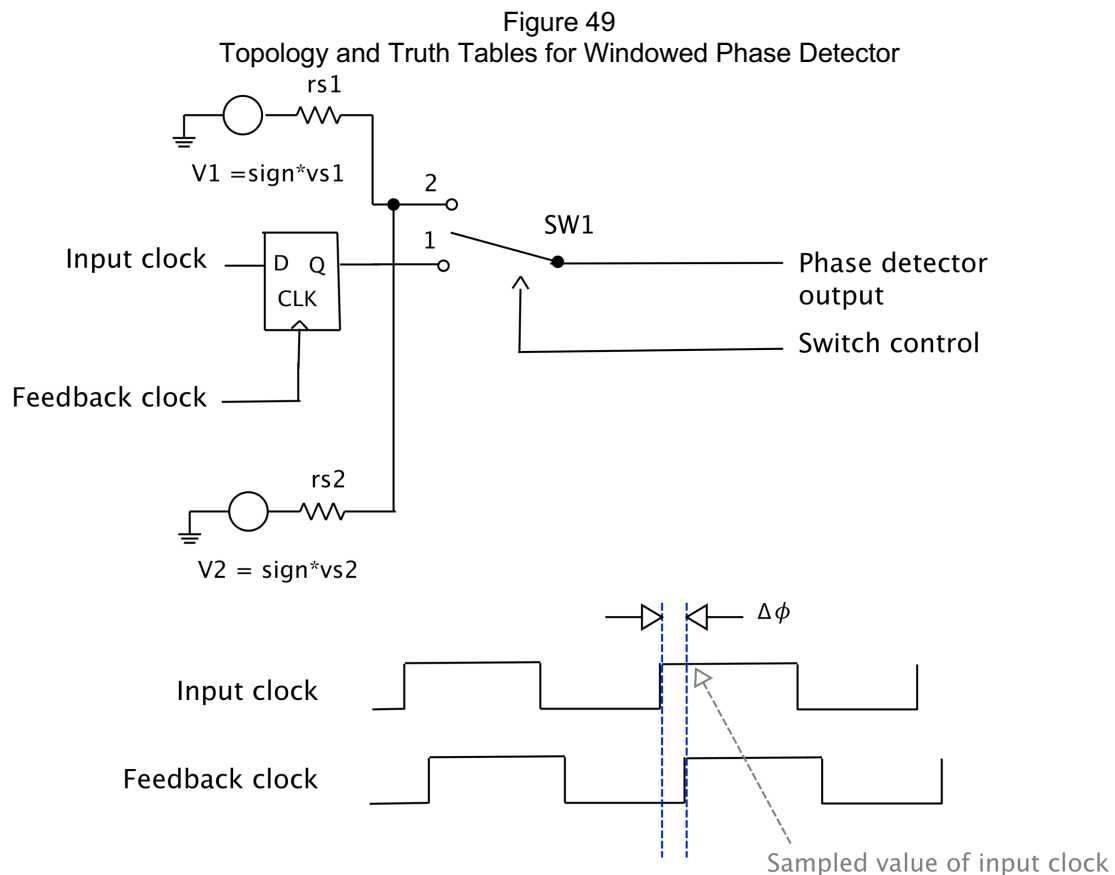


8. Windowed Phase Detector Phase-locked Loop Simulations

8.1 Description of Windowed Phase Detector

trans_pll includes a “windowed phase detector” as one of its clock recovery phase detectors. This type of phase detector is most useful in phase-locked loops where the comparison frequency is less than about 100 kHz or the VCO gain is very low (such as a quartz based VCO). The phase detector is also potential alternative to a phase-frequency detector when the input clock has occasional missing clock pulses. The latter will not tolerate missing clock pulses without inducing a significant phase disturbance.

When the loop is close to phase-lock, the phase detector utilizes the high gain of a D flip-flop phase detector. However, if the absolute value of the relative phase between the input clock and feedback clock exceeds its time window parameter tw , programmable current sources drive the control voltage in a direction towards phase-lock. If the sampled input clock is a logic high and the absolute value of the relative phase between the input clock and feedback clock exceeds tw , the current sources drive the control voltage lower. If the sampled value is a logic low, the current sources act in the opposite direction to increase the control voltage. The topology and its truth tables are shown in Figure 49.



	$ \Delta\phi \geq tw$	$ \Delta\phi < tw$
SW1 Position:	2	1 (DFF output)
Sampled value of input clock		
	logic low	logic high
sign:	+1	-1

The log file for a simulation using a windowed phase detector includes entries that indicate the state of the phase detector as the simulation progresses.

The current source values used when the absolute value of the phase between input and feedback clock exceeds t_w are programmable through the input ASCII file voltage source/resistor pairs vs1/rs1 and vs2/rs2. Both vs1 and vs2 must be positive valued voltage sources.

8.2 Windowed Phase Detector Simulations

8.2.1 Phase-locked Loop Acquisition Simulation

The ASCII input file for an 8 kHz phase-locked loop with a windowed phase detector is provided in Figure 50 with the resulting output log in Figure 51. The log file contains messages indicating the mode of the windowed phase detector. After about 235 ms, the phase detector remains in its high gain DFF state as the phase between the input clock and feedback clock are less the value chosen for t_w (250 ns or 2 mUI). The control voltage and phase simulation results are shown in Figure 52 and Figure 53 respectively. The time at which the phase detector changes from its current slew mode to its DFF mode is evident at about 235 ms in Figure 52. Phase-lock with the phase detector in its DFF mode is declared at 310 ms.

8.2.2 Impact of Initial Clock Phase and initial Control Voltage on Acquisition Time

The input ASCII file for this simulation sets the initial input clock phase to 110 degrees and the initial control voltage to 0.0. To better illustrate the operational modes of the phase detector, two additional sets of simulations were conducted. In each set, the initial clock phase is varied from -180 degrees to 120 degrees in 60 degree steps. In the first set of simulations the initial control voltage is set to 0.0 V, and in the second set the initial control voltage is set to the maximum supply voltage of 3.30 V. The control voltage and relative phase for each set of simulations are shown in Figure 54 through Figure 57. The period of time the phase detector remains in its slew-rate mode is a strong function of both the initial phase of the input clock and the initial control voltage relative the center voltage of the VCXO.

Figure 50
Input File for 8 kHz Windowed Phase Detector Based Phase-locked Loop Simulation

```
* Comment lines begin with a *
* Phase detector options, choose dff, pfd, pd_windowed, exor, none_vpd=1,
* none_vpd=0, hogge, alexander
*
phase_detector: pd_windowed
*
* Phase detector related inputs (not all are used for each phase detector)
*
tauff(sec): 0.00000001
tauh(sec): 2.5E-10
taucp_min(sec): 1E-10
pfd_deadzone(ui): 0
*
* pd_windowed phase detector related inputs
*
tw(sec): 0.00000025
tacq_max(sec): 0.001
*
* CDR phase detector options
*
* Phase threshold beyond which amplitude of Alexander phase detector saturates
*
alexander_phase_detector_phase_threshold(UI): 0
*
* VCO related inputs
*
fo(VCO_freq_in_Hz): 1544000
vco_clock_duty_cycle(fo): 0.5
*
* Choose model for control voltage characteristic
* polynomial or poly allows up to ninth order polynomial  $\text{delta}f = c_0 + c_1*vc + c_2*vc + \dots c_8*vc$ :  $\text{delta}f = c_0 + c_1*vc + c_2*vc + \dots c_8*vc$ 
* tanh function requires four coefficients  $\text{delta}f = c_0 + c_1*\tanh(c_2*(vc-c_3))$ :  $\text{delta}f = c_0 + c_1*\tanh(c_2*(vc-c_3))$ 
*
vco_control_voltage_model: poly
vc_polynomial_coeff(hz/V): -0.000165,0.000100
*
* Example  $k_{vco} = c_1*fo = 2.0*500e6 = 1.0 \text{ GHz/V}$ :  $k_{vco} = c_1*fo = 2.0*500e6 = 1.0 \text{ GHz/V}$ 
*  $V_{co} = -(c_0/c_1) = -(-1/2) = 0.5 \text{ V}$ 
*
vco_bandwidth(Hz): 8000
*
* Clock or data input parameters
*
fin(input_freq_in_Hz): 8000
feedback_divider: 193
feedback_divider_delay(sec): 0
*
* Feedback clock duty cycle (not used if divider is 1.0)
*
feedback_divider_clock_duty_cycle: 0.10
*
* Input data pattern (single bit per line, text file)
*
filename_with_input_data_pattern
*
* Enter type of noise on input (gaussian or uniform)
* random_noise_sigma is sigma for gaussian
* noise, and for uniform noise range of [-sigma,+sigma]
*
noise_type: gaussian
random_noise_sigma(UI_on_fin): 0
noise_bandwidth(Hz): 1000000000
*
* Add sinusoidal jitter or perform jitter transfer
*
jitter_amp(UIpp): 0
jitter_freqs(Hz): 0
*
* Initial phase of input
*
phase0(degrees): 110.0
```


Figure 50 (cont.)

Input File for 8 kHz Windowed Phase Detector Based Phase-locked Loop Simulation

```
*
* Start and stop times of printed output, sinusoidal input jitter starts at TSTART_jt
*
TSTART(sec): 0
TSTART_jt(sec): 0.5
TSTOP(sec): 1.0
*
* Enter desired timestep or the word default
*
deltat(sec): default
*
* Enter desired lock detector time constant - enter 0 for no lock detector function
*
lock_detect_tau(sec): 0.01
*
* Enter phase-lock detector thresholds (used if lock detector time constant > 0)
*
lock_detect_delta_vc_limit(%): default
lock_detect_rphase_pp_limit(UIpp): default
lock_detect_rphase_pp_loss_lock_limit(UIpp): default
*
* Enter desired number of points to save
*
number_of_print_points: 50000
*
* Inputs for phase or frequency step
*
phase_freq_step_time(sec): 100
phase_step(sec): 0
freq_step(ppm): 0
*
* Inputs for application of SSC
*
ssc_start_time(sec): 100
max_pos_ssc(ppm): 0
max_neg_ssc(ppm): 0
freq_ssc(Hz): 0
*
* Phase detector and loop filter inputs
*
phase_detector_logic_hi(V): 3.3
phase_detector_logic_low(V): 0
signal_source_vs1(V): 3.3
signal_source_vs2(V): 3.3
power_supply_max(V): 3.3
power_supply_min(V): 0
rs1(ohms): 10000
rs2(ohms): 10000
R1(ohms): 392000
RZERO(ohms): 12000
R3(ohms): 1
C1(farads): 9.4E-09
C2(farads): 4.7E-09
initvc(V): 0
*
* Output filename and format (csv only)
*
output_base_filename: pd_windowed_8_kHz_loop
*
* Optionally plot output characteristics and vco control voltage characteristics.
* Enter value of zero to omit all plots or any non-zero value to create plots.
* Note - plotting requires Octave or gnuplot be installed. The value of plot_outputs
* is forced to zero if one or the other programs is not installed
*
plot_outputs: 1
*
* Enter preferred plotting tool of octave or gnuplot
* Plots will be created using preferred plotting tool if
* the executable is found.
*
plot_preference: gnuplot
*
* End of trans_pll inputs
*
```

Figure 51
Example of a *trans_pll* 8 kHz Windowed Based Phase Detector Phase-locked Loop Log File

```
trans_pll v1.76 7/18/2026

Input filename: pd_windowed_8kHz_loop_input_params_example.txt
Output filename: pd_windowed_8_kHz_loop_071826_18_14_13.csv
Value of deltat set by period of VCO.
Value of deltat is 6.48e-09
Data will be printed every 20.0004 us.
Sample signals every 1544000 time points to determine lock status (10.000 ms)
Lock detect thresholds:
    Control voltage tolerance about center voltage for phase-lock: 100.0 m%
    Relative phase tolerance for phase-lock: 150.0 mUI
    Relative phase threshold to declare loss of phase-lock: 300.0 mUI
Using polynomial of degree 1 to approximate VCO transfer curve.
Assuming a deadzone of 0.000e+00 UI with fin = 8.000000e+03 Hz.
Time = 0.000 s   0 % complete, Vc = 0.000 V
    Averages: vc = 0.000 V (deltaV = -100.00 %), relative phase = 305.556 mUI (0.000 UIpp).
Windowed phase detector using current source based phase detector at time = 1.000000e-03.
Relative phase is less than detector window of 2.000 mUI at time = 1.000000e-03.
Time = 100.000 ms  10 % complete, Vc = 3.300 V
    Averages: vc = 3.300 V (deltaV = 100.00 %), relative phase = -182.730 mUI (13.200 mUIpp).
Time = 200.000 ms  20 % complete, Vc = 3.300 V
    Averages: vc = 3.300 V (deltaV = 100.00 %), relative phase = -50.729 mUI (13.200 mUIpp).
Relative phase is within detector window of 2.000 mUI at time = 2.319171e-01.
Windowed phase detector using DFF based phase detector.
Windowed phase detector using current source based phase detector at time = 2.356646e-01.
Relative phase is greater than detector window of 2.000 mUI at time = 2.356646e-01.
Relative phase is within detector window of 2.000 mUI at time = 2.356983e-01.
Windowed phase detector using DFF based phase detector.
Time = 300.000 ms  30 % complete, Vc = 1.520 V
    Averages: vc = 1.648 V (deltaV = -147.00 m%), relative phase = 42.924 uUI (67.260 uUIpp).
Lock detected at time 310.0000 ms.
Time = 400.000 ms  40 % complete, Vc = 1.764 V
    Averages: vc = 1.649 V (deltaV = -90.08 m%), relative phase = 39.516 uUI (100.557 uUIpp).
Time = 500.000 ms  50 % complete, Vc = 1.529 V
    Averages: vc = 1.650 V (deltaV = -683.83 u%), relative phase = 47.008 uUI (101.633 uUIpp).
Time = 600.000 ms  60 % complete, Vc = 1.675 V
    Averages: vc = 1.651 V (deltaV = 38.79 m%), relative phase = 42.881 uUI (101.391 uUIpp).
Time = 700.000 ms  70 % complete, Vc = 1.534 V
    Averages: vc = 1.656 V (deltaV = 340.69 m%), relative phase = 39.681 uUI (66.384 uUIpp).
Time = 800.000 ms  80 % complete, Vc = 1.518 V
    Averages: vc = 1.652 V (deltaV = 123.06 m%), relative phase = 39.233 uUI (66.383 uUIpp).
Time = 900.000 ms  90 % complete, Vc = 1.676 V
    Averages: vc = 1.652 V (deltaV = 109.38 m%), relative phase = 33.891 uUI (99.896 uUIpp).
Time = 1000.000 ms 100 % complete, Vc = 1.767 V
    Averages: vc = 1.647 V (deltaV = -167.93 m%), relative phase = 28.026 uUI (71.943 uUIpp).
Maximum CLKIN duty cycle = 50.00 % ( $\pm$  5.2 m%), Minimum CLKIN duty cycle = 50.00 % ( $\pm$  5.2 m%)
(difference of 449.2 n%)
Done! Last phase-lock event observed at about 310.0000 ms.
Elapsed time: 49.286406 seconds
```

Figure 52

Control Voltage versus Time
 1.54 MHz VCO, $K_{vco} = 154.40 \text{ Hz/V}$, divider = 193, divider delay = 0.0 s
 RZERO = 12.00 kohms, $R_2 = 1.0 \text{ ohms}$, $C_1 = 9.40 \text{ nf}$, $C_2 = 4.70 \text{ nf}$
 $I_{cp} = 8.4 \text{ uA}$, $I_{cn} = 0.0 \text{ A}$ ($R_1 = 0.39 \text{ Mohms}$), leakage current = -330.55 uA
 Slew rate detector: $IS_1 = 330.00 \text{ uA}$ ($RS_1 = 10.00 \text{ kohms}$), $IS_2 = 330.00 \text{ uA}$ ($RS_2 = 10.00 \text{ kohms}$)

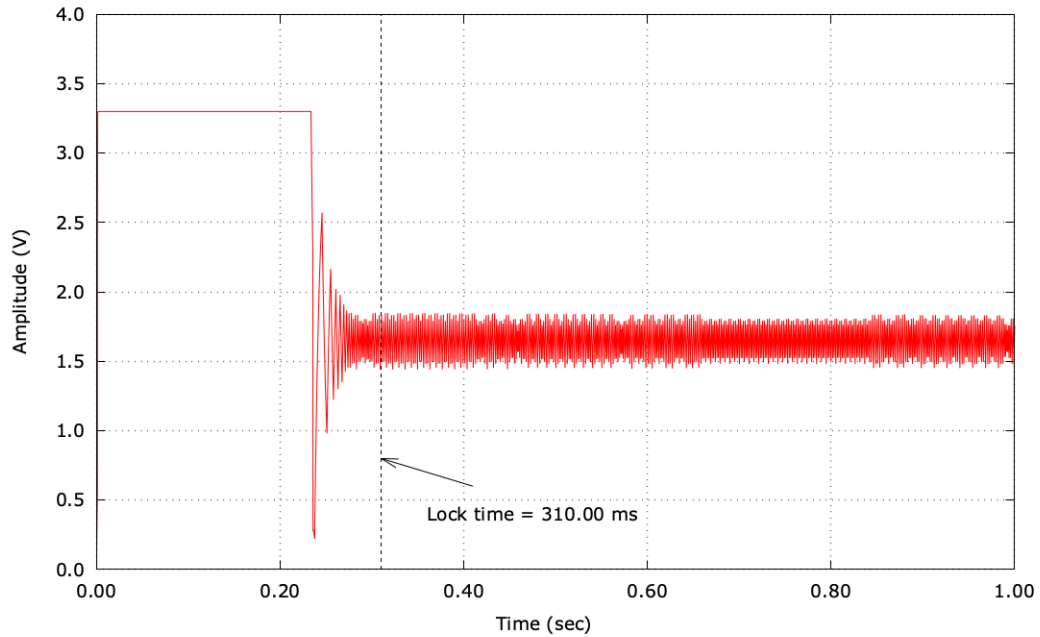


Figure 53

Input and Output Phases versus Time
 1.54 MHz VCO, $K_{vco} = 154.40 \text{ Hz/V}$, divider = 193, divider delay = 0.0 s
 RZERO = 12.00 kohms, $R_2 = 1.0 \text{ ohms}$, $C_1 = 9.40 \text{ nf}$, $C_2 = 4.70 \text{ nf}$
 $I_{cp} = 8.4 \text{ uA}$, $I_{cn} = 0.0 \text{ A}$ ($R_1 = 0.39 \text{ Mohms}$), leakage current = -330.55 uA
 Slew rate detector: $IS_1 = 330.00 \text{ uA}$ ($RS_1 = 10.00 \text{ kohms}$), $IS_2 = 330.00 \text{ uA}$ ($RS_2 = 10.00 \text{ kohms}$)

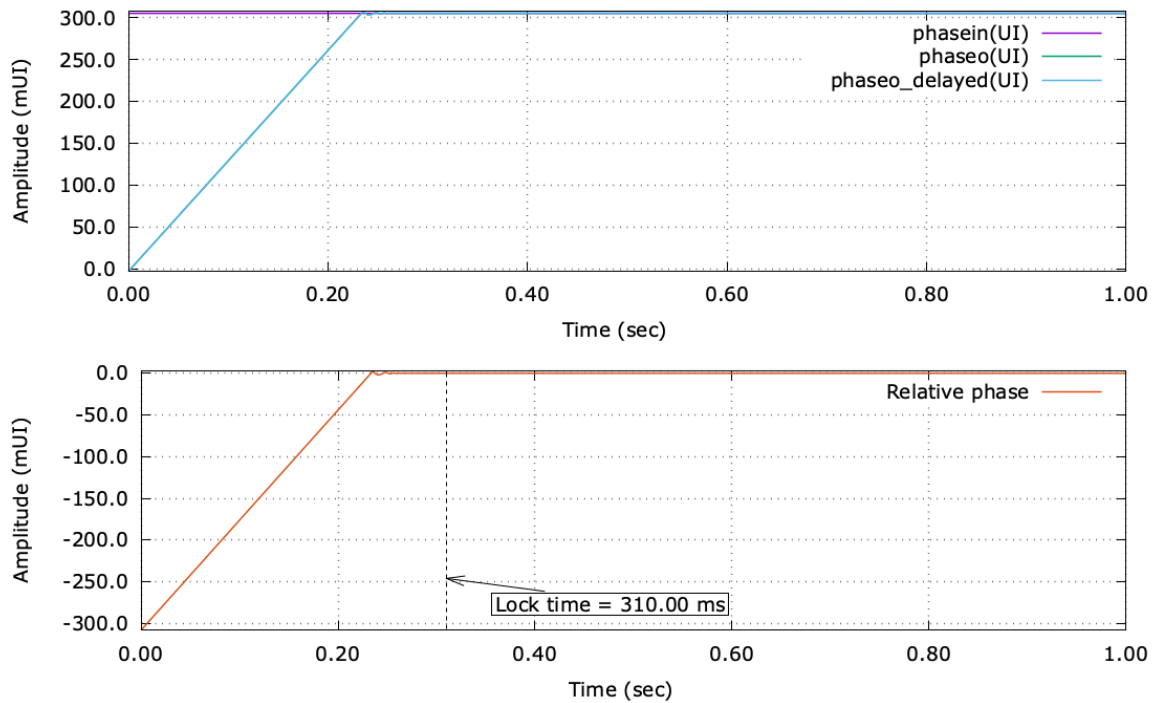


Figure 54

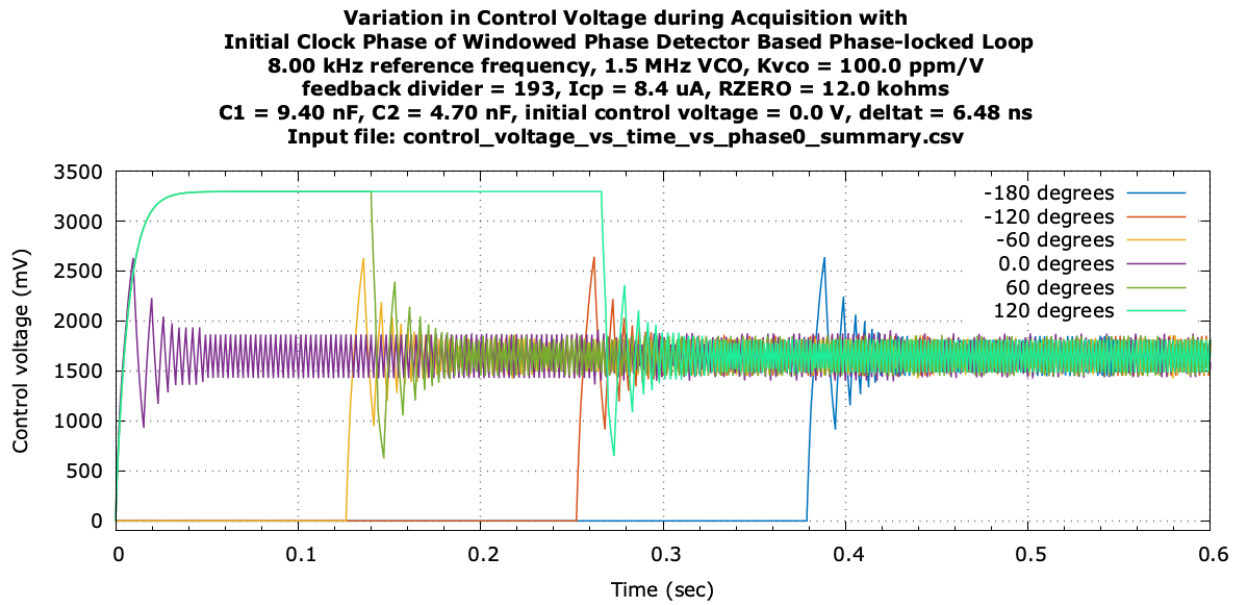


Figure 55

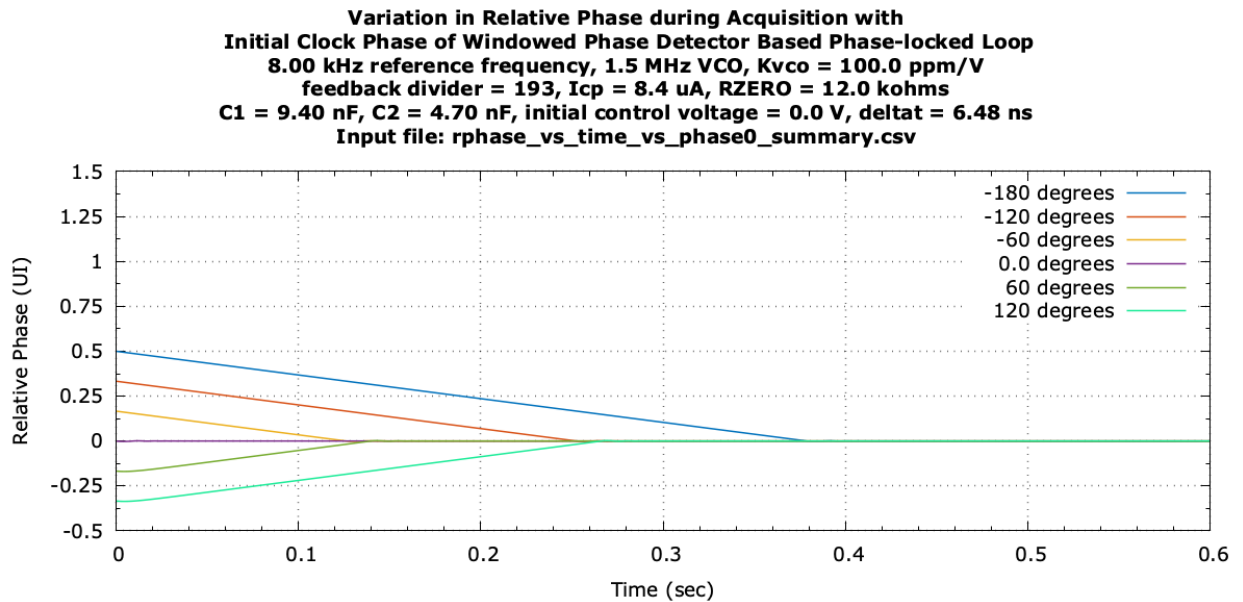


Figure 56

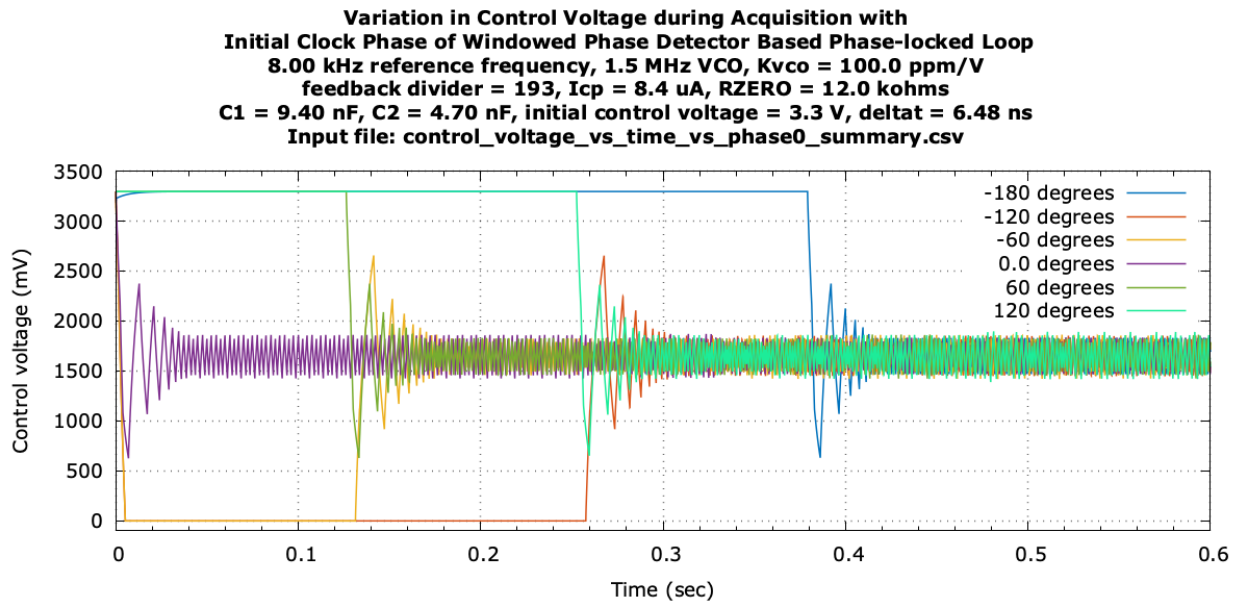
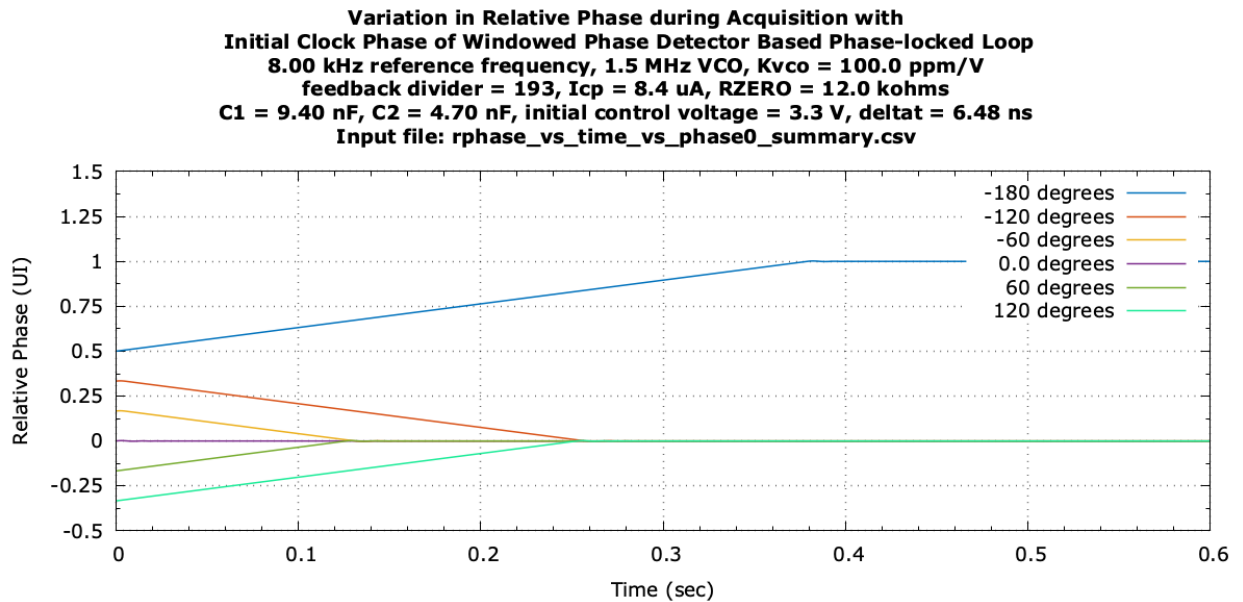


Figure 57



8.2.3 Impact of Jitter Amplitude on Jitter Transfer

Finally, a set of jitter transfer simulations as a function of input jitter amplitude was performed using the ASCII input file parameters shown in Figure 58. The input jitter amplitude is set to the variable "my_jitter_amp_ui_pp" and was varied between 50 mUIpp and 400 mUIpp. For amplitudes and frequencies of sinusoidal jitter where the absolute value of the relative phase remains within the phase detector time window t_w , the jitter transfer characteristic will appear as that for a DFF phase detector. However, when the absolute value of the relative phase exceeds t_w , the phase detector DFF is disabled, and the phase detector output is slew-rate limited by the current sources set by ASCII input file parameters $vs1/rs1$ and $vs2/rs2$. The windowed phase detector gain will reflect the high gain of its DFF phase detector when the absolute value of the relative phase is less than t_w . As a result, the -3 dB bandwidth of the phase-locked loop decreases as the portion of time the slew-rate phase detector is active increases relative to the DFF phase detector.

Figure 59 compares the jitter transfer characteristic for applied jitter amplitudes between 50 mUIpp and 400 mUIpp. As expected, the -3 dB bandwidth decreases with increasing jitter amplitude.

Figure 58
Input File for 8 kHz Windowed Phase Detector Based Phase-locked Loop Simulation

```
* Comment lines begin with a *
* Phase detector options, choose dff, pfd, pd_windowed, exor, none_vpd=1,
* none_vpd=0, hogge, alexander
*
phase_detector: pd_windowed
*
* Phase detector related inputs (not all are used for each phase detector)
*
tauff(sec): 0.00000001
tauh(sec): 2.5E-10
taucp_min(sec): 1E-10
pfd_deadzone(ui): 0
*
* pd_windowed phase detector related inputs
*
tw(sec): 0.00000025
tacq_max(sec): 0.001
*
* CDR phase detector options
*
* Phase threshold beyond which amplitude of Alexander phase detector saturates
*
alexander_phase_detector_phase_threshold(UI): 0
*
* VCO related inputs
*
fo(VCO_freq_in_Hz): 1544000
vco_clock_duty_cycle(fo): 0.5
*
* Choose model for control voltage characteristic
* polynomial or poly allows up to ninth order polynomial  $\text{delta}f = c_0 + c_1*vc + c_2*vc + \dots c_8*vc$ :  $\text{delta}f = c_0 + c_1*vc + c_2*vc + \dots c_8*vc$ 
* tanh function requires four coefficients  $\text{delta}f = c_0 + c_1*\tanh(c_2*(vc-c_3))$ :  $\text{delta}f = c_0 + c_1*\tanh(c_2*(vc-c_3))$ 
*
vco_control_voltage_model: poly
vc_polynomial_coeff(hz/V): -0.000165,0.000100
*
* Example  $k_{vco} = c_1*fo = 2.0*500e6 = 1.0 \text{ GHz/V}$ :  $k_{vco} = c_1*fo = 2.0*500e6 = 1.0 \text{ GHz/V}$ 
*  $V_{co} = -(c_0/c_1) = -(-1/2) = 0.5 \text{ V}$ 
*
vco_bandwidth(Hz): 8000
*
* Clock or data input parameters
*
fin(input_freq_in_Hz): 8000
feedback_divider: 193
feedback_divider_delay(sec): 0
*
* Feedback clock duty cycle (not used if divider is 1.0)
*
feedback_divider_clock_duty_cycle: 0.10
*
* Input data pattern (single bit per line, text file)
*
filename_with_input_data_pattern
*
* Enter type of noise on input (gaussian or uniform)
* random_noise_sigma is sigma for gaussian
* noise, and for uniform noise range of [-sigma,+sigma]
*
noise_type: gaussian
random_noise_sigma(UI_on_fin): 0
noise_bandwidth(Hz): 1000000000
*
* Add sinusoidal jitter or perform jitter transfer
*
jitter_amp(UIpp): my_jitter_amp_ui_pp
jitter_freqs(Hz):
100.0,80.0,40.0,20.0,10.0,8.0,4.0,2.0,1.0,0.80,0.75,0.70,0.65,0.60,0.55,0.50,0.45,0.40,0.35,0.30,0.25,0.20,0.10,0.08,
0.04,0.02,0.01
```

Figure 58 (cont.)

Input File for 8 kHz Windowed Phase Detector Based Phase-locked Loop Simulation

```
* Initial phase of input
*
phase0(degrees): 110.0
*
* Start and stop times of printed output, sinusoidal input jitter starts at TSTART_jt
*
TSTART(sec): 0
TSTART_jt(sec): 0.5
TSTOP(sec): 1.0
*
* Enter desired timestep or the word default
*
deltat(sec): default
*
* Enter desired lock detector time constant - enter 0 for no lock detector function
*
lock_detect_tau(sec): 0.01
*
* Enter phase-lock detector thresholds (used if lock detector time constant > 0)
*
lock_detect_delta_vc_limit(%): default
lock_detect_rphase_pp_limit(UIpp): 1.0
lock_detect_rphase_pp_loss_lock_limit(UIpp): 1.5
*
* Enter desired number of points to save
*
number_of_print_points: 50000
*
* Inputs for phase or frequency step
*
phase_freq_step_time(sec): 100
phase_step(sec): 0
freq_step(ppm): 0
*
* Inputs for application of SSC
*
ssc_start_time(sec): 100
max_pos_ssc(ppm): 0
max_neg_ssc(ppm): 0
freq_ssc(Hz): 0
*
* Phase detector and loop filter inputs
*
phase_detector_logic_hi(V): 3.3
phase_detector_logic_low(V): 0
signal_source_vs1(v): 3.3
signal_source_vs2(v): 3.3
power_supply_max(V): 3.3
power_supply_min(V): 0
rs1(ohms): 10000
rs2(ohms): 10000
R1(ohms): 392000
RZERO(ohms): 12000
R3(ohms): 1
C1(farads): 9.4E-09
C2(farads): 4.7E-09
initvc(V): 0
*
* Output filename and format (csv only)
*
output_base_filename: pd_windowed_8_kHz_loop
*
* Optionally plot output characteristics and vco control voltage characteristics.
* Enter value of zero to omit all plots or any non-zero value to create plots.
* Note - plotting requires Octave or gnuplot be installed. The value of plot_outputs
* is forced to zero if one or the other programs is not installed
*
plot_outputs: 1
*
* Enter preferred plotting tool of octave or gnuplot
* Plots will be created using preferred plotting tool if
* the executable is found.
*
plot_preference: gnuplot
```


Figure 59

Impact of Jitter Amplitude on Jitter Transfer of Windowed Phase Detector Based Phase-locked Loop
1.5440 MHz VCO, Kvco = 154.40 Hz/V, feedback divider = 193
vs1 = 3.3 V, rs1 = 10.0 kohms, vs2 = 3.3 V, rs2 = 10.0 kohms, tw = 250.0 nsec (2.0 mUI)
VDD = 3.3 V, R1 = 392.0 kohms, RZERO = 12.0 kohms, C1 = 9.4 nF, C2 = 4.7 nF, Lock detect τ = 10.0 msec

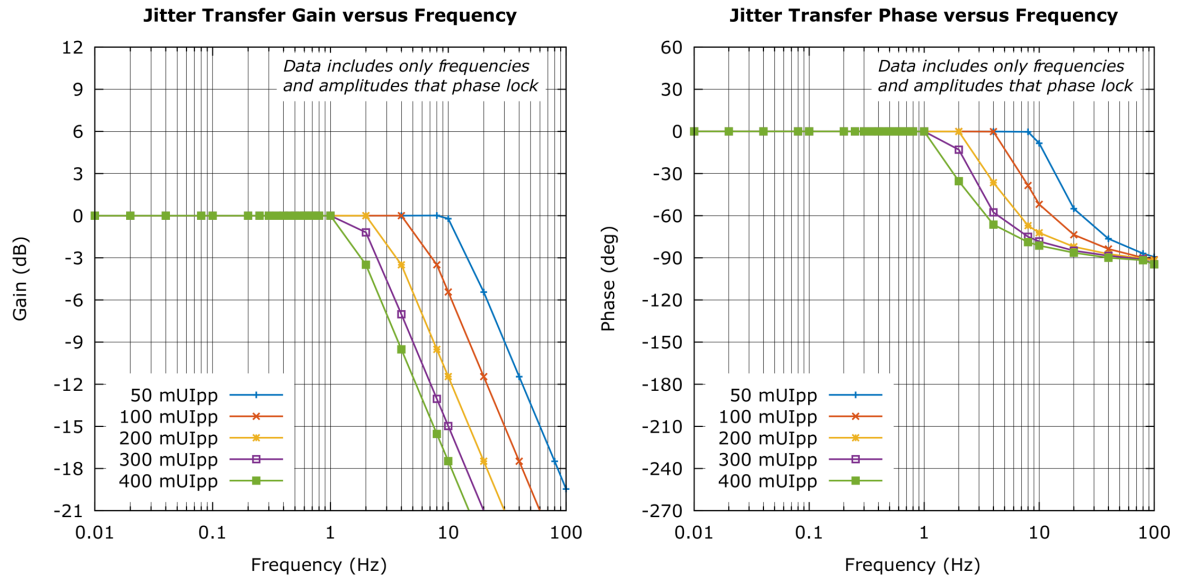


Figure 60 plots the jitter transfer -3 dB bandwidth and damping factor as a function of jitter amplitude. The latter remains relatively constant with amplitude, while the -3 dB bandwidth is inversely proportional to amplitude with the approximate relationship provided in Equation [1].

$$\text{-3 dB bandwidth (Hz)} \approx 760/A_j$$

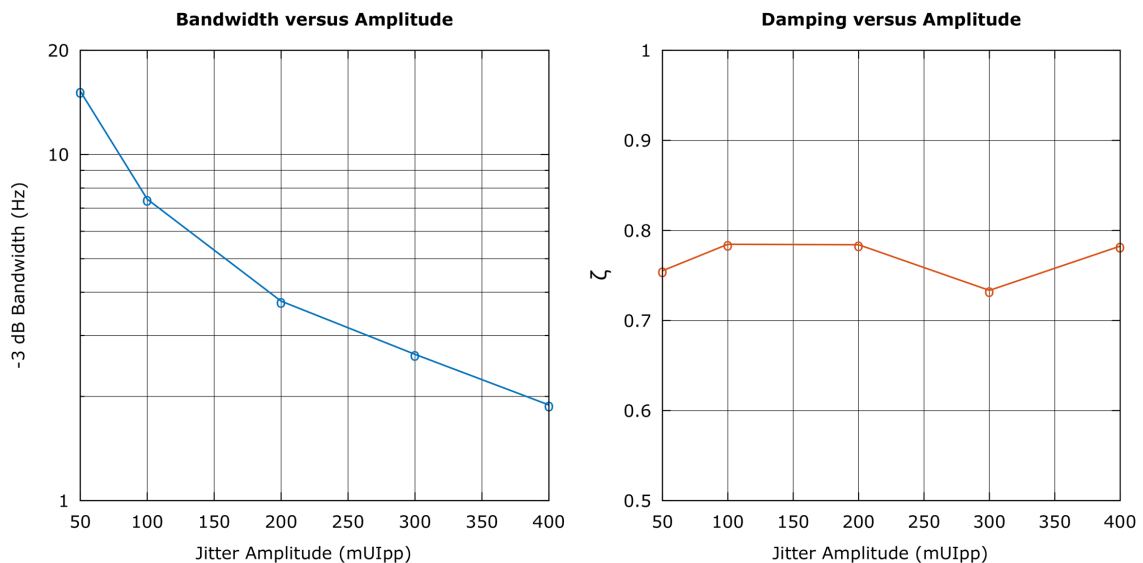
where:

A_j = input jitter amplitude in mUIpp

Eq. [1]

Figure 60

Impact of Jitter Amplitude on Bandwidth and Damping of Windowed Phase Detector Based Phase-locked Loop
1.5440 MHz VCO, Kvco = 154.40 Hz/V, feedback divider = 193
vs1 = 3.3 V, rs1 = 10.0 kohms, vs2 = 3.3 V, rs2 = 10.0 kohms, tw = 250.0 nsec (2.0 mUI)
VDD = 3.3 V, R1 = 392.0 kohms, RZERO = 12.0 kohms, C1 = 9.4 nF, C2 = 4.7 nF, Lock detect τ = 10.0 msec



Analyzing the log files for each jitter transfer simulation provides a view of the relative time for the two phase detector modes, DFF and slew-rate, for each jitter frequency and amplitude. Figure 61 illustrates the relative time that the windowed phase detector is in its slew-rate mode for each jitter amplitude and frequency, and Figure 62 compares the relative times for both the slew-rate mode (solid curves) and high gain DFF mode (dotted curves). For low frequencies, the phase-locked loop tracks the applied jitter and amplitudes, and the relative phase is always less than t_w . However, with increasing frequencies and amplitudes, the relative phase exceeds t_w in magnitude and the slew-rate mode becomes active. When the phase-locked loop falls out of frequency lock, the two modes occur periodically at a rate determined by the frequency difference between the input clock and the divided VCO clock.

Figure 61

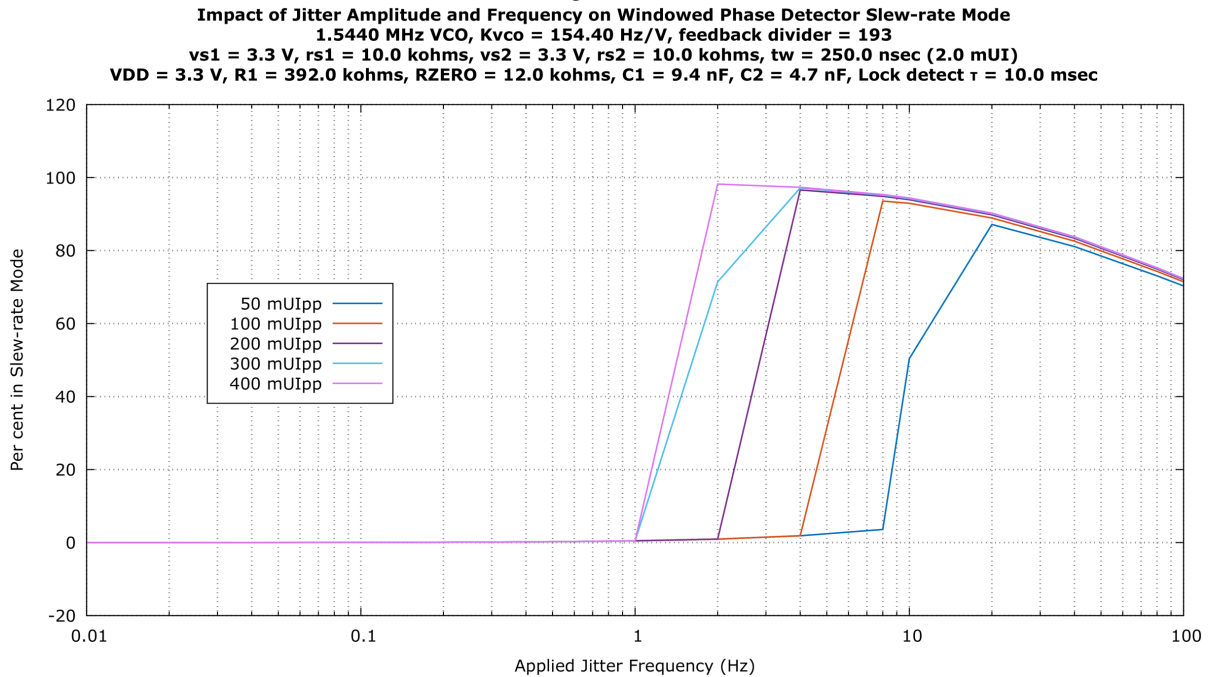
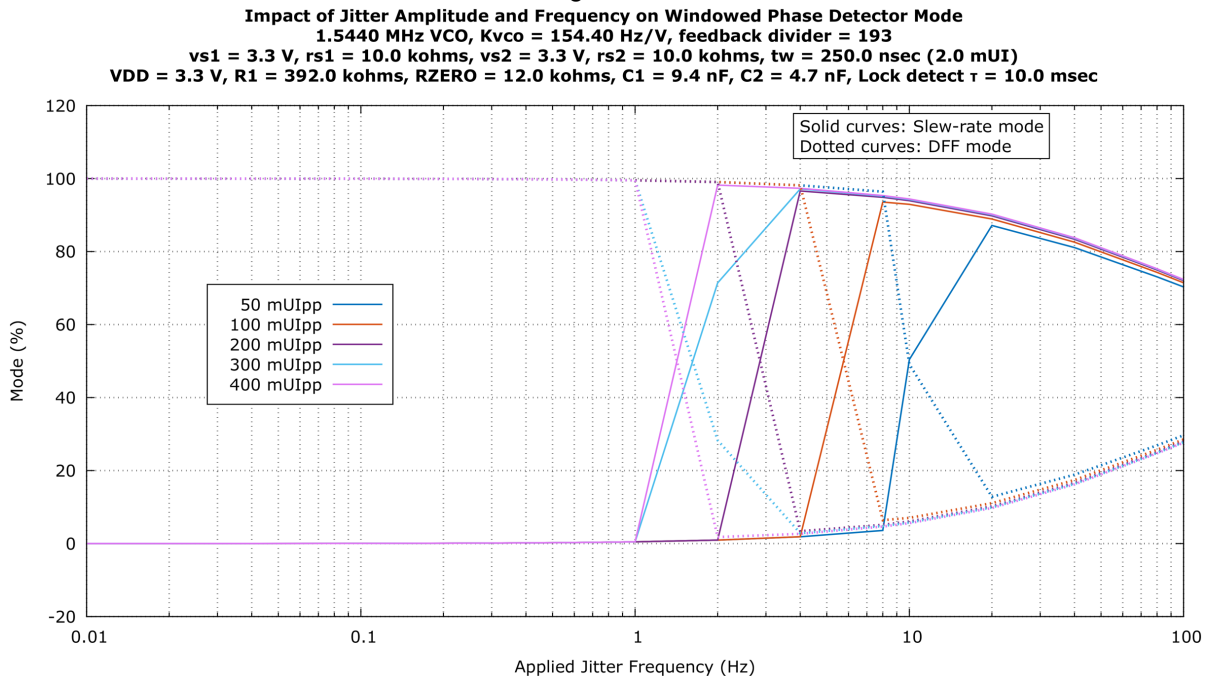


Figure 62



9. Comparison of *trans_pll* Simulation Result with Measured Data

This section documents the correlation between *trans_pll* simulation results and measured data for a phase-locked loop with a non-linear phase detector.

The phase-locked loop in simulation and measurement is driven by an 8 kHz reference clock. The loop is composed of a DFF phase detector, a 1.544 MHz VCXO with a gain of 100 ppm/V, a loop filter, and a feedback divider of 193. The supply voltage is 3.30 V and the center voltage of the VCXO is about 1350 mV. The output signal of the DFF phase detector is a voltage and the loop filter is composed of a series resistor followed by a capacitor C2 in parallel with a series resistor R1 and capacitor C1. In both the measurement and in simulation, the capacitors are initialized to zero prior to initiating acquisition. In the simulation, the DFF phase detector parameters *tauff*, *tauh*, and *taucp_min* are 1 ns, 250 ps, and 100 ps respectively.

A comparison of the control voltage during acquisition of the measured and simulated results is provided in Figure 63. Also provided in Figure 63 is a curve fit to the simulated result with its equation and coefficients.

The *trans_pll* simulated and measured control voltage correlate well as the two waveform envelopes and periodicities are nearly identical. A difference in the two is evident between 35 and 45 seconds, and this is assumed to be a result of the phase noise in the measured feedback clock which is not included in the feedback clock of the simulation. Even a small amount of phase noise in the feedback clock will cause a limit cycle like behavior in the control voltage evident in the measured control voltage as it is amplified by the non-linear DFF phase detector.

To further validate *trans_pll* simulation results, a set of jitter transfer simulations were performed using an input jitter amplitude between 50 mUIpp and 200 mUIpp and between 10 mHz and 100 Hz. To expedite the simulations, the 1.544 MHz VCXO, and its feedback divider of 193 was replaced with an 8 kHz VCXO and a feedback divider of 1. The gain and center voltage of the 8 kHz VCXO are identical to the 1.544 MHz VCXO (100 ppm/V and 1350 mV). The jitter transfer simulation results are provided in Figure 64. Also included in Figure 64 as solid curves are the second-order curve fits to the simulated jitter transfer functions. Note, however, that the curve fits for jitter amplitudes of 50 mUIpp and 100 mUIpp do not capture the simulated jitter transfer characteristic well for jitter frequency between 0.10 and 1.0 Hz. There is a better correlation between the curve fit to the 200 mUIpp transfer function and its simulated data than for the 50 mUIpp and 100 mUIpp cases.

To study this result further, the 200 mUI step response was simulated using *trans_pll* and compared to the pulse response derived from the inverse Laplace transform of the 200 mUIpp second-order curve fit to its transfer function. The two normalized step responses are compared in Figure 65. Although the periodicity of the two responses are similar, the magnitude of the excursions of the *trans_pll* step response far exceed that of the transfer function derived response. Intuitively, this result is consistent with the non-linear DFF phase detector which is not modeled by the linearized transfer function derived response. With the linearized response, the open loop gain of the phase detector and phase-locked loop remain constant with jitter amplitude. However, in the *trans_pll* simulation, the open loop gain of the phase detector and phase-locked loop are reduced considerably as the relative phase of the input clock and feedback clock deviates from zero. Therefore, the loop gain of the phase-locked loop in the *trans_pll* simulation is periodically reduced and is hypothesized as the reason for the difference between the envelopes of the two phase step responses.

To validate this hypothesis, parameters for a phase-frequency detector based phase-locked loop were chosen to provide a jitter transfer function that is similar to the second-order curve fit to the 200 mUIpp amplitude DFF phase detector based phase-locked loop. The small-signal transfer function of the phase-frequency detector based phase-locked loop is compared to the 200 mUI non-linear DFF phase detector based phase-locked loop in Figure 66. The 200 mUI step response of the resulting phase-frequency detector based phase-locked loop was simulated using *trans_pll* and Figure 67 compares its pulse response to the 200 mUI step response of the non-linear DFF phase detector based phase-locked loop and the pulse response derived from its jitter transfer function. The step responses for the phase-frequency detector based phase-locked loop and the pulse response derived from the inverse Laplace Transform of the jitter transfer of the 200 mUIpp DFF phase detector based phase-locked loop are nearly identical. Hence, the use of a linear phase detector in lieu of the non-linear DFF based phase detector eliminates the larger magnitude of phase excursions as hypothesized.

Figure 63

Measured and Simulated DFF Phase Detector based Phase-locked Loop Control Voltage and Curve Fit

$$v_c(t) = 1355 + (0 - 1355) \cdot \exp(-t/3.0085) \text{ mV}$$

8.0000 kHz reference frequency, 1.5440 MHz VCO, $K_{vco} = 100.0 \text{ ppm/V}$
loop divider = 193, RZERO = 1.2 kohms, R1 = 392.0 kohms
C1 = 9.4 uF, C2 = 4.7 uF, initial control voltage = 0.0 V, $\text{deltat} = 1.0 \text{ ns}$

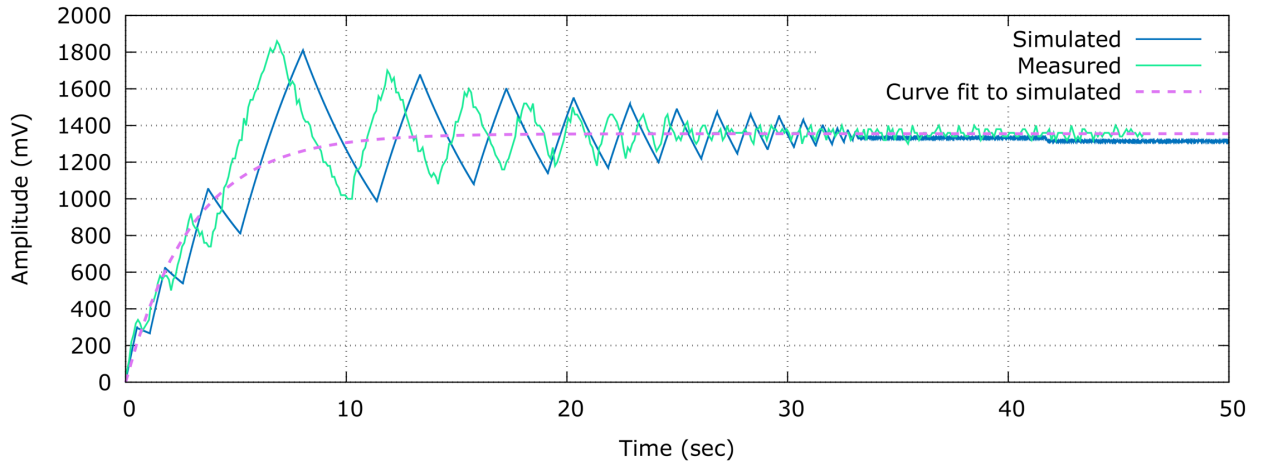


Figure 64

Impact of Jitter Amplitude on Jitter Transfer of DFF Phase Detector Based Phase-locked Loop
8.0000 kHz VCO, $K_{vco} = 800.00 \text{ mV}$, feedback divider = 1, Lock detect $\tau = 3.0 \text{ sec}$
VDD = 3.3 V, R1 = 392.0 kohms, RZERO = 1.2 kohms, C1 = 9.4 uF, C2 = 4.7 uF

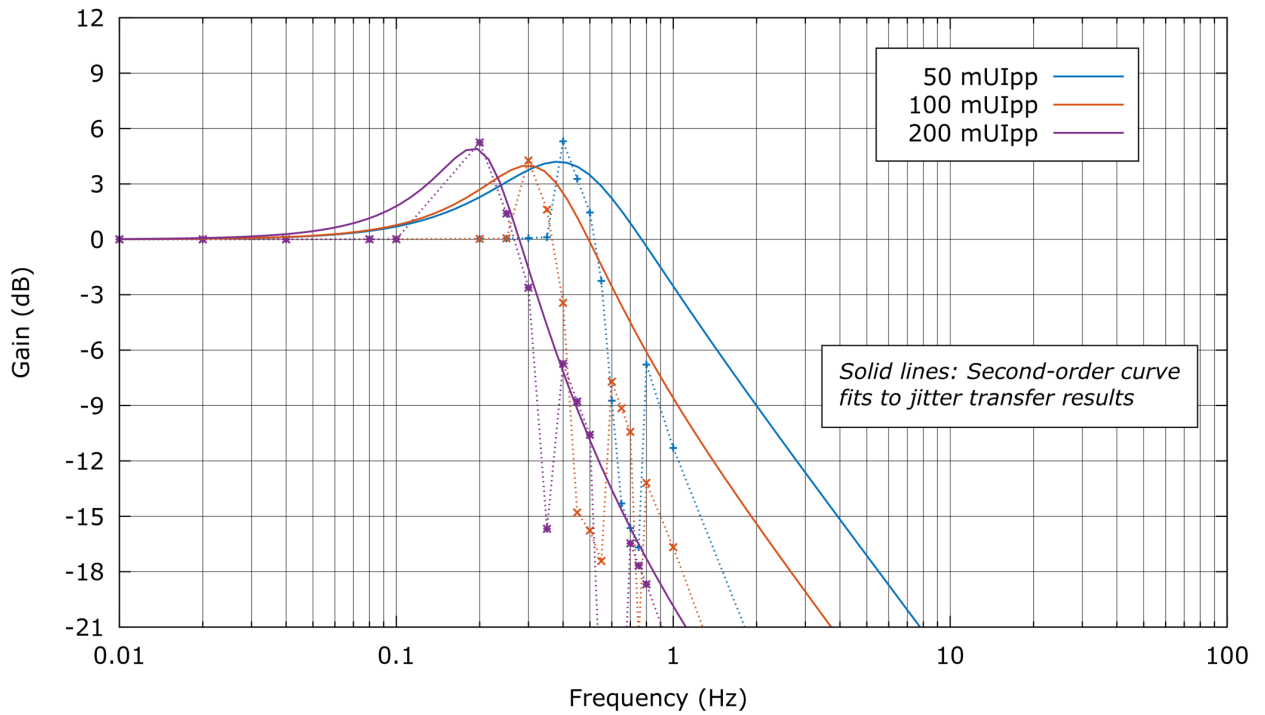


Figure 65

Pulse Response of dff Based Phase-locked Loop to 200 mUI Phase Step
8.0 kHz reference frequency, 8.0 kHz VCO, Kvco = 100.0 ppm/V
loop divider = 1, RZERO = 1.20 kohms, R1 = 392.00 kohms
C1 = 9.4 uF, C2 = 4.7 uF, initial control voltage = 1.4 V

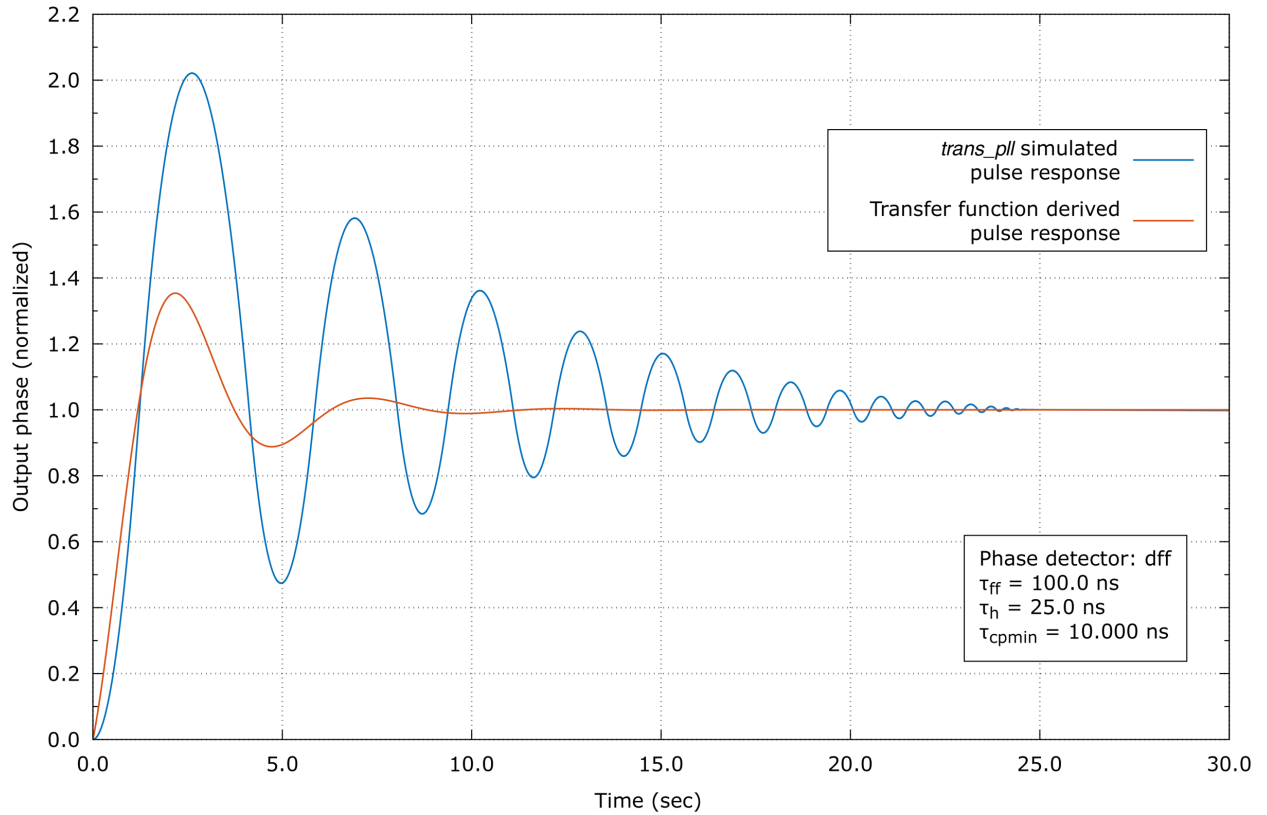


Figure 66

Analog Charge Pump Phase-frequency Detector Based Phase-Locked Loop Closed Loop Transfer Function

Icp = 17.0 uA, Rzero = 110.00 Kohm, C1 = 9.6 uF, C3 = 2.80 uF

VCO frequency = 8000 Hz, Kvco = 0.8 Hz/V, vco modulation bandwidth = 1.6 MHz, divider = 1

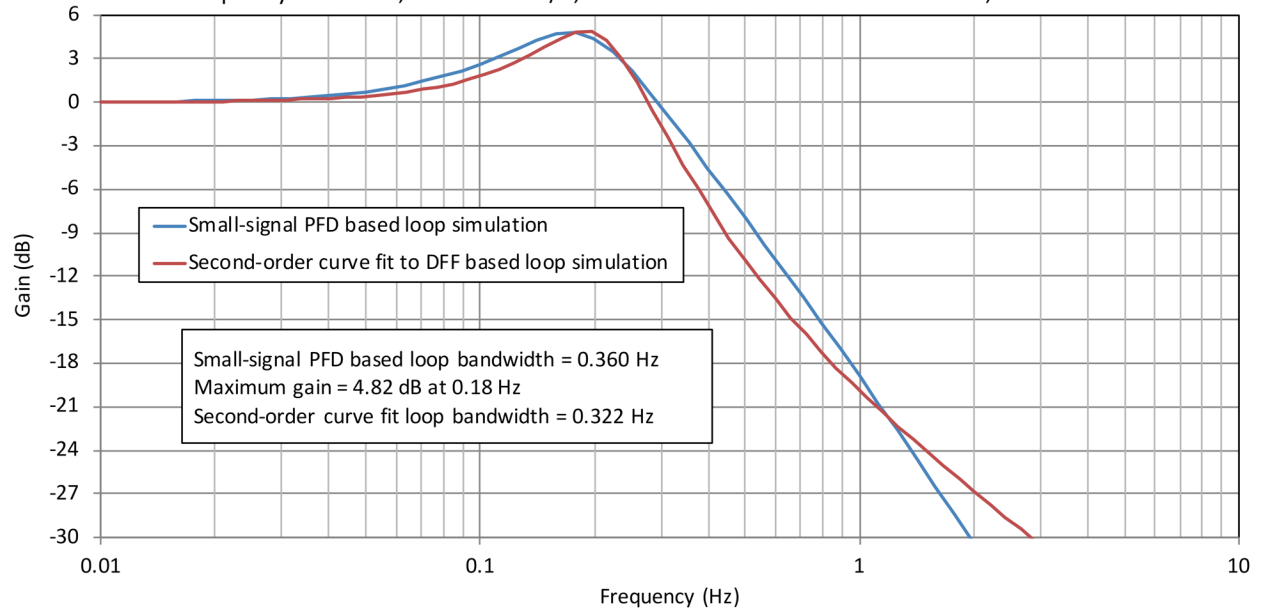
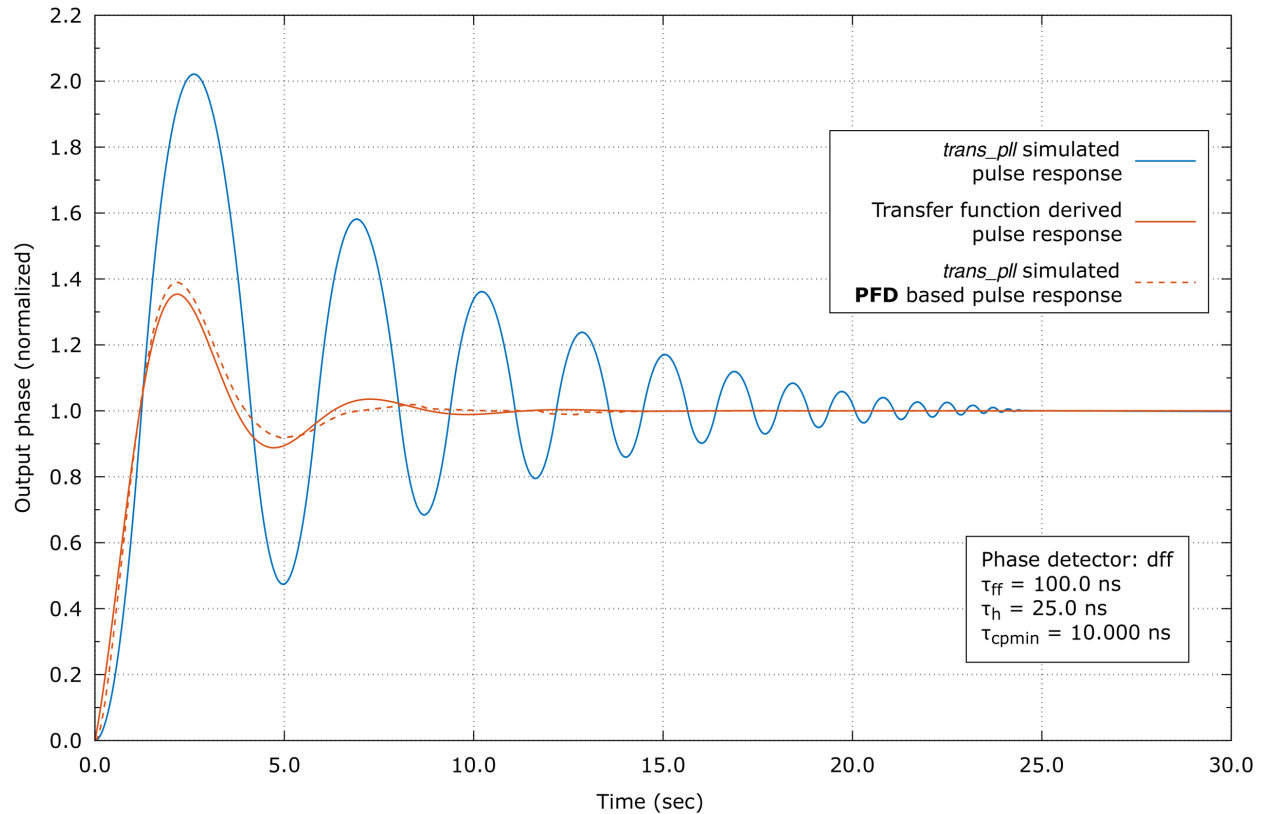


Figure 67

Pulse Response of dff Based Phase-locked Loop to 200 mUI Phase Step
8.0 kHz reference frequency, 8.0 kHz VCO, $K_{vco} = 100.0$ ppm/V
loop divider = 1, RZERO = 1.20 kohms, R1 = 392.00 kohms
C1 = 9.4 uF, C2 = 4.7 uF, initial control voltage = 1.4 V



10. Summary

A description of the development history, use, and examples of a C based executable, *trans_pll*, that simulates the performance of large-signal phase-locked loops was presented. Its various capabilities and options were outlined through a set of representative simulations. An example of its correlation with measured data was also included.

The program and its documentation are available for download at the URL provided in reference [4] or [5]. Installation instructions are provided in Appendix I.

Comments or questions are welcomed by the author.

Shawn Logan

Version History

v1.0 Initial release 8/5/2026

Appendix I Installation Instructions

Version 1.76 of the program is available in the compressed tar file “trans_pll_1p76_071826.tar.gz” and may be downloaded from the URL provided in reference [4] or [5]. To install the program and create the executable “*trans_pll*”, enter Command line [1] in the directory in which you wish to locate the program.

```
$ tar -xvzf trans_pll_1p76_071826.tar.gz      Command line [1]
```

This will create a directory trans_pll_v1p76_071826 and extract its directory structure:

Documentation/ README.txt	examples/ include/	plotting_routines/ src/
------------------------------	-----------------------	----------------------------

Navigate to the “src” subdirectory, and issue the following two UNIX commands:

\$ make	Command line [2]
\$ make clean	Command line [3]

Issuing these two commands will create the executable “*trans_pll*” and delete object files no longer needed. In addition, Command line [2] will also attempt to create a symbolic link to *trans_pll* in your \$HOME/bin directory if this directory exists. Assuming your \$HOME/bin directory is contained in your executable search path (UNIX PATH variable), this will allow you to execute *trans_pll* from any of your subdirectories using Command line [4] as documented in the program manual in the Documentation directory.

```
$ trans_pll <input_ASCII_filename>      Command line [4]
```

References

- [1] <http://www.gnuplot.info/>
- [2] John W. Eaton, David Bateman, Søren Hauberg, Rik Wehbring (2023). GNU Octave version 8.2.0 manual: a high-level interactive language for numerical computations.
URL <https://www.gnu.org/software/octave/doc/v8.2.0/>
- [3] https://github.com/shawnmlogan/psd_sppowr
- [4] https://github.com/shawnmlogan/trans_pll
- [5] https://1drv.ms/u/c/12fa6404c01a3e73/IQCSM5HPINUSTaSnICh-OcdkAbwG6d6jKG6EESzn2kn_UxU?e=aOmPQT